ARMENU

TIZIM ARXITEKTURASI

II — Ma'lumot va masshtab

Modulli monolit · Multi-tenant · 1 mln foydalanuvchiga tayyor
Django 5 · PostgreSQL+pgvector · Redis · 8 AI provayder (MCP)

Qismlar: 9 ta
Hajm: 19 594 qator · 900 KB
ADR: 15 ta arxitektura qarori
Kod namunalari: 944 blok
Jadvallar: 2 969 qator
Versiya: 1.0 · 2026-09-04

✔ **Kim uchun:** tizim arxitektori · texnik direktor · investor (texnik due diligence) · dasturchi.
Har qaror sababi bilan yozilgan: nega shunday, muqobili nima edi, qachon qayta ko'riladi.

MUNDARIJA — II — Ma'lumot va masshtab

3. **Ma'lumot arxitekturasi**

   31 jadval ER, indeks strategiyasi selektivlik hisobi bilan, partitioning SQL+cron, sharding rejasi, 4 qatlamli kesh, menu_version naqshi, zero-downtime migratsiya, S3 tuzilishi

4. **Masshtablash**

   S0→S5 bosqichlari, matematik sig'im hisoblari, 1 mln bir vaqtda to'liq tahlil (WS 1mln→90k), 14 bo'g'iz nuqtasi, auto-scaling, ishlash byudjeti, k6 yuklama testlari

5. **Ishonchlilik**

   SLO va error budget, FMEA, 15+ yiqilish ssenariysi, graceful degradation zanjirlari, circuit breaker, retry, bulkhead, self-healing 15 mexanizm, chaos engineering, falokatdan tiklash

QISM 3

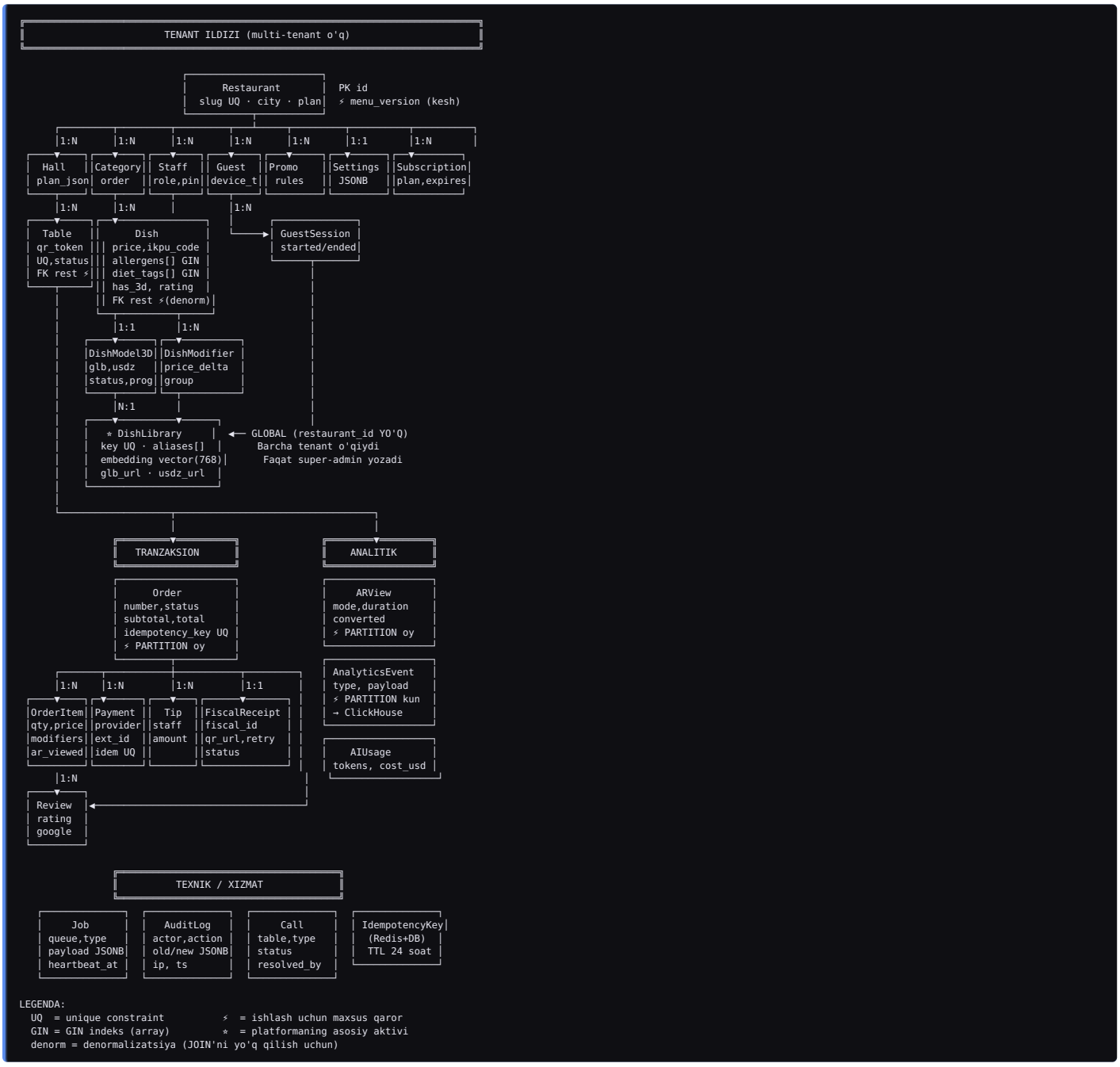
Ma'lumot arxitekturasi

Versiya: 1.0 · **Sana:** 2026-09-08
Kim uchun: DBA · backend arxitektor · dasturchi · SRE
Oldingi: `02_KOMPONENTLAR.md` · **Keyingi:** `04_MASSHTABLASH.md`

> Bu hujjat **ma'lumot qatlamining yagona haqiqat manbai**. Modellar ta'rifi `TZ_MASTER/04_DATA_MODEL_VA_API.md` da; bu yerda —**fizik dizayn**: hajm, indeks, partition, sharding, kesh, hayotiy sikl, migratsiya, butunlik.

---

1. ER DIAGRAMMA — TO'LIQ SXEMA



1.1 Bog'lanish kardinalliklari

Ota	→	Bola	Kardinallik	on_delete	Sabab
Restaurant	→	Hall	1:N	CASCADE	Restoran o'chsa zal ma'nosiz
Hall	→	Table	1:N	CASCADE	—
Restaurant	→	Table	1:N	CASCADE	⚡ **denorm FK** — `hall` orqali JOIN'siz filtr
Restaurant	→	Category	1:N	CASCADE	—
Category	→	Dish	1:N	CASCADE	—
Restaurant	→	Dish	1:N	CASCADE	⚡ **denorm FK** — menyu so'rovi 1 JOIN kam
Dish	→	DishModel3D	1:1	CASCADE	Taom o'chsa model kerak emas
DishLibrary	→	DishModel3D	1:N	**SET_NULL**	⚡ Kutubxona yozuvi o'chsa restoran modeli qolsin
Restaurant	→	Order	1:N	**PROTECT**	🟠 Moliyaviy hujjat — hech qachon kaskad o'chirilmaydi
Table	→	Order	1:N	SET_NULL	Stol o'chsa buyurtma tarixi qolsin
Order	→	OrderItem	1:N	CASCADE	Item mustaqil ma'noga ega emas
Order	→	Payment	1:N	PROTECT	🟠 To'lov yozuvi buzilmaydi
Order	→	FiscalReceipt	1:1	PROTECT	🟠 Soliq hujjati
Guest	→	Order	1:N	SET_NULL	🟡 GDPR — mehmon o'chsa buyurtma anonimlashadi

---

2. JADVALLAR TASNIFI — 4 SINF

Har jadval **aniq bir sinfga** tegishli. Sinf → indeks, partition, backup, o'chirish siyosatini belgilaydi.

```
┌──────────────────────────────────────────────────────┐
│  A. TENANT-SCOPED (restaurant_id BOR) — 19 jadval    │
│     🔴 Qoida: HAR so'rovda restaurant_id filtri MAJBURIY │
│     Backup: to'liq · Sharding kaliti: restaurant_id  │
├──────────────────────────────────────────────────────┤
│  B. GLOBAL (restaurant_id YO'Q) — 3 jadval           │
│     🟢 Barcha tenant o'qiydi, faqat super-admin yozadi │
│     Backup: to'liq · Sharding: REPLIKATSIYA (har shardda nusxa) │
├──────────────────────────────────────────────────────┤
│  C. ANALITIK (ko'p yoziladi, kam o'qiladi) — 4 jadval │
│     🔴 Partition MAJBURIY · Arxiv S3 · PITR shart emas │
├──────────────────────────────────────────────────────┤
│  D. TEXNIK (tizim ishlashi uchun) — 5 jadval         │
│     ○ Qisqa umr · Avtomatik tozalanadi               │
└──────────────────────────────────────────────────────┘
```

2.1 To'liq tasnif jadvali

Jadval	Sinf	tenant kaliti	Partition	Backup PITR	Auto-tozalash
`restaurants_restaurant`	A	`id` (o'zi)	✗	✓	✗
`tables_hall`	A	`restaurant_id`	✗	✓	✗
`tables_table`	A	`restaurant_id`	✗	✓	✗
`menu_category`	A	`restaurant_id`	✗	✓	✗
`menu_dish`	A	`restaurant_id`	✗	✓	✗
`menu_dishmodifier`	A	`dish→restaurant`	✗	✓	✗
`ar3d_dishmodel3d`	A	`dish→restaurant`	✗	✓	✗
`accounts_staff`	A	`restaurant_id`	✗	✓	✗
`guest_guest`	A	`restaurant_id`	✗	✓	◍ 2 yil passiv
`guest_guestsession`	A	`guest→restaurant`	✗	✓	◍ 1 yil
`orders_order`	A	`restaurant_id`	⚡ **oy (RANGE)**	✓	✗ (arxiv)
`orders_orderitem`	A	`order→restaurant`	⚡ oy (order bilan)	✓	✗
`payments_payment`	A	`order→restaurant`	✗	✓ ●	✗
`payments_tip`	A	`order→restaurant`	✗	✓	✗
`fiscal_fiscalreceipt`	A	`order→restaurant`	✗	✓ ●	✗
`orders_call`	A	`table→restaurant`	✗	○	✓ 90 kun
`reviews_review`	A	`restaurant_id`	✗	✓	✗
`billing_subscription`	A	`restaurant_id`	✗	✓ ●	✗
`tenants_settings`	A	`restaurant_id`	✗	✓	✗
`ar3d_dishlibrary` ★	**B**	—	✗	✓ ●	✗
`menu_ikpu_catalog`	B	—	✗	✓	✗
`i18n_translation_cache`	B	—	✗	○	✓ 180 kun
`ar3d_arview`	C	`restaurant_id`	⚡ **oy (RANGE)**	○	✓ 13 oy → S3
`analytics_analyticsevent`	C	`restaurant_id`	⚡ **kun (RANGE)**	○	✓ 90 kun → CH
`ai_aiusage`	C	`restaurant_id`	⚡ oy	✓	✓ 24 oy
`audit_auditlog`	C	`restaurant_id`	⚡ oy	✓ ●	✓ 5 yil (qonun)
`jobs_job`	D	payload ichida	✗	○	✓ 7 kun (done)
`core_idempotencykey`	D	—	✗	○	✓ 24 soat
`core_ratelimit`	D	—	✗ (Redis)	✗	✓ TTL
`core_featureflag`	D	—	✗	✓	✗
`core_healthcheck`	D	—	✗	✗	✓ 24 soat

Belgilar: ● kritik (moliyaviy/huquqiy) · ✓ standart · ○ backup shart emas · ◍ shartli

2.2 Nima uchun `DishLibrary` global

```
✗ AGAR har restoranga nusxa qilinsa:
   5 000 restoran × 60 taom × (GLB 2 MB + USDZ 3 MB) = 1 500 000 MB = 1,5 TB
   S3 narxi: 1,5 TB × $0,023 = $34,5/oy  +  CDN trafik ~$900/oy

✓ GLOBAL kutubxona (havola, nusxa emas):
   500 noyob taom × 5 MB = 2,5 GB
   S3 narxi: $0,06/oy  +  CDN kesh hit rate 99% (bir xil URL!)

   ⚡ TEJASH: storage 600× · CDN xarajati ~30× (bir fayl — million so'rov)
   ⚡ Bu shunchaki tejash emas — bu HIMOYA DEVORI (moat):
     raqobatchi 500 ta 3D modelni qaytadan yaratishi kerak.
```

3. HAJM VA O'SISH PROGNOZI (1 000 RESTORAN)

3.1 Asosiy taxminlar (hisob bazasi)

Parametr	Qiymat	Izoh
Restoran soni	1 000	S3 bosqichi
O'rtacha stol / restoran	25	kichik 10, o'rta 25, katta 80
O'rtacha taom / restoran	60	menyu hajmi
O'rtacha kategoriya	8	—
Xodim / restoran	8	ofitsiant 5, oshpaz 2, admin 1
Stol aylanmasi / kun	4	bir stol kuniga 4 marta band
Buyurtma / restoran / kun	25 × 4 = **100**	—
Taom / buyurtma	3,5	OrderItem koeffitsienti
AR ko'rish / buyurtma	2,2	3D bor taomlar uchun
Analitik event / sessiya	12	sahifa, skroll, bosish, qidiruv

3.2 Jadval hajmlari — hisob bilan

Jadval	Formula	Boshlang'ich qator	Qator/kun	1 yildan keyin	Qator hajmi	Jami (1 yil)
`restaurant`	1 000	1 000	+3	2 100	~2 KB	**4 MB**
`hall`	1000×2	2 000	+6	4 200	~1,5 KB	6 MB
`table`	1000×25	25 000	+75	52 000	~250 B	13 MB
`category`	1000×8	8 000	+24	16 800	~400 B	7 MB
`dish`	1000×60	60 000	+200	133 000	~2,5 KB	**330 MB**
`dishmodifier`	dish×2,5	150 000	+500	332 000	~300 B	100 MB
`dishmodel3d`	dish×0,35	21 000	+70	46 000	~600 B	28 MB
`staff`	1000×8	8 000	+30	19 000	~500 B	10 MB
`guest`	—	0	+18 000	6 570 000	~350 B	**2,3 GB**
`guestsession`	—	0	+100 000	36 500 000	~150 B	**5,5 GB**
`order`	—	0	**+100 000**	**36 500 000**	~450 B	**16,4 GB**
`orderitem`	order×3,5	0	**+350 000**	**127 750 000**	~200 B	**25,6 GB**
`payment`	order×0,7	0	+70 000	25 550 000	~300 B	7,7 GB
`fiscalreceipt`	order×0,7	0	+70 000	25 550 000	~400 B	10,2 GB
`call`	—	0	+12 000	(90 kun) 1 080 000	~150 B	162 MB
`review`	order×0,08	0	+8 000	2 920 000	~600 B	1,8 GB
`arview`	—	0	**+220 000**	**80 300 000**	~180 B	**14,5 GB**
`analyticsevent`	—	0	**+1 200 000**	(90 kun) 108 000 000	~250 B	**27 GB**
`auditlog`	—	0	+60 000	21 900 000	~500 B	11 GB
`aiusage`	—	0	+5 000	1 825 000	~250 B	456 MB
`job`	—	0	+40 000	(7 kun) 280 000	~1 KB	280 MB
`dishlibrary` *	—	500	+2	1 230	~4 KB	5 MB

JAMI ma'lumot (1 yil, 1 000 restoran):

```
Tranzaksion (A sinf)          ≈  70 GB
Analitik   (C sinf)           ≈  53 GB   → ClickHouse'ga ko'chiriladi
Texnik     (D sinf)           ≈   0,3 GB
──────────────────────────────────────
XOM MA'LUMOT                  ≈ 123 GB
+ Indekslar (~45% jadval hajmi) ≈  55 GB
+ WAL + bloat rezervi (25%)     ≈  45 GB
══════════════════════════════════════
DISK TALABI (1 yil)           ≈ 223 GB   → 500 GB NVMe olinadi (2× rezerv)
```

3.3 Issiq / iliq / sovuq taqsimoti

> **Issiq** = kuniga bir necha marta o'qiladi, RAM'da (shared_buffers) turishi kerak.
> **Iliq** = haftada o'qiladi, disk (NVMe) yetarli.
> **Sovuq** = deyarli o'qilmaydi, arxivga.

Jadval	♨ Issiq	🌡 Iliq	🧊 Sovuq	Issiq ulush	Izoh
`restaurant`	butun	—	—	100%	4 MB — doim RAM'da
`dish`	butun	—	—	100%	330 MB — menyu asosi, doim RAM
`dishmodifier`	butun	—	—	100%	100 MB
`table`	butun	—	—	100%	13 MB — QR skanerlash
`dishlibrary` *	butun	—	—	100%	5 MB
`order`	oxirgi 2 kun	30 kun	>30 kun	**0,5%**	200k qator issiq (~90 MB)
`orderitem`	oxirgi 2 kun	30 kun	>30 kun	**0,5%**	700k qator (~140 MB)
`payment`	oxirgi 7 kun	90 kun	>90 kun	2%	sverka
`fiscalreceipt`	oxirgi 1 kun	30 kun	>30 kun	0,3%	retry uchun
`guest`	faol 30 kun	1 yil	>1 yil	8%	qayta tashrif
`arview`	joriy oy	3 oy	>3 oy	8%	dashboard
`analyticsevent`	joriy kun	7 kun	>7 kun	1%	⚡ ClickHouse'da
`auditlog`	—	30 kun	>30 kun	0%	faqat tergovda

⚡ Muhim xulosa:

```
Issiq ma'lumot jami ≈ 330 (dish) + 100 (mod) + 13 (table) + 4 (rest)
                    + 90 (order 2kun) + 140 (orderitem) + 200 (guest faol)
                    + 5 (library)  ≈  882 MB
Indeks (issiq qismi)               ≈  600 MB
──────────────────────────────────────────
ISSIQ ISHCHI TO'PLAM               ≈  1,5 GB

→ PostgreSQL shared_buffers = 8 GB bo'lsa — BUTUN ISSIQ TO'PLAM RAM'DA ✅
→ Disk o'qish faqat tarixiy hisobotlarda bo'ladi
→ Shuning uchun 1 000 restoranda ham DB 8 CPU / 32 GB da yashaydi
```

---

4. INDEKS STRATEGIYASI

4.1 Indeks tanlash qoidalari (5 ta qat'iy)

#	Qoida	Sabab
1	**Har tenant jadvalida `restaurant_id` birinchi ustun**	Barcha so'rov shu bilan boshlanadi → prefiks scan
2	Composite tartibi: **tenglik → diapazon → tartib**	B-tree faqat shu tartibda to'liq ishlaydi
3	**Selektivlik < 5% bo'lsa indeks qo'yilmaydi**	Planner baribir Seq Scan tanlaydi
4	Array ustunga **GIN**, matn qidiruvga **trgm**, vektorga **HNSW**	B-tree bu tiplarda ishlamaydi
5	**Partial indeks** — filtr doim bir xil bo'lsa	Hajm 10-50× kichik

4.2 To'liq indeks jadvali

`menu_dish` — eng ko'p o'qiladigan jadval

#	Indeks	Tur	Qaysi so'rov	Selektivlik	Hajm (133k qator)
D1	`(restaurant_id, is_available, "order")`	B-tree	Menyu qurish (asosiy)	1/1000 = 0,1% 🟢	4,5 MB
D2	`(category_id, "order")`	B-tree	Kategoriya bo'yicha	1/8000 🟢	3,8 MB
D3	`(restaurant_id) WHERE has_3d`	B-tree partial	AR bor taomlar	35% dan 0,035% 🟢	0,5 MB
D4	`diet_tags`	**GIN**	`diet_tags @> ARRAY['vegan']`	5-15% 🟡	2,1 MB
D5	`allergens`	**GIN**	`NOT allergens && ARRAY['nuts']`	10-20% 🟡	2,3 MB
D6	`(name_uz gin_trgm_ops)`	**GIN trgm**	Qidiruv `ILIKE '%osh%'`	o'zgaruvchan	12 MB
D7	`ikpu_code`	B-tree	Fiskal sverka	0,5% 🟢	2,8 MB

```
-- D1: ASOSIY INDEKS — menyu so'rovining 100%i shu orqali
CREATE INDEX idx_dish_menu ON menu_dish (restaurant_id, is_available, "order")
    INCLUDE (category_id, price, has_3d);
-- ⚡ INCLUDE → INDEX-ONLY SCAN: jadvalga umuman murojaat qilinmaydi
--    Natija: 60 qator o'qish = 1 indeks bloki = ~0,15 ms

-- D3: PARTIAL — faqat 3D bor taomlar (35%)
CREATE INDEX idx_dish_has3d ON menu_dish (restaurant_id)
    WHERE has_3d = true AND is_available = true;
-- Hajm: to'liq indeksdan 3× kichik, AR sahifasi uchun ideal

-- D6: TRGM — qidiruv (uch tilli)
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX idx_dish_search ON menu_dish
    USING GIN ((name_uz || ' ' || name_ru || ' ' || name_en) gin_trgm_ops);
-- ⚠ Faqat >100k qatorda foyda beradi; kichik restoranda Seq Scan tezroq
```

Composite tartibi — nega aynan `(restaurant_id, is_available, "order")`:

```
So'rov:  WHERE restaurant_id = 42 AND is_available = true ORDER BY "order"
         └─ tenglik ─┘         └─ tenglik ─┘            └ tartib ┘

✓ (restaurant_id, is_available, "order")
   → B-tree: 42 ni topadi → true ni topadi → "order" allaqachon saralangan
   → Sort qadami YO'Q. Cost ≈ 8

× ("order", restaurant_id, is_available)
   → "order" bo'yicha butun jadvalni kezadi, keyin filtrlaydi
   → Cost ≈ 2400 (300× yomon)

× (restaurant_id, "order", is_available)
   → is_available filtri indeksdan KEYIN qo'llanadi (Filter, Index Cond emas)
   → 60 o'rniga 95 qator o'qiladi + qo'shimcha filtr
```

`orders_order` — yozish og'ir, o'qish ko'p

#	Indeks	Tur	So'rov	Selektivlik	Izoh
O1	`(restaurant_id, status, created_at DESC)`	B-tree	KDS/ofitsiant paneli	0,01% 🟢	Har partitionda
O2	`(restaurant_id, created_at DESC)`	B-tree	Hisobotlar	0,1% 🟢	—
O3	`(table_id, status)`	B-tree	Stol holati	0,004% 🟢	—
O4	`idempotency_key`	B-tree **UNIQUE**	Dublikat bloki	1 qator 🟢	🔴 kritik
O5	`(restaurant_id, number, created_at::date)`	B-tree UNIQUE	Kunlik raqam	1 qator 🟢	—
O6	`(restaurant_id) WHERE status IN ('new','accepted','cooking')`	partial	⚡ **Faol buyurtmalar**	0,05% 🟢	Eng issiq

```
-- O6: ENG MUHIM PARTIAL INDEKS
CREATE INDEX idx_order_active ON orders_order (restaurant_id, created_at)
    WHERE status IN ('new', 'accepted', 'cooking', 'ready');

-- Hisob: 1 000 restoran × 36,5 mln qator, lekin faol — bir vaqtda ~5 000 ta
-- To'liq indeks:   36 500 000 qator × 40 B = 1 460 MB
-- Partial indeks:       5 000 qator × 40 B =   0,2 MB  ⚡ 7300× kichik!
-- → Butunlay RAM'da, KDS so'rovi p99 < 2 ms
```

`ar3d_dishlibrary` — pgvector semantik qidiruv

#	Indeks	Tur	So'rov
L1	`key` UNIQUE	B-tree	To'g'ridan-to'g'ri olish
L2	`(category_key, is_active)`	B-tree	Kategoriya ko'rish
L3	`aliases`	**GIN**	Nom bo'yicha moslash
L4	`embedding`	**HNSW**	⚡ Semantik qidiruv
L5	`(-used_by_count)`	B-tree	Mashhurlar

```
-- L4: HNSW — ANN (approximate nearest neighbour)
CREATE INDEX idx_library_embedding ON ar3d_dishlibrary
    USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);

-- Parametr tanlovi (1 230 qator uchun):
--   m = 16              → har tugun 16 bog'lanish (default, kichik to'plamga yetarli)
--   ef_construction=64  → qurish sifati (yuqori = sekin qurish, aniqroq qidiruv)
--   ef_search = 40      → so'rovda (SET hnsw.ef_search = 40)
--
-- Hajm: 1 230 × 768 × 4 B (float) = 3,8 MB vektor + ~1,2 MB graf = 5 MB
-- Tezlik: IVFFlat 12 ms → HNSW 0,8 ms (15× tez)
-- Aniqlik (recall@10): 0,98
--
-- ⚠ NEGA IVFFlat EMAS: IVFFlat qurishdan oldin ma'lumot bo'lishini talab qiladi
--    va yangi yozuvlarda sifat tushadi. Kutubxona doim o'sadi → HNSW.
```

Moslashtirish so'rovi (yangi taom qo'shilganda):

```
# apps/ar3d/services/library_match.py
def match_library(dish: Dish) -> list[tuple[DishLibrary, float]]:
    """3 bosqichli moslashtirish — arzondan qimmatga."""

    # 1-BOSQICH: aniq alias (GIN, ~0,2 ms, bepul)
    exact = DishLibrary.objects.filter(
        aliases__overlap=[dish.name_uz.lower(), dish.name_ru.lower()],
        is_active=True,
    ).first()
    if exact:
        return [(exact, 1.0)]

    # 2-BOSQICH: trigram o'xshashlik (~1,5 ms, bepul)
    fuzzy = DishLibrary.objects.annotate(
        sim=TrigramSimilarity('name_uz', dish.name_uz)
    ).filter(sim__gt=0.45, is_active=True).order_by('-sim')[:3]
    if fuzzy and fuzzy[0].sim > 0.75:
        return [(x, float(x.sim)) for x in fuzzy]

    # 3-BOSQICH: semantik (HNSW, ~0,8 ms + embedding narxi $0,00002)
    emb = get_embedding(f"{dish.name_uz} {dish.desc_uz[:200]}")   # keshlanadi
    return DishLibrary.objects.filter(is_active=True).annotate(
        dist=CosineDistance('embedding', emb)
    ).order_by('dist')[:5]
```

Qolgan jadvallar — indeks xulosasi

Jadval	Indekslar	Jami hajm
`table`	`qr_token` UQ · `(restaurant_id, status)` · `(restaurant_id, number)` UQ	6 MB
`orderitem`	`(order_id)` · `(dish_id, created_at)` [hisobot]	9 GB
`payment`	`(order_id)` · `idempotency_key` UQ · `(provider, ext_id)` UQ · `(status, created_at) WHERE status='pending'`	2,5 GB
`fiscalreceipt`	`(order_id)` UQ · `(status, retry_count) WHERE status != 'ok'` ⚡	1,1 GB
`guest`	`(restaurant_id, device_token)` UQ · `(restaurant_id, phone)` · `tg_user_id`	1,2 GB
`arview`	`(restaurant_id, created_at)` · `(dish_id, created_at)` · `session_id`	5,8 GB
`job`	`(queue, status, id) WHERE status IN ('pending','running')` ⚡	12 MB
`auditlog`	`(restaurant_id, created_at DESC)` · `(model, object_id)`	4 GB

```
-- Job navbat indeksi — polling har 200 ms da bo'ladi, tez bo'lishi SHART
CREATE INDEX idx_job_queue ON jobs_job (queue, priority DESC, id)
    WHERE status IN ('pending', 'running');
-- 280 000 qatordan faqat ~200 tasi shu holatda → indeks 12 KB, RAM'da

-- Fiskal retry indeksi — self-healing daemon har 60s da o'qiydi
CREATE INDEX idx_fiscal_retry ON fiscal_fiscalreceipt (retry_count, sent_at)
    WHERE status IN ('pending', 'failed') AND retry_count < 5;
-- 25 mln qatordan ~50 tasi → indeks 3 KB
```

4.3 Indeks byudjeti va nazorat

```
QOIDA: indekslar jami hajmi jadval hajmining 60% dan oshmasin.
       Oshsa — yozish tezligi tushadi (har INSERT N ta indeksni yangilaydi).

Hisob: OrderItem — 25,6 GB jadval, 9 GB indeks = 35% ✅
       Dish      — 330 MB jadval, 27 MB indeks = 8%  ✅
       ARView    — 14,5 GB jadval, 5,8 GB indeks = 40% 🔘 (partition bilan boshqariladi)
```

Ishlatilmayotgan indeksni topish (oyda 1 marta):

```
SELECT schemaname, relname AS jadval, indexrelname AS indeks,
       idx_scan AS ishlatilgan,
       pg_size_pretty(pg_relation_size(indexrelid)) AS hajm
FROM pg_stat_user_indexes
WHERE idx_scan < 50                       -- deyarli ishlatilmagan
  AND indexrelname NOT LIKE '%_pkey'
  AND pg_relation_size(indexrelid) > 10*1024*1024
ORDER BY pg_relation_size(indexrelid) DESC;
-- Natija: bunday indeks O'CHIRILADI (yozishni sekinlashtiradi, foyda yo'q)
```

---

5. PARTITIONING

5.1 Nima uchun partition kerak

```
MUAMMO (partitionsiz, 3 yil):
  orders_order = 110 mln qator, 50 GB
  ├─ VACUUM: 4 soat (butun jadval)
  ├─ Indeks qayta qurish: 2 soat, jadval bloklanadi
  ├─ "2024 yilni o'chir" → DELETE 36 mln qator = 6 soat + bloat
  └─ Yangi ustun qo'shish: butun jadvalni qayta yozadi

YECHIM (oy bo'yicha partition):
  orders_order_2026_09 = 3 mln qator, 1,4 GB
  ├─ VACUUM: 5 daqiqa (faqat joriy partition)
  ├─ "2024 ni o'chir" → DETACH + DROP = 0,3 soniya ⚡
  ├─ Eski partition READ ONLY → hech qachon vacuum qilinmaydi
  └─ ⚡ PARTITION PRUNING: WHERE created_at >= '2026-09-01'
                          → planner faqat 1 partitionni ko'radi
```

5.2 Partition qilinadigan jadvallar

Jadval	Kalit	Interval	Saqlash	Partition soni	Bir partition hajmi
`orders_order`	`created_at`	**OY**	36 oy DB + arxiv	36 + 3 kelajak	~1,4 GB
`orders_orderitem`	`created_at`	**OY**	order bilan bir xil	39	~2,2 GB
`ar3d_arview`	`created_at`	**OY**	13 oy → S3	13 + 2	~1,2 GB
`analytics_analyticsevent`	`created_at`	**KUN**	90 kun → ClickHouse	90 + 7	~300 MB
`audit_auditlog`	`created_at`	**OY**	60 oy (qonun)	60 + 2	~200 MB
`ai_aiusage`	`created_at`	**OY**	24 oy	24 + 2	~20 MB

5.3 Yaratish (Django + raw SQL)

Django `Meta.managed = False` + qo'lda partition boshqaruvi (django-postgres-extra ishlatilmaydi — ortiqcha bog'liqlik).

```
# apps/orders/models.py
class Order(models.Model):
    # ... maydonlar ...
    class Meta:
        db_table = 'orders_order'
        managed = True
        # ⚠ PK = (id, created_at) — PostgreSQL partition kaliti PK ga kirishi SHART
        constraints = [
            models.UniqueConstraint(fields=['id', 'created_at'], name='order_pk_part'),
        ]
```

```
-- migrations/0002_partition_orders.py ichidagi RunSQL

-- 1. Ota jadval (partitioned)
CREATE TABLE orders_order (
    id            BIGSERIAL,
    restaurant_id BIGINT NOT NULL REFERENCES restaurants_restaurant(id),
    table_id      BIGINT NULL,
    number        VARCHAR(20) NOT NULL,
    status        VARCHAR(20) NOT NULL DEFAULT 'new',
    subtotal      NUMERIC(14,2) NOT NULL DEFAULT 0,
    service_fee   NUMERIC(14,2) NOT NULL DEFAULT 0,
    total         NUMERIC(14,2) NOT NULL DEFAULT 0,
    idempotency_key VARCHAR(64),
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    -- ...
    PRIMARY KEY (id, created_at)          -- ⚡ partition kaliti PK da
) PARTITION BY RANGE (created_at);

-- 2. Indekslar OTA jadvalda — bolalarga avtomatik tarqaladi
CREATE INDEX ON orders_order (restaurant_id, status, created_at DESC);
CREATE INDEX ON orders_order (restaurant_id, created_at DESC);
CREATE UNIQUE INDEX ON orders_order (idempotency_key, created_at);

-- 3. DEFAULT partition — kalit hech qaysi diapazonga tushmasa (himoya to'ri)
CREATE TABLE orders_order_default PARTITION OF orders_order DEFAULT;
```

5.4 Avtomatik yaratish (cron)

```
# apps/core/management/commands/ensure_partitions.py
"""
Har kuni 03:00 da ishlaydi (cron).
Kelajakdagi 3 oylik partition doim tayyor turadi.
⚡ QOIDA: partition tugab qolsa — INSERT DEFAULT'ga tushadi (yiqilmaydi),
   lekin ogohlantirish yuboriladi.
"""
from django.core.management.base import BaseCommand
from django.db import connection
from dateutil.relativedelta import relativedelta
from datetime import date

OYLIK = {                       # jadval → nechta oy oldinga
    'orders_order':      3,
    'orders_orderitem':  3,
    'ar3d_arview':       3,
    'audit_auditlog':    3,
    'ai_aiusage':        3,
}
KUNLIK = {'analytics_analyticsevent': 7}


class Command(BaseCommand):
    help = "Kelajakdagi partitionlarni yaratadi (idempotent)"

    def handle(self, *args, **opts):
        with connection.cursor() as cur:
            for tbl, oy_soni in OYLIK.items():
                base = date.today().replace(day=1)
                for i in range(oy_soni + 1):
                    start = base + relativedelta(months=i)
                    end   = start + relativedelta(months=1)
                    name  = f"{tbl}_{start:%Y_%m}"
                    cur.execute(f"""
                        CREATE TABLE IF NOT EXISTS {name}
                        PARTITION OF {tbl}
                        FOR VALUES FROM ('{start}') TO ('{end}');
                    """)
                    self.stdout.write(f"  ✔ {name}")

            for tbl, kun_soni in KUNLIK.items():
                for i in range(kun_soni + 1):
                    start = date.today() + relativedelta(days=i)
                    end   = start + relativedelta(days=1)
                    name  = f"{tbl}_{start:%Y_%m_%d}"
                    cur.execute(f"""
                        CREATE TABLE IF NOT EXISTS {name}
                        PARTITION OF {tbl}
                        FOR VALUES FROM ('{start}') TO ('{end}');
                    """)
```

```
# /etc/cron.d/armenu-partitions
0 3 * * *  armenu  /opt/armenu/venv/bin/python /opt/armenu/manage.py ensure_partitions
30 3 * * 0 armenu  /opt/armenu/venv/bin/python /opt/armenu/manage.py archive_partitions
```

5.5 Arxivlash va o'chirish

```
# apps/core/management/commands/archive_partitions.py
"""
Haftada 1 marta: eski partitionni S3'ga chiqarib, DB'dan uzadi.
⚡ DETACH → COPY → DROP tartibi. DETACH bloklamaydi (CONCURRENTLY).
"""
SAQLASH_OY = {
    'orders_order':      36,   # 3 yil — buxgalteriya talabi
    'orders_orderitem':  36,
    'ar3d_arview':       13,   # 13 oy — YoY solishtirish uchun
    'audit_auditlog':    60,   # 5 yil — O'zR qonuni
    'ai_aiusage':        24,
}

def archive(table: str, part: str, cutoff: date):
    with connection.cursor() as cur:
        # 1. Uzish — jadvalni bloklamaydi
        cur.execute(f"ALTER TABLE {table} DETACH PARTITION {part} CONCURRENTLY;")

    # 2. S3'ga eksport (Parquet — ClickHouse/Athena o'qiy oladi)
    path = f"s3://armenu-archive/{table}/{part}.parquet"
    export_to_parquet(part, path)          # COPY ... TO PROGRAM

    # 3. Tekshirish — qator soni mos keldimi
    if verify_row_count(part, path):
        with connection.cursor() as cur:
            cur.execute(f"DROP TABLE {part};")
        log.info("arxivlandi", table=table, part=part, path=path)
    else:
        alert("🔴 Arxiv qator soni mos kelmadi", part=part)   # DROP QILINMAYDI
```

```
-- Nima uchun DETACH CONCURRENTLY (PG 14+):
--   ALTER TABLE ... DETACH PARTITION      → ACCESS EXCLUSIVE lock, butun jadval bloklanadi
--   ALTER TABLE ... DETACH ... CONCURRENTLY → SHARE UPDATE EXCLUSIVE, o'qish/yozish davom etadi ✅
```

5.6 Partition pruning tekshiruvi

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, number, total FROM orders_order
WHERE restaurant_id = 42
  AND created_at >= '2026-09-01' AND created_at < '2026-10-01'
  AND status = 'paid';

-- ✅ KUTILGAN NATIJA:
--  Append  (cost=0.43..812.10 rows=1240)
--    ->  Index Scan using orders_order_2026_09_rest_status_idx
--          on orders_order_2026_09          ← FAQAT 1 PARTITION
--          Index Cond: (restaurant_id = 42 AND status = 'paid')
--          Buffers: shared hit=18
--
-- ❌ AGAR 36 ta partition ko'rinsa → pruning ishlamayapti.
--    Sabab odatda: created_at ga funksiya qo'llangan (DATE(created_at) = ...)
--    yoki parametr sifatida now() ishlatilgan (planner qiymatni bilmaydi).
```

---

6. SHARDING REJASI (S5 — 5 000+ RESTORAN)

6.1 Qachon kerak bo'ladi

> **Sharding — oxirgi chora.** Undan oldin: kesh → replica → partition → PgBouncer.

Signal	Chegara	Ma'nosi
Write RPS	> 2 500 uzluksiz	Primary yozish limitiga yaqin (3 000)
WAL generatsiyasi	> 200 MB/min	Replikatsiya kechikadi
DB hajmi	> 2 TB	Backup vaqti 6 soatdan oshadi
`pg_stat_activity` kutish	lock_wait > 5%	Contention
Vertikal masshtab	32 CPU / 128 GB yetmasa	Yo'l tugadi

Prognoz: 1 000 restoran ≈ 300 write RPS. Sharding**5 000+ restoranda**
(≈1 500 write RPS bo'lganda tayyorgarlik, 2 500 da ijro).

6.2 Shard kaliti — `restaurant_id`

```
SABAB: 99,7% so'rov bitta restoran ichida.
       Cross-restaurant so'rov faqat: (a) platforma hisoboti, (b) DishLibrary.

shard_id = hash(restaurant_id) % SHARD_COUNT

⚠ MUAMMO: SHARD_COUNT o'zgarsa — hamma ko'chadi (rehash).
✅ YECHIM: VIRTUAL SHARD (bucket) — sanoat standarti

   bucket_id = hash(restaurant_id) % 1024      ← HECH QACHON O'ZGARMAYDI
   shard_id  = bucket_map[bucket_id]           ← faqat xarita o'zgaradi

   4 shard → 8 shard: 1024 bucketning 512 tasi ko'chadi, formula o'sha-o'sha.
```

```
BUCKET → SHARD XARITASI (4 shard misolida)

  bucket    0 — 255   →  shard_0   (PG primary + replica)
  bucket  256 — 511   →  shard_1
  bucket  512 — 767   →  shard_2
  bucket  768 — 1023  →  shard_3

  bucket_map Redis'da + DB'da (core_shardmap jadvali), TTL yo'q, versiyalangan.
```

6.3 Citus vs qo'lda routing

Mezon	Citus	Qo'lda routing (Django router)
Ishga tushirish	1 hafta	3-4 hafta
Kod o'zgarishi	~yo'q (SQL shaffof)	Har so'rovda `.using(shard)`
Cross-shard JOIN	✅ avtomatik	❌ ilovada birlashtiriladi
Cross-shard tranzaksiya	✅ 2PC	❌ yo'q (saga kerak)
Rebalans	✅ onlayn (`citus_rebalance_start`)	❌ qo'lda skript
Narx	Community bepul / Managed $$$	Bepul
Vendor lock-in	🟡 o'rta (PG kengaytmasi)	🟢 yo'q
Operatsion murakkablik	🟡 coordinator SPOF	🔴 barcha yuk jamoada

QAROR: 🟢 Citus (Community, self-hosted).

Sabablar:
1. Django kodi deyarli o'zgarmaydi → 15 haftalik reja buzilmaydi
2. `DishLibrary` → **reference table** (har shardda nusxa, JOIN bepul)
3. Platforma hisobotlari → coordinator avtomatik parallel bajaradi
4. Rebalans onlayn — restoranlar sezmaydi

```
-- Citus sozlash
SELECT citus_add_node('shard-1.internal', 5432);
SELECT citus_add_node('shard-2.internal', 5432);
SELECT citus_add_node('shard-3.internal', 5432);
SELECT citus_add_node('shard-4.internal', 5432);

-- 1. GLOBAL jadvallar → reference (har shardda to'liq nusxa)
SELECT create_reference_table('ar3d_dishlibrary');
SELECT create_reference_table('menu_ikpu_catalog');
SELECT create_reference_table('core_featureflag');

-- 2. TENANT jadvallar → distributed, bir xil kalit → colocation
SELECT create_distributed_table('restaurants_restaurant', 'id',
                                shard_count => 1024);
SELECT create_distributed_table('menu_dish',        'restaurant_id',
                                colocate_with => 'restaurants_restaurant');
SELECT create_distributed_table('tables_table',     'restaurant_id',
                                colocate_with => 'restaurants_restaurant');
SELECT create_distributed_table('orders_order',     'restaurant_id',
                                colocate_with => 'restaurants_restaurant');
SELECT create_distributed_table('orders_orderitem', 'restaurant_id',
                                colocate_with => 'restaurants_restaurant');
-- ⚡ COLOCATION: bir restoranning HAMMA ma'lumoti bitta shardda
--    → JOIN mahalliy, tranzaksiya mahalliy, tarmoq yo'q
```

⚠ Colocation uchun `orderitem` ga `restaurant_id` denormalizatsiya qilinadi:

```
# migrations/00XX_orderitem_restaurant.py
class Migration(migrations.Migration):
    operations = [
        # 1. NULL bilan qo'shish (bloklamaydi)
        migrations.AddField('OrderItem', 'restaurant',
            models.ForeignKey('tenants.Restaurant', null=True, on_delete=CASCADE)),
        # 2. Backfill batch'lar bilan (quyida §10.4)
        migrations.RunPython(backfill_restaurant_id, migrations.RunPython.noop),
        # 3. NOT NULL qilish (VALIDATE alohida)
        migrations.RunSQL(
            "ALTER TABLE orders_orderitem ADD CONSTRAINT oi_rest_nn "
            "CHECK (restaurant_id IS NOT NULL) NOT VALID;",
            reverse_sql="ALTER TABLE orders_orderitem DROP CONSTRAINT oi_rest_nn;"),
        migrations.RunSQL(
            "ALTER TABLE orders_orderitem VALIDATE CONSTRAINT oi_rest_nn;"),
    ]
```

6.4 Migratsiya bosqichlari (0 downtime)

```
┌─ BOSQICH 0 — TAYYORGARLIK (sharding'dan 6 oy oldin) ──────────┐
│ • Har tenant jadvalga restaurant_id denormalizatsiya (orderitem, tip…) │
│ • Cross-tenant so'rovlarni topish va yo'q qilish (CI testi)            │
│ • FK'lar shard kaliti ichida ekanini tekshirish                        │
│ • Sequence → UUID yoki snowflake ID (global unique)                    │
│ ⏱ 4-6 hafta · Risk: 🟢 past (faqat sxema)                              │
└────────────────────────────────────────────────────────────────┘
                                ↓
┌─ BOSQICH 1 — COORDINATOR (single node Citus) ─────────────────┐
│ • Mavjud PG'ni Citus coordinator qilib sozlash                         │
│ • create_distributed_table (1 node — hali taqsimlanmagan)              │
│ • Ishlash bir xil qoladi, kod ishlaydi                                 │
│ ⏱ 1 hafta · Risk: 🟡 o'rta · Rollback: undistribute_table()            │
└────────────────────────────────────────────────────────────────┘
                                ↓
┌─ BOSQICH 2 — WORKER QO'SHISH ──────────────────────────────────┐
│ • 4 ta worker node qo'shiladi                                          │
│ • citus_rebalance_start(shard_transfer_mode => 'block_writes' EMAS,    │
│                         'auto' — logical replication bilan onlayn)     │
│ • Har shard ~250 GB ko'chadi, tezlik ~80 MB/s → 50 daqiqa/shard        │
│ • Kechasi 02:00-06:00 oynasida                                         │
│ ⏱ 1 tun · Risk: 🔴 yuqori · Rollback: rebalansni to'xtatish            │
└────────────────────────────────────────────────────────────────┘
                                ↓
┌─ BOSQICH 3 — NAZORAT (2 hafta) ────────────────────────────────┐
│ • citus_stat_statements — sekin so'rovlar                              │
│ • Cross-shard so'rov ulushi < 1% bo'lishi kerak                        │
│ • Shard hajmi nomutanosibligi < 20%                                    │
└────────────────────────────────────────────────────────────────┘
```

6.5 Cross-shard so'rovlar muammosi

So'rov	Muammo	Yechim
Platforma daromadi (barcha restoran)	4 shardga fan-out	✅ Citus parallel bajaradi (~120 ms)
"Eng mashhur taom" (global)	4 shard + birlashtirish	✅ Materialized view, kechasi yangilanadi
`DishLibrary` JOIN `Dish`	Cross-shard JOIN	✅ **Reference table** — mahalliy JOIN
Buyurtma + to'lov bir tranzaksiyada	2PC kerak bo'lishi mumkin	✅ Colocation — ikkalasi bir shardda
Global `idempotency_key` UNIQUE	Shardlar orasida noyoblik yo'q	⚠ **Redis'da tekshiriladi** (DB emas)
Kunlik buyurtma raqami `#047`	Sequence shard-lokal	✅ Raqam restoran ichida — muammo yo'q
Restoran ko'chishi (shard almashish)	Ma'lumot ko'chirish	⚠ `citus_move_shard_placement` + qisqa lock

```
# ⚠ ANTI-NAQSH — sharding'dan keyin ISHLAMAYDI
Order.objects.filter(created_at__gte=today).count()      # butun platformada
# → 4 shardga so'rov, sekin, coordinator'ni bo'g'adi

# ✅ TO'G'RI — analitika ClickHouse'dan
ClickHouse.query("SELECT count() FROM orders WHERE date = today()")
```

---

7. KESH ARXITEKTURASI — 4 QATLAM

7.1 Umumiy sxema

```
    👤 FOYDALANUVCHI
        │  GET /v1/menu/registon/?lang=uz
        ▼
┌──────────────────────────────────────────────────────────────┐
│ QATLAM 1 — BRAUZER (localStorage + HTTP cache)               │
│ ──────────────────────────────────────────────────────────── │
│ Nima: menyu JSON, taom rasmlari, savat holati, til tanlovi   │
│ Kalit: armenu:menu:{slug}:{lang}:v{version}                  │
│ TTL: 5 daqiqa (menyu) · 7 kun (statik) · sessiya (savat)     │
│ Hajm: ~250 KB/restoran                                       │
│ Invalidation: version raqami URL'da → eski kalit shunchaki qoladi │
│ 🎯 HIT RATE MAQSAD: 45%  (bir mehmon o'rtacha 4 marta menyuni ochadi) │
│ ⚡ Foyda: takroriy ochish 0 ms, oflayn ham ko'rinadi         │
└──────────────────────────────────────────────────────────────┘
        │  miss (55%)
        ▼
┌──────────────────────────────────────────────────────────────┐
│ QATLAM 2 — CDN (Cloudflare)                                  │
│ ──────────────────────────────────────────────────────────── │
│ Nima: 3D fayllar (GLB/USDZ), rasmlar, JS/CSS, ⚡ menyu JSON ham │
│ Kalit: to'liq URL + Vary: Accept-Language                    │
│ TTL: 60s (menyu API) · 30 kun (3D) · 1 yil (hash'li statik)  │
│ Invalidation: URL'dagi ?v={menu_version} → yangi URL = yangi kesh │
│ 🎯 HIT RATE MAQSAD: 92% (3D uchun 99% — bir xil kutubxona URL) │
│ ⚡ Foyda: 75% trafik serverga UMUMAN yetib kelmaydi          │
└──────────────────────────────────────────────────────────────┘
        │  miss (8% menyu, 1% 3D)
        ▼
┌──────────────────────────────────────────────────────────────┐
│ QATLAM 3 — REDIS (db=5, cache)                               │
│ ──────────────────────────────────────────────────────────── │
│ Nima: qurilgan menyu JSON, restoran obyekti, stol→restoran xarita, │
│       sessiya, rate-limit hisoblagich, idempotency, AI javob keshi │
│ Kalit: {tur}:{scope}:{param}:v{version}   (quyida §7.2)      │
│ TTL: menyu 60s · restoran 600s · qr→table 3600s · AI 24 soat │
│ Hajm: 1 000 restoran × 250 KB = 250 MB + sessiya 400 MB ≈ 700 MB │
│ 🎯 HIT RATE MAQSAD: > 90%                                    │
│ ⚡ Foyda: 12 000 RPS menyu → 1 200 RPS DB                    │
└──────────────────────────────────────────────────────────────┘
        │  miss (<10%)
        ▼
┌──────────────────────────────────────────────────────────────┐
│ QATLAM 4 — POSTGRESQL                                        │
│ ──────────────────────────────────────────────────────────── │
│ shared_buffers = 8 GB → issiq to'plam (1,5 GB) to'liq RAM'da │
│ effective_cache_size = 24 GB (OS page cache hisobga olinadi) │
│ 🎯 Buffer hit ratio maqsad: > 99%                            │
│ SELECT → read replica (DB router) · INSERT/UPDATE → primary  │
└──────────────────────────────────────────────────────────────┘
```

7.2 Kesh kaliti formati (qat'iy standart)

```
{turi}:{ko'lam}:{identifikator}[:{parametr}]:v{versiya}

MISOLLAR:
  menu:rest:registon:uz:v42           ← menyu JSON (restoran+til+versiya)
  menu:rest:registon:ru:v42
  dish:rest:15:uz:v42                 ← bitta taom
  rest:obj:registon:v42               ← restoran obyekti
  qr:table:a3f2c891-...               ← QR token → (restaurant_id, table_id)
  sess:guest:gs_a3f2c891              ← mehmon sessiyasi (TTL 4 soat)
  idem:order:550e8400-...             ← idempotency (TTL 24 soat)
  rl:ip:213.230.99.1:menu             ← rate limit (TTL 60s)
  ai:chat:sha256(savol+lang)          ← AI javob keshi (TTL 24 soat)
  lib:search:sha256(query)            ← kutubxona qidiruv (TTL 1 soat)

⚠ QAT'IY QOIDALAR:
  1. Kalit HECH QACHON foydalanuvchi kiritgan matnni to'g'ridan olmaydi → sha256
  2. Tenant ko'lami kalitda MAJBURIY (menu:rest:{slug}) — sizish bo'lmasin
  3. Versiya oxirida — invalidation shu orqali
  4. Maksimal kalit uzunligi 200 bayt
```

7.3 Kesh qatlamlari — solishtirma jadval

Qatlam	Kechikish	Sig'im	Narx	Invalidation tezligi	Xato bahosi
1. Brauzer	**0 ms**	5-10 MB	$0	🟡 sekin (TTL kutiladi)	Eski menyu ko'rinadi (5 daq)
2. CDN	15-40 ms	∞	$200/oy	🟢 URL o'zgarishi bilan darhol	Eski narx (60s)
3. Redis	0,3-1 ms	700 MB	$80/oy	🟢 darhol	60s
4. DB	3-20 ms	500 GB	$400/oy	—	—

7.4 Kesh o'qish/yozish kodi

```
# apps/core/cache.py
from django.core.cache import cache
from django.db import transaction
import hashlib, json

MENU_TTL = 60
REST_TTL = 600
QR_TTL   = 3600


def menu_key(slug: str, lang: str, version: int) -> str:
    return f"menu:rest:{slug}:{lang}:v{version}"


def get_menu(slug: str, lang: str) -> dict:
    """
    ⚡ 2 bosqichli o'qish:
       1) restoran obyekti (menu_version bilan) — o'zi keshlangan
       2) shu versiya bilan menyu kaliti
    """
    rest = get_restaurant(slug)                     # keshdan, 0,3 ms
    key = menu_key(slug, lang, rest['menu_version'])

    data = cache.get(key)
    if data is not None:
        metrics.inc('cache.menu.hit')
        return data

    metrics.inc('cache.menu.miss')

    # ⚡ STAMPEDE HIMOYASI — bir vaqtda 1 000 miss bo'lsa
    lock_key = f"lock:{key}"
    if cache.add(lock_key, "1", timeout=10):         # SET NX
        try:
            data = build_menu(slug, lang)            # DB, ~35 ms
            cache.set(key, data, MENU_TTL)
        finally:
            cache.delete(lock_key)
    else:
        # Boshqa process qurmoqda — 50 ms kutamiz, keyin DB'dan (fallback)
        time.sleep(0.05)
        data = cache.get(key) or build_menu(slug, lang)

    return data
```

7.5 Kesh isitish (warming)

```
# apps/core/management/commands/warm_cache.py
"""
Har kuni 07:30 da (restoranlar ochilishidan oldin) ishlaydi.
Faol restoranlarning menyusini oldindan Redis'ga yuklaydi.
⚡ Foyda: kunning birinchi mehmoni ham 3 ms javob oladi (35 ms emas)
"""
def handle(self, *args, **opts):
    qs = Restaurant.objects.filter(is_active=True).only('slug', 'langs_enabled')
    for rest in qs.iterator(chunk_size=200):
        for lang in rest.langs_enabled:
            get_menu(rest.slug, lang)          # keshga yozadi
    # 1 000 restoran × 3 til = 3 000 qurish × 35 ms = 105 s ✅
```

---

8. KESH INVALIDATION — `menu_version` NAQSHI

8.1 Muammo: nega `delete_pattern` ISHLATILMAYDI

```
# ✗ QAT'IY TAQIQ
cache.delete_pattern("menu:rest:registon:*")
```

#	Sabab	Batafsil
1	`KEYS` / `SCAN` **bloklaydi**	`KEYS` — O(n), 500k kalitda Redis 400 ms muzlaydi. Redis **bir oqimli** — shu paytda BARCHA so'rov kutadi
2	**Redis Cluster'da ishlamaydi**	Kalitlar 16 384 slotga tarqalgan; pattern bir nechта node'da — atomik emas
3	**Atomik emas**	3 til × 2 ko'rinish = 6 kalit. 3 tasi o'chib, 3 tasi qolsa → mijoz eski/yangi aralash ko'radi
4	**Race condition**	Delete bilan bir vaqtda boshqa process eski ma'lumotni qayta yozadi (quyida §8.4)
5	**CDN'ni tozalamaydi**	Redis tozalandi, CDN'da eski JSON 60s qoladi

8.2 ✅ YECHIM: versiya raqami

```
PRINTSIP: keshni O'CHIRMAYMIZ — KALITNI O'ZGARTIRAMIZ.
          Eski kalitlar TTL bo'yicha o'z-o'zidan o'ladi.

  Restaurant.menu_version = 42
       │
       ├─► Redis kaliti:  menu:rest:registon:uz:v42
       ├─► CDN URL:       /v1/menu/registon/?lang=uz&v=42
       ├─► ETag:          "v42-uz"
       └─► localStorage:  armenu:menu:registon:uz:v42

  Admin taom narxini o'zgartirdi:
       menu_version 42 → 43   (bitta UPDATE, 1 ms)
       │
       ├─► Redis: menu:...:v43 — YO'Q → miss → qayta quriladi ✅
       ├─► CDN:   ?v=43 — yangi URL → origin'ga boradi ✅
       ├─► ETag:  "v43-uz" ≠ "v42-uz" → 200 (304 emas) ✅
       └─► Brauzer: yangi kalit → localStorage miss ✅

  ⚡ HAMMA QATLAM BIR VAQTDA, ATOMIK, 1 ms ICHIDA.
  ⚡ Eski v42 kalitlari 60 soniyada TTL bo'yicha o'ladi — hech kim tegmaydi.
```

8.3 Implementatsiya (signal)

```python
# apps/menu/signals.py
from django.db.models.signals import post_save, post_delete
from django.dispatch import receiver
from django.db.models import F
from django.db import transaction
from apps.tenants.models import Restaurant
from apps.menu.models import Dish, Category, DishModifier
from apps.ar3d.models import DishModel3D

MENYUGA_TASIR_QILADI = (Dish, Category, DishModifier, DishModel3D)


def bump_menu_version(restaurant_id: int):
    """
    ⚡ F() ifodasi — atomik INCREMENT, race condition YO'Q.
    ⚠ .save() bilan qilinsa: read-modify-write → parallel 2 admin → 1 ta bump yo'qoladi.
    """
    Restaurant.objects.filter(id=restaurant_id).update(
        menu_version=F('menu_version') + 1
    )
    # Restoran obyekti keshini ham yangilaymiz (u menu_version'ni saqlaydi)
    cache.delete(f"rest:obj:{restaurant_id}")
    cache.delete(f"rest:obj:slug:{slug_of(restaurant_id)}")


@receiver(post_save, sender=Dish)
@receiver(post_delete, sender=Dish)
@receiver(post_save, sender=Category)
@receiver(post_delete, sender=Category)
@receiver(post_save, sender=DishModifier)
@receiver(post_save, sender=DishModel3D)
def on_menu_change(sender, instance, **kwargs):
    rid = _resolve_restaurant_id(instance)
    if rid:
        # ⚡ on_commit — tranzaksiya rollback bo'lsa bump ham bo'lmaydi
        transaction.on_commit(lambda: bump_menu_version(rid))
```

⚠ Bulk operatsiyalarda signal ishlamaydi — qo'lda chaqiriladi:

```python
# ✗ signal chaqirilmaydi
Dish.objects.filter(category_id=5).update(is_available=False)

# ✅ TO'G'RI
with transaction.atomic():
    Dish.objects.filter(category_id=5).update(is_available=False)
    transaction.on_commit(lambda: bump_menu_version(rid))
```

8.4 Race condition tahlili

```
SSENARIY: menyu keshi mavjud emas, admin bir vaqtda narxni o'zgartiradi.

  T0   Web-1: cache.get("menu:...:v42") → MISS
  T1   Web-1: build_menu() boshlanadi — DB'dan eski narx 45 000 o'qiladi
  T2   Admin: narx 50 000 → UPDATE dish; menu_version 42 → 43
  T3   Web-1: cache.set("menu:...:v42", eski_narx, 60)   ← ESKI kalitga yozdi

  ✅ NATIJA: zarar YO'Q.
     Yangi so'rovlar menu_version=43 o'qiydi → "menu:...:v43" so'raydi → MISS
     → yangi narx bilan quriladi. "…v42" kaliti hech kimga kerak emas, 60s da o'ladi.

  ⚡ MANA SHU — versiya naqshining ASOSIY USTUNLIGI:
     delete_pattern bo'lganda T3 dagi yozuv AMAL QILGAN kalitni ifloslantirardi
     va mijoz 60 soniya eski narxni ko'rardi (moliyaviy xato!).
```

Ikkinchi race — `menu_version` o'qishi:

```
  T0  Web-1: rest = get_restaurant()  → menu_version = 42  (keshdan, 600s TTL!)
  T1  Admin: version 43 ga o'tdi, rest:obj keshi o'chirildi
  T2  Web-1: hali ham v42 kaliti bilan ishlaydi (o'sha so'rov ichida)

  ⚠ Bu qabul qilinadigan: bitta so'rov davomida eski versiya ishlatiladi.
     Keyingi so'rov rest:obj keshini miss qiladi → 43 ni oladi.
     Maksimal kechikish = 0 (kesh o'chirilgan), lekin in-flight so'rovlar eski.
```

8.5 Invalidation matritsasi

O'zgarish	`menu_version`	Qo'shimcha o'chiriladigan	CDN purge	Kechikish
Taom narxi	✅ +1	`dish:rest:{id}:*`	❌ (v o'zgardi)	< 1 s
Stop-list (`is_available`)	✅ +1	—	❌	< 1 s
Yangi taom	✅ +1	—	❌	< 1 s
Kategoriya tartibi	✅ +1	—	❌	< 1 s
3D model tayyor	✅ +1	`ar:dish:{id}`	❌	< 1 s
Restoran logotipi	✅ +1	`rest:obj:*`	✅ **purge kerak**	5-30 s
Restoran nofaol	✅ +1	`rest:obj:*`, `qr:table:*`	✅	5-30 s
QR token yangilash	❌	`qr:table:{eski}`	❌	darhol
Xodim PIN	❌	`staff:{id}`	❌	darhol
Buyurtma holati	❌	— (kesh yo'q)	❌	WS orqali
Tarif o'zgarishi	✅ +1	`rest:obj:*`, `plan:{id}`	❌	< 1 s

```
# Fayl o'zgarganda (logotip, rasm) — CDN purge kerak
def purge_cdn(urls: list[str]):
    """Cloudflare API. Faqat FAYL o'zgarganda (JSON emas!)."""
    requests.post(
        f"https://api.cloudflare.com/client/v4/zones/{ZONE}/purge_cache",
        headers={"Authorization": f"Bearer {CF_TOKEN}"},
        json={"files": urls}, timeout=5,
    )
    # ⚠ Rate limit: 1 000 purge/kun (Pro). Shuning uchun JSON uchun
    #    ishlatilmaydi — faqat rasm/fayl uchun (kuniga ~20 marta).
```

9. MA'LUMOT HAYOTIY SIKLI

9.1 4 bosqichli sikl



9.2 Har ma'lumot turi uchun siyosat

Ma'lumot	♨ Issiq	🌡 Iliq	❄ Sovuq	● O'chirish	Asos
Buyurtma (`order`)	30 kun	12 oy	36 oy	3 yildan keyin agregat qoladi	Buxgalteriya 3 yil
To'lov (`payment`)	30 kun	12 oy	**60 oy**	❌ hech qachon	Moliyaviy tekshiruv
Fiskal chek	7 kun	12 oy	**60 oy**	❌ hech qachon	🟡 O'zR Soliq kodeksi
Audit log	—	12 oy	**60 oy**	5 yildan keyin	🟡 Qonun
Mehmon (`guest`)	faol 30 kun	12 oy	—	🟡 **24 oy passiv → anonimlashtirish**	Shaxsiy ma'lumot
GuestSession	4 soat	90 kun	—	12 oy	—
AR ko'rish	30 kun	13 oy	S3 (agregat)	13 oy → agregat qoladi	Analitika
AnalyticsEvent	1 kun	7 kun	ClickHouse 24 oy	90 kun (PG'dan)	Analitika
Chaqiruv (`call`)	1 kun	30 kun	—	90 kun	Operatsion
Sharh (`review`)	30 kun	∞	—	❌	Reputatsiya
AI so'rov (`aiusage`)	7 kun	24 oy	—	24 oy	Xarajat nazorati
Job	1 soat	—	—	7 kun (done), 30 kun (failed)	—
3D fayllar (S3)	∞	—	—	Taom o'chsa +90 kun	Tiklash oynasi

9.3 GDPR va O'zR "Shaxsiy ma'lumotlar to'g'risida"gi qonuni

> **O'zR Qonuni № ZRU-547 (02.07.2019)** — shaxsiy ma'lumotlar O'zbekiston hududidagi serverlarda saqlanishi shart (lokalizatsiya talabi).

Shaxsiy ma'lumot deb hisoblanadigan maydonlar:

Jadval	Maydon	Toifa	Chora
`guest`	`phone`	🔴 identifikator	Anonimlashtirish
`guest`	`name`	🔴 identifikator	Anonimlashtirish
`guest`	`tg_user_id`	🔴 identifikator	Anonimlashtirish
`guest`	`device_token`	🟡 psevdo-ID	Anonimlashtirish
`staff`	`user.email`, `phone`	🔴 identifikator	Ishdan bo'shasa +1 yil
`order`	`note` (mijoz matni)	🟡 bo'lishi mumkin	3 yildan keyin tozalanadi
`review`	`text`	🟡 bo'lishi mumkin	Saqlanadi (ommaviy)
`arview`	`session_id`, `device`	🟢 anonim	—
`auditlog`	`ip`	🟡	12 oydan keyin /24 ga qisqartiriladi

```
# apps/guest/management/commands/anonymize_guests.py
"""
Oyda 1 marta. 24 oy faol bo'lmagan mehmonlarni anonimlashtiradi.
⚡ O'CHIRMAYDI — anonimlashtiradi. Sabab: buyurtma tarixi (moliyaviy hujjat)
   FK bilan bog'liq va o'chirilishi mumkin emas.
"""
from datetime import timedelta
import hashlib

CHEGARA = timedelta(days=730)

def handle(self, *args, **opts):
    cutoff = timezone.now() - CHEGARA
    qs = Guest.objects.filter(last_seen__lt=cutoff, is_anonymized=False)

    for chunk in batched(qs.iterator(chunk_size=1000), 1000):
        with transaction.atomic():
            for g in chunk:
                # Barqaror psevdonim (statistika buzilmasin)
                pseudo = hashlib.sha256(
                    f"{g.id}:{settings.ANON_SALT}".encode()
                ).hexdigest()[:16]
                g.phone         = ''
                g.name          = f'anon_{pseudo}'
                g.tg_user_id    = None
                g.device_token  = None
                g.is_anonymized = True
            Guest.objects.bulk_update(chunk, [
                'phone', 'name', 'tg_user_id', 'device_token', 'is_anonymized'
            ])
    # Order.guest_id saqlanadi → hisobotlar buzilmaydi, shaxs aniqlanmaydi ✓
```

Mehmon so'rovi bo'yicha o'chirish (right to erasure):

```
# POST /v1/guest/forget/  (SMS tasdiqlash bilan)
def forget_me(guest: Guest):
    with transaction.atomic():
        anonymize(guest)                                    # yuqoridagi
        Review.objects.filter(guest=guest).update(          # sharh anonim
            guest=None, author_name='Anonim')
        ARView.objects.filter(guest=guest).update(guest=None)
        # ⚭ Order/Payment/FiscalReceipt — TEGILMAYDI (qonuniy majburiyat)
        AuditLog.log(guest, action='gdpr_erasure', actor='self')
    return {"status": "ok", "note": "Moliyaviy hujjatlar qonun talabi bilan saqlanadi"}
```

9.4 Arxiv formati va o'qish

```
S3 TUZILISHI:
  s3://armenu-archive/
    orders/orders_order_2024_01.parquet          (siqilgan, ~180 MB)
    orderitem/orders_orderitem_2024_01.parquet
    arview/ar3d_arview_2025_03.parquet
    auditlog/audit_auditlog_2023_11.parquet

FORMAT: Parquet + ZSTD
  ├─ Siqish: CSV ga nisbatan 8×, PG jadvalga nisbatan 5×
  ├─ Ustunli → "faqat total ustunini o'qi" — 20× tez
  └─ ClickHouse, Athena, DuckDB, pandas — hammasi o'qiydi

O'QISH (arxivdan hisobot):
  SELECT sum(total) FROM s3('s3://armenu-archive/orders/*.parquet')
  WHERE restaurant_id = 42 AND toYear(created_at) = 2024;
  → ClickHouse orqali, ~8 soniya
```

---

10. MIGRATSIYA STRATEGIYASI (ZERO-DOWNTIME)

10.1 Asosiy muammo

```
⚠ Deploy paytida ESKI va YANGI kod BIR VAQTDA ishlaydi (rolling restart, 2-5 daqiqa).
   Migratsiya ikkalasiga ham mos kelishi SHART.

   t=0    40 instans × eski kod
   t=30s  migratsiya bajarildi       ← BU PAYT ikkala kod ham ishlashi kerak
   t=60s  20 eski + 20 yangi
   t=180s 40 instans × yangi kod
```

10.2 🔴 QAT'IY TAQIQLANGAN operatsiyalar (bitta deployda)

⛔ Taqiqlangan	Nima uchun	✅ To'g'ri yo'l
`ADD COLUMN ... NOT NULL` (DEFAULT'siz)	Butun jadval qayta yoziladi + eski kod NULL yozadi → xato	3 bosqich (§10.3)
`DROP COLUMN`	Eski kod hali o'sha ustunni SELECT qiladi	2 deploy (§10.5)
`RENAME COLUMN`	Eski kod eski nomni izlaydi	Yangi qo'sh → ko'chir → eski o'chir
`ALTER TYPE` (varchar→int)	Jadval bloklanadi + eski kod tipni bilmaydi	Yangi ustun + backfill
`CREATE INDEX` (CONCURRENTLY'siz)	🔴 ACCESS EXCLUSIVE — jadval to'liq bloklanadi	`CONCURRENTLY`
`ADD CONSTRAINT` (NOT VALID'siz)	Butun jadval skan qilinadi, bloklanadi	`NOT VALID` + `VALIDATE`
`ALTER TABLE SET NOT NULL` (to'g'ridan)	Full scan + lock	CHECK NOT VALID → VALIDATE → SET NOT NULL

10.3 ✅ Ustun QO'SHISH — 3 deploy naqshi

```
┌─ DEPLOY 1: SXEMA (kod o'zgarmaydi) ──────────────────────────┐
│  ALTER TABLE menu_dish ADD COLUMN spice_level SMALLINT DEFAULT 0;      │
│                                                                        │
│  ⚡ PostgreSQL 11+ da DEFAULT bilan ADD COLUMN — METADATA-ONLY.        │
│     Jadval qayta yozilmaydi, lock ~2 ms, 133k qatorda ham darhol.      │
│  ⚠ DEFAULT'siz + NOT NULL bo'lsa → 133k qator qayta yoziladi (12 s     │
│     lock) → SO'ROVLAR YIQILADI.                                        │
│                                                                        │
│  Eski kod: ustunni bilmaydi, SELECT * da oladi, e'tibor bermaydi ✅    │
└────────────────────────────────────────────────────────────────────────┘
                                  ↓  (bir necha soat / kun)
┌─ DEPLOY 2: KOD YOZADI ─────────────────────────────────────────┐
│  • Model'ga maydon qo'shiladi                                  │
│  • Serializer YOZADI, lekin O'QISHDA default bilan ishlaydi    │
│  • Backfill boshlanadi (batch, §10.4)                          │
└────────────────────────────────────────────────────────────────┘
                                  ↓
┌─ DEPLOY 3: MAJBURIY QILISH ────────────────────────────────────┐
│  ALTER TABLE menu_dish ADD CONSTRAINT dish_spice_nn             │
│      CHECK (spice_level IS NOT NULL) NOT VALID;   -- lock 2 ms  │
│  ALTER TABLE menu_dish VALIDATE CONSTRAINT dish_spice_nn;       │
│      -- SHARE UPDATE EXCLUSIVE: o'qish/yozish davom etadi ✅    │
└────────────────────────────────────────────────────────────────┘
```

10.4 Katta jadval backfill (batch)

```
# apps/orders/migrations/00XX_backfill_restaurant.py
"""
127 mln qatorli orderitem jadvalini to'ldirish.
⚡ QOIDA: bitta UPDATE'da 5 000 dan ko'p qator YO'Q.
   Sabab: uzun tranzaksiya → WAL o'sadi, replica kechikadi, lock ushlanadi.
"""
BATCH = 5000
PAUZA = 0.05          # 50 ms — replica kechikmasligi uchun


def backfill_restaurant_id(apps, schema_editor):
    OrderItem = apps.get_model('orders', 'OrderItem')
    max_id = OrderItem.objects.aggregate(m=Max('id'))['m'] or 0
    cur = 0
    while cur < max_id:
        with transaction.atomic():
            schema_editor.execute("""
                UPDATE orders_orderitem oi
                SET restaurant_id = o.restaurant_id
                FROM orders_order o
                WHERE oi.order_id = o.id
                  AND oi.restaurant_id IS NULL
                  AND oi.id > %s AND oi.id <= %s
            """, [cur, cur + BATCH])
        cur += BATCH
        time.sleep(PAUZA)

        # ⚡ Replica kechikishini kuzatish — 10 s dan oshsa to'xtaymiz
        if replica_lag_seconds() > 10:
            time.sleep(5)

# HISOB: 127 750 000 / 5 000 = 25 550 batch
#        Har batch ~80 ms ish + 50 ms pauza = 130 ms
#        Jami: 25 550 × 0,13 s = 3 322 s ≈ 55 daqiqa
#        ⚡ Fon rejimida ishlaydi, xizmat uzilmaydi ✅
```

10.5 Ustun O'CHIRISH — 2 deploy

```
┌─ DEPLOY 1: KODDAN OLIB TASHLASH ──────────────────────────────┐
│  • Model'da maydon qoladi, lekin HECH QAYERDA ishlatilmaydi    │
│  • Serializer, admin, hisobotdan olib tashlanadi               │
│  • ⚡ Barcha queryset'ga .defer('eski_ustun') yoki .only(...)  │
│  • 1-2 hafta kutiladi (rollback oynasi)                        │
└────────────────────────────────────────────────────────────────┘
                                  ↓
┌─ DEPLOY 2: DB'DAN O'CHIRISH ──────────────────────────────────┐
│  ALTER TABLE menu_dish DROP COLUMN eski_ustun;                 │
│  ⚡ DROP COLUMN — metadata-only, tez. Joy VACUUM FULL'da bo'shaydi. │
└────────────────────────────────────────────────────────────────┘

⚠ SELECT * XAVFI: Django SELECT * qilmaydi (ustunlarni sanaydi) —
   shuning uchun DEPLOY 1 da model'dan olib tashlash O'ZI YETARLI EMAS:
   eski instansda hali eski model bor → SELECT eski_ustun → 42703 xato.
   Shuning uchun tartib: kod → kutish → DB.
```

10.6 Indeks yaratish (katta jadval)

```
-- ✗ NOTO'G'RI — 36 mln qatorli jadvalni 8 daqiqa bloklaydi
CREATE INDEX idx_order_new ON orders_order (waiter_id, created_at);

-- ✓ TO'G'RI
CREATE INDEX CONCURRENTLY idx_order_new ON orders_order (waiter_id, created_at);
-- Jadvalni 2 marta skanlaydi, 2× sekin, LEKIN bloklamaydi

-- ⚠ CONCURRENTLY tranzaksiya ichida ishlamaydi:
```

```
class Migration(migrations.Migration):
    atomic = False                    # ⚡ MAJBURIY
    operations = [
        migrations.RunSQL(
            "CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_order_waiter "
            "ON orders_order (waiter_id, created_at DESC);",
            reverse_sql="DROP INDEX CONCURRENTLY IF EXISTS idx_order_waiter;",
        ),
    ]

# ⚠ CONCURRENTLY yiqilsa → INVALID indeks qoladi. Tekshirish:
#   SELECT indexrelid::regclass FROM pg_index WHERE NOT indisvalid;
#   → DROP INDEX CONCURRENTLY ... va qaytadan
```

10.7 Migratsiya CI tekshiruvi

```
# tests/test_migrations_safe.py
XAVFLI_NAQSHLAR = [
    (r'ADD COLUMN.*NOT NULL(?!.*DEFAULT)', 'NOT NULL ustun DEFAULT bilan bo\'lsin'),
    (r'CREATE INDEX(?! CONCURRENTLY)',     'CONCURRENTLY ishlatilsin'),
    (r'ADD CONSTRAINT(?!.*NOT VALID)',     'NOT VALID + VALIDATE bosqichi'),
    (r'ALTER COLUMN.*TYPE',                'Tip o\'zgartirish taqiqlangan'),
    (r'DROP COLUMN',                       'Faqat 2-deploy naqshida (# noqa-drop)'),
    (r'RENAME COLUMN',                     'Rename taqiqlangan'),
]

def test_yangi_migratsiyalar_xavfsiz():
    for f in yangi_migratsiya_fayllari():          # git diff origin/main
        matn = f.read_text()
        if '# migration-safe-ok' in matn:          # qo'lda tasdiqlangan
            continue
        for naqsh, xabar in XAVFLI_NAQSHLAR:
            assert not re.search(naqsh, matn, re.I), f"{f.name}: {xabar}"
```

10.8 Backward compatibility matritsasi

O'zgarish	Eski kod + yangi DB	Yangi kod + eski DB	Xavfsizmi
Ustun qo'shish (DEFAULT bilan)	✓ e'tibor bermaydi	✗ ustun yo'q	🟢 DB birinchi
Ustun o'chirish	✗ SELECT xato	✓ ishlatmaydi	🟢 Kod birinchi
Indeks qo'shish	✓	✓	🟢 Istalgan tartib
Yangi jadval	✓	✗	🟢 DB birinchi
Enum'ga qiymat qo'shish	✓ (choices Python'da)	✓	🟢
Enum'dan qiymat olib tashlash	✗ eski yozuvlar	✗	🟡 Ikki bosqich
FK qo'shish	✓	✗	🟢 DB birinchi
`unique_together` qo'shish	🟡 eski kod dublikat yozadi	✓	🟡 Kod birinchi + tozalash

---

11. MA'LUMOT BUTUNLIGI

11.1 Himoya qatlamlari

```
┌──────────────────────────────────────────────────┐
│ 1. FRONTEND validatsiya   → 🟢 UX uchun (ISHONILMAYDI)   │
├──────────────────────────────────────────────────┤
│ 2. DRF serializer         → 🟢 format, tip, diapazon     │
├──────────────────────────────────────────────────┤
│ 3. SERVIS qatlami         → 🟢 biznes qoidalari          │
├──────────────────────────────────────────────────┤
│ 4. Django model.clean()   → 🟢 model darajasi            │
├──────────────────────────────────────────────────┤
│ 5. DB CONSTRAINT          → 🟡 OXIRGI VA ISHONCHLI CHEGARA│
│    Kod xato qilsa ham DB ruxsat bermaydi                 │
└──────────────────────────────────────────────────┘

⚡ PRINTSIP: har biznes-invariant DB darajasida ham himoyalanadi.
   Sabab: ma'lumotni faqat Django yozmaydi — migratsiya, admin,
   shell, boshqa servis ham yozadi.
```

11.2 CHECK constraint'lar

```
class Order(models.Model):
    class Meta:
        constraints = [
            models.CheckConstraint(
                check=Q(subtotal__gte=0) & Q(service_fee__gte=0)
                    & Q(discount__gte=0) & Q(total__gte=0),
                name='order_summa_manfiy_emas'),
            models.CheckConstraint(
                check=Q(total=F('subtotal') + F('service_fee') - F('discount')),
                name='order_total_togri'),          # 🟡 moliyaviy invariant
            models.CheckConstraint(
                check=Q(discount__lte=F('subtotal')),
                name='order_chegirma_subtotaldan_oshmaydi'),
            models.CheckConstraint(
                check=Q(status__in=['new','accepted','cooking','ready',
                                    'served','paid','cancelled']),
                name='order_status_togri'),
        ]


class OrderItem(models.Model):
    class Meta:
        constraints = [
            models.CheckConstraint(
                check=Q(qty__gte=1) & Q(qty__lte=99),
                name='orderitem_qty_1_99'),          # ⚡ butun son, 0 emas
            models.CheckConstraint(check=Q(price__gte=0),
                name='orderitem_narx_manfiy_emas'),
        ]


class Dish(models.Model):
    class Meta:
        constraints = [
            models.CheckConstraint(check=Q(price__gt=0),
                name='dish_narx_musbat'),
            models.CheckConstraint(
                check=Q(old_price__isnull=True) | Q(old_price__gt=F('price')),
                name='dish_eski_narx_kattaroq'),     # chegirma mantiqi
            models.CheckConstraint(
                check=Q(rating_avg__gte=0) & Q(rating_avg__lte=5),
                name='dish_reyting_0_5'),
        ]


class Payment(models.Model):
    class Meta:
        constraints = [
            models.CheckConstraint(check=Q(amount__gt=0),
                name='payment_summa_musbat'),        # 🟡 0 to'lov bo'lmaydi
        ]
```

11.3 UNIQUE constraint'lar

Jadval	Constraint	Nima uchun	Buzilsa
`restaurant`	`slug`	URL noyob	Ikki restoran bir URL'da 🟡
`table`	`qr_token`	🟡 QR — kirish kaliti	Boshqa restoran menyusi ochiladi 🟡🟡
`table`	`(restaurant, number)`	Stol raqami takrorlanmaydi	Ofitsiant adashadi
`order`	`idempotency_key`	🟡 **Dublikat buyurtma**	Mijozdan 2 marta pul 🟡
`order`	`(restaurant, number, kun)`	Kunlik raqam noyob	Chek raqami takrorlanadi
`payment`	`idempotency_key`	🟡 Dublikat to'lov	Moliyaviy zarar 🟡
`payment`	`(provider, ext_id)`	Provayder webhook 2 marta keladi	Ikki marta hisoblash 🟡
`fiscalreceipt`	`order` (1:1)	Bir buyurtma — bir chek	Soliq xatosi
`dishlibrary`	`key`	Kutubxona kaliti	Moslashtirish buziladi
`guest`	`(restaurant, device_token)`	Qurilma bir mehmon	Bonus dublikati
`dishmodel3d`	`dish` (1:1)	Bir taom — bir model	AR chalkashadi

```
-- ⚡ Kunlik buyurtma raqami — partial unique + funksional indeks
CREATE UNIQUE INDEX uq_order_kunlik_raqam
    ON orders_order (restaurant_id, number, (created_at AT TIME ZONE 'Asia/Tashkent')::date)
    WHERE status != 'cancelled';
-- Bekor qilingan buyurtma raqami qayta ishlatilishi mumkin
```

11.4 Tranzaksiya izolyatsiya darajalari

Operatsiya	Izolyatsiya	Sabab
Menyu o'qish	`READ COMMITTED` (default)	Kesh baribir ishlatiladi
Buyurtma yaratish	`READ COMMITTED` + `SELECT FOR UPDATE`	Faqat stol qulflanadi, butun jadval emas
To'lov qayd etish	`READ COMMITTED` + idempotency	UNIQUE constraint himoya qiladi
Kunlik raqam berish	`READ COMMITTED` + advisory lock	Sequence emas — raqam har kuni 1 dan boshlanadi
Kassa yopish / smena	`REPEATABLE READ`	Hisobot davomida ma'lumot o'zgarmasin
Moliyaviy hisobot	`REPEATABLE READ` (replica'da)	Bir xil snapshot
Analitika	`READ COMMITTED` (replica)	Aniqlik kritik emas

```
⚠ SERIALIZABLE ISHLATILMAYDI.
   Sabab: 1 000 RPS da serialization failure ko'payadi → retry bo'roni.
   Buning o'rniga: aniq lock (FOR UPDATE) + UNIQUE constraint + idempotency.
```

11.5 `select_for_update` — qayerda va nega

```
# ✅ 1. STOL HOLATI — bir stolga bir vaqtda 2 buyurtma
def create_order(session, table_id, items, idem_key):
    with transaction.atomic():
        table = (Table.objects
                 .select_for_update(of=('self',))      # ⚡ faqat table qatori
                 .select_related('restaurant')
                 .get(id=table_id, restaurant_id=session.restaurant_id))
        # ... buyurtma yaratiladi ...
        table.status = 'ordering'
        table.save(update_fields=['status'])
    # Lock tranzaksiya oxirida bo'shaydi (~80 ms)


# ✅ 2. KUNLIK RAQAM — advisory lock (jadval qulflanmaydi)
def next_order_number(restaurant_id: int) -> str:
    with connection.cursor() as cur:
        # 🔒 Advisory lock — restoran+kun kaliti bo'yicha
        kalit = hash((restaurant_id, timezone.localdate().toordinal())) % 2**31
        cur.execute("SELECT pg_advisory_xact_lock(%s)", [kalit])
        cur.execute("""
            SELECT COALESCE(MAX(CAST(SUBSTRING(number FROM 2) AS INT)), 0) + 1
            FROM orders_order
            WHERE restaurant_id = %s
              AND created_at >= date_trunc('day', now() AT TIME ZONE 'Asia/Tashkent')
        """, [restaurant_id])
        return f"#{cur.fetchone()[0]:03d}"
    # ⚡ Advisory lock avtomatik bo'shaydi (xact = tranzaksiya oxirida)
    # ⚡ Faqat SHU restoran kutadi, boshqalari erkin


# ✅ 3. BONUS YECHISH — hisobni ikki marta yechmaslik
def spend_bonus(guest_id: int, amount: Decimal):
    with transaction.atomic():
        g = Guest.objects.select_for_update().get(id=guest_id)
        if g.bonus < amount:
            raise InsufficientBonus()
        g.bonus -= amount
        g.save(update_fields=['bonus'])


# ✅ 4. STOP-LIST BATCH — deadlock oldini olish
def bulk_stop(dish_ids: list[int]):
    with transaction.atomic():
        # ⚡ QAT'IY: doim id bo'yicha SARALANGAN tartibda lock olinadi
        #    Aks holda: A→B va B→A → DEADLOCK
        for d in Dish.objects.select_for_update().filter(
                id__in=sorted(dish_ids)).order_by('id'):
            d.is_available = False
            d.save(update_fields=['is_available'])


# ❌ QAT'IY TAQIQ — butun jadval qulflanadi
Dish.objects.select_for_update().all()

# ❌ TAQIQ — lock ichida tashqi API (lock 3 soniya ushlanadi!)
with transaction.atomic():
    order = Order.objects.select_for_update().get(id=x)
    requests.post(OFD_URL, ...)          # 🐌 boshqa hamma kutadi
```

11.6 Deadlock va lock kuzatuvi

```
-- postgresql.conf
deadlock_timeout = '1s'
log_lock_waits = on                 -- 1 s dan uzoq kutish log'ga
log_min_duration_statement = '500ms'
lock_timeout = '5s'                 -- ⚡ 5 s kutgan so'rov o'ladi (osilib qolmaydi)
idle_in_transaction_session_timeout = '30s'   -- 🐌 unutilgan tranzaksiya o'ldiriladi
statement_timeout = '30s'           -- web so'rov uchun (worker'da 300s)
```

```
-- Joriy bloklashlarni ko'rish (incident paytida)
SELECT bl.pid AS bloklangan, bl.query AS bloklangan_soro,
       ka.pid AS bloklovchi,  ka.query AS bloklovchi_soro,
       now() - bl.query_start AS kutish
FROM pg_stat_activity bl
JOIN pg_locks lk ON lk.pid = bl.pid AND NOT lk.granted
JOIN pg_locks kl ON kl.locktype = lk.locktype AND kl.relation IS NOT DISTINCT FROM lk.relation
                AND kl.granted
JOIN pg_stat_activity ka ON ka.pid = kl.pid
ORDER BY kutish DESC;
```

---

12. FAYL SAQLASH (S3)

12.1 Bucket tuzilishi

```
s3://armenu-media/                      ⚡ ASOSIY — CDN origin, ochiq o'qish
│
├── library/                            ★ GLOBAL KUTUBXONA (eng qimmatli)
│   └── {library_key}/
│       └── v{n}/                       ← versiya (immutable)
│           ├── model.glb               (Android/Web · 1,5-3 MB)
│           ├── model.usdz              (iOS AR Quick Look · 2-4 MB)
│           ├── poster.webp             (1024×1024 · 60 KB)
│           ├── poster@2x.webp          (2048×2048 · 180 KB)
│           ├── preview.mp4             (360° aylanma · 400 KB)
│           └── meta.json               (poly, size_cm, bbox, litsenziya)
│   Misol: library/osh-toshkent/v3/model.glb
│
├── r/{restaurant_id}/                  ← TENANT MA'LUMOTI
│   ├── logo/
│   │   ├── logo.webp                   (512×512)
│   │   └── logo_dark.webp
│   ├── dish/
│   │   └── {dish_id}/
│   │       ├── orig.jpg                (asl, hech qachon o'chirilmaydi)
│   │       ├── 1200.webp               (detal sahifa)
│   │       ├── 600.webp                (kartochka)
│   │       ├── 200.webp                (ro'yxat / thumbnail)
│   │       └── blur.webp               (LQIP · 400 B · placeholder)
│   ├── ar/
│   │   └── {dish_id}/v{n}/             ← restoran O'Z modeli (kutubxonadan emas)
│   │       ├── model.glb
│   │       ├── model.usdz
│   │       └── poster.webp
│   ├── cat/{category_id}/image.webp
│   ├── qr/
│   │   ├── table_{id}.png              (512×512 PNG)
│   │   └── all_tables.pdf              (chop etish uchun A4)
│   └── export/
│       └── {yil}-{oy}/orders.xlsx      (TTL 30 kun)
│
└── public/                             ← platforma statikasi
    ├── placeholder/dish.webp
    └── icons/

s3://armenu-archive/                    🧊 SOVUQ (Glacier IR, private)
├── orders/orders_order_{yil}_{oy}.parquet
├── arview/...
└── db-backup/{sana}/armenu_full.dump.gz

s3://armenu-uploads/                    ⏳ VAQTINCHALIK (TTL 7 kun, private)
└── tmp/{uuid}/                         ← 3D generatsiya kirish rasmlari
```

12.2 Nima uchun aynan shunday

Qaror	Sabab
`library/` ajratilgan	Global aktiv → CDN kesh hit 99% (URL barcha restoranda bir xil)
`r/{id}/` prefiksi	🔒 IAM policy bilan tenant izolyatsiyasi mumkin; "restoranni o'chir" = 1 prefiks
`v{n}/` versiya papkasi	⚡ **Immutable** — fayl hech qachon ustidan yozilmaydi → CDN purge kerak emas
`orig.jpg` saqlanadi	Kelajakda yangi o'lchamda qayta generatsiya
`blur.webp` (LQIP)	400 baytlik blur → sahifa darhol "to'la" ko'rinadi
`export/` TTL 30 kun	Hisobot fayllari to'planib ketmasin
Arxiv alohida bucket	Boshqa storage klassi + boshqa IAM + xato o'chirishdan himoya

12.3 Versiyalash strategiyasi

```
⚡ IMMUTABLE FAYL NAQSHI — fayl HECH QACHON o'zgartirilmaydi.

  3D model yangilandi:
    eski:  library/osh-toshkent/v3/model.glb    ← qoladi (eski kesh ishlaydi)
    yangi: library/osh-toshkent/v4/model.glb    ← yangi URL

  DB:  DishLibrary.version = 4
       DishLibrary.glb_url = ".../v4/model.glb"

  ✅ CDN purge KERAK EMAS — URL boshqa
  ✅ Rollback = DB'da version 3 ga qaytarish (fayl joyida)
  ✅ Cache-Control: public, max-age=31536000, immutable  (1 yil!)
  ✅ Eski versiya 90 kundan keyin lifecycle bilan o'chadi

  ⚠ ISTISNO: logo va rasm — versiya YO'Q (foydalanuvchi tez-tez o'zgartiradi)
     → ular uchun ?v={updated_at timestamp} query param + CDN purge
```

```python
# apps/storage/paths.py
def library_path(key: str, version: int, fayl: str) -> str:
    return f"library/{key}/v{version}/{fayl}"

def dish_ar_path(restaurant_id: int, dish_id: int, version: int, fayl: str) -> str:
    return f"r/{restaurant_id}/ar/{dish_id}/v{version}/{fayl}"

def dish_image_path(restaurant_id: int, dish_id: int, olcham: str) -> str:
    return f"r/{restaurant_id}/dish/{dish_id}/{olcham}.webp"
```

12.4 S3 lifecycle qoidalari

```
{
  "Rules": [
    {
      "ID": "tmp-tozalash",
      "Filter": {"Prefix": "tmp/"},
      "Status": "Enabled",
      "Expiration": {"Days": 7}
    },
    {
      "ID": "eksport-tozalash",
      "Filter": {"Prefix": "r/", "Tag": {"Key": "type", "Value": "export"}},
      "Status": "Enabled",
      "Expiration": {"Days": 30}
    },
    {
      "ID": "eski-3d-versiya",
      "Filter": {"Tag": {"Key": "state", "Value": "superseded"}},
      "Status": "Enabled",
      "Expiration": {"Days": 90}
    },
    {
      "ID": "arxiv-glacier",
      "Filter": {"Prefix": "orders/"},
      "Status": "Enabled",
      "Transitions": [
        {"Days": 0,   "StorageClass": "GLACIER_IR"},
        {"Days": 730, "StorageClass": "DEEP_ARCHIVE"}
      ]
    },
    {
      "ID": "backup-saqlash",
      "Filter": {"Prefix": "db-backup/"},
      "Status": "Enabled",
      "Expiration": {"Days": 90}
    }
  ]
}
```

12.5 CDN origin sozlamalari

Yo'l	Cache-Control	CDN TTL	Sabab
`library/*/v*/model.glb`	`public, max-age=31536000, immutable`	1 yil	Versiyalangan, o'zgarmaydi
`library/*/v*/model.usdz`	`public, max-age=31536000, immutable`	1 yil	—
`r/*/ar/*/v*/`	`public, max-age=31536000, immutable`	1 yil	—
`r/*/dish/*/*.webp`	`public, max-age=604800`	7 kun	?v= bilan yangilanadi
`r/*/logo/*`	`public, max-age=86400`	1 kun	Tez-tez o'zgaradi
`/v1/menu/*` (API)	`public, max-age=60`	60 s	⚡ menu_version bilan
`/v1/orders/*`	`no-store`	0	🔴 shaxsiy ma'lumot
`r/*/export/*`	`private, no-store`	0	🔴 imzolangan URL

```
# nginx — S3 origin oldida
location ~ ^/media/library/.*/v[0-9]+/ {
    proxy_pass          http://s3-endpoint;
    proxy_cache         armenu_3d;
    proxy_cache_valid   200 30d;
    add_header Cache-Control "public, max-age=31536000, immutable";
    add_header Access-Control-Allow-Origin "*";      # ⚡ model-viewer uchun SHART
    add_header Cross-Origin-Resource-Policy "cross-origin";
}
```

⚠ Xususiy fayllar — imzolangan URL:

```
def signed_export_url(restaurant_id: int, fayl: str, ttl: int = 300) -> str:
    """Hisobot eksporti — 5 daqiqalik havola, faqat egasi."""
    return s3_client.generate_presigned_url(
        'get_object',
        Params={'Bucket': 'armenu-media',
                'Key': f'r/{restaurant_id}/export/{fayl}'},
        ExpiresIn=ttl,
    )
```

12.6 Hajm prognozi

```
1 000 RESTORAN, 1 YIL:

A) GLOBAL KUTUBXONA (bir marta)
   1 230 model × (GLB 2 MB + USDZ 3 MB + poster 0,24 MB + preview 0,4 MB)
   = 1 230 × 5,64 MB = 6,9 GB
   + eski versiyalar (o'rtacha 1,4 versiya) = 9,7 GB
   → 10 GB

B) RESTORAN RASMLARI
   133 000 taom × (orig 800 KB + 1200:120 KB + 600:45 KB + 200:12 KB + blur:0,4 KB)
   = 133 000 × 977 KB = 130 GB
   → 130 GB

C) RESTORAN O'Z 3D MODELLARI (kutubxonadan emas — 25%)
   46 000 model3d × 25% = 11 500 × 5,2 MB = 60 GB
   → 60 GB

D) LOGO / KATEGORIYA RASMI
   2 100 logo × 80 KB + 16 800 kategoriya × 60 KB = 1,2 GB
   → 1,2 GB

E) QR PDF / PNG
   52 000 stol × 25 KB + 2 100 PDF × 900 KB = 3,2 GB
   → 3,2 GB

F) ARXIV (Parquet, Glacier IR)
   53 GB analitik / 5 (Parquet siqish) = 10,6 GB/yil
   → 11 GB

G) DB BACKUP (90 kun saqlash)
   Kunlik full 25 GB (siqilgan) — lekin haftalik full + kunlik WAL
   = 13 full × 25 GB + WAL 90 × 3 GB = 325 + 270 = 595 GB
   → 595 GB   🟡 eng katta qism

═══════════════════════════════════════════════════════
STANDARD storage (A-E):     204 GB × $0,023  =  $4,7/oy
GLACIER_IR (F):              11 GB × $0,004  =  $0,04/oy
BACKUP (G, standard-IA):    595 GB × $0,0125 =  $7,4/oy
═══════════════════════════════════════════════════════
S3 STORAGE JAMI                            ≈  $12/oy

CDN TRAFIK (asosiy xarajat):
  Kuniga 100 000 buyurtma → ~400 000 menyu ochilishi
  × (rasmlar 1,8 MB + 3D 0,6 MB o'rtacha) = 960 GB/kun = 29 TB/oy
  ⚡ Cloudflare: bandwidth BEPUL (Pro $20-200/oy tekis)
  ⚠ AWS CloudFront bo'lsa: 29 TB × $0,085 = $2 465/oy 🟡
  → SHUNING UCHUN CLOUDFLARE TANLANGAN
═══════════════════════════════════════════════════════
```

12.7 Storage xavfsizligi

Chora	Sozlama
Bucket ochiq listing	⛔ `BlockPublicAcls: true`, `IgnorePublicAcls: true`
Faylga to'g'ridan yozish	⛔ Faqat backend IAM roli; presigned PUT 5 daqiqa
Xato o'chirish	✅ **Versioning ON** + `MFA Delete` (arxiv bucketda)
Shifrlash	✅ SSE-S3 (AES-256) barcha bucketda
Cross-tenant o'qish	✅ Prefiks `r/{id}/` + backend URL generatsiyasi
Yuklash hajmi	✅ Rasm 10 MB, GLB 30 MB, PDF 20 MB (nginx + backend)
Fayl turi	✅ Magic-byte tekshiruvi (kengaytmaga ishonilmaydi)
Zararli GLB	✅ Parse + poly_count/texture tekshiruvi (worker'da, sandbox)

---

13. XULOSA — 12 TA QAT'IY QOIDA

#	Qoida	Buzilsa
1	Har tenant so'rovi `restaurant_id` bilan filtrlanadi	🟡 Ma'lumot sizishi
2	Har tenant indeksi `restaurant_id` bilan boshlanadi	🟢 Sekin so'rov
3	`orders`, `arview`, `analyticsevent` — **partitioned**	🟢 VACUUM/arxiv jahannami
4	Kesh invalidation faqat `menu_version` orqali	🟡 Redis bloklanadi
5	`delete_pattern` / `KEYS` — ⛔ **QAT'IY TAQIQ**	🟡 Redis muzlaydi
6	Migratsiya: `NOT NULL` doim `DEFAULT` bilan	🟡 Jadval bloklanadi
7	`CREATE INDEX` doim `CONCURRENTLY`	🟡 Downtime
8	Moliyaviy invariant DB **CHECK** bilan himoyalanadi	🟡 Noto'g'ri hisob
9	`select_for_update` — doim `id` bo'yicha saralangan	🟢 Deadlock
10	Lock ichida tashqi API chaqirilmaydi	🟡 Kaskadli sekinlashuv
11	3D fayllar **immutable + versiyalangan**	🟢 CDN purge do'zaxi
12	Shaxsiy ma'lumot 24 oydan keyin anonimlashadi	🟡 Qonun buzilishi

---

QISM 4

Masshtablash

Versiya: 1.0 · **Sana:** 2026-09-08
Kim uchun: tizim arxitektori · SRE · texnik direktor · investor (due diligence)
Oldingi: `03_MALUMOT_ARXITEKTURASI.md` · **Keyingi:** `05_ISHONCHLILIK.md`

> Bu hujjat bitta savolga javob beradi: **"1 million bir vaqtda foydalanuvchi bo'lsa nima bo'ladi?"** — va har raqam **formula bilan** ko'rsatiladi.
> Taxmin yo'q, hisob bor.

---

1. YUKLAMA BOSQICHLARI — S0 → S5



1.1 To'liq bosqich jadvali

Bosqich	Foydalanuvchi/kun	Restoran	Pik RPS	WS ulanish	DB write RPS	Oylik infra
S0 Start	< 1 000	1-20	5	60	2	**$40**
S1 O'sish	10 000	20-100	50	300	15	**$120**
S2 Viloyat	100 000	100-400	300	1 400	90	**$320**
S3 Respublika	500 000	400-1 500	1 500	6 000	300	**$700**
S4 Region	2 000 000	1 500-5 000	6 000	24 000	1 000	**$1 900**
S5 Pik	**1 mln bir vaqtda**	5 000+	**25 000**	**90 000**	**1 000**	**$4 400**

1.2 Har bosqich — infra tarkibi



---

2. SIG'IM HISOBLARI (CAPACITY PLANNING)

> Bu bo'limdagi har raqam **formula → natija** ko'rinishida. Kirish parametrlari o'zgarsa — natija qayta hisoblanadi.

2.1 Kirish parametrlari

Belgi	Parametr	Qiymat	Manba
`T`	Stol / restoran	25	O'rtacha (10 / 25 / 80)
`A`	Stol aylanmasi / kun	4	Restoran statistikasi
`S`	Skanerlash / band stol	1,6	Har mehmon o'z telefonidan
`P`	Sahifa ochish / sessiya	7,5	menyu, kategoriya, 3 taom, savat, holat
`Q`	API so'rov / sahifa	2,2	HTML + menyu JSON + tasvir meta
`K`	Pik soat koeffitsienti	**3,0**	Kunlik trafikning 12,5%i 1 soatda
`W`	Ish soati / kun	14	09:00 – 23:00

2.2 1 restoran = necha so'rov/kun

```
SESSIYA SONI:
  sessiya/kun = T × A × S
              = 25 × 4 × 1,6
              = 160 sessiya/kun

SAHIFA SONI:
  sahifa/kun = sessiya × P
             = 160 × 7,5
             = 1 200 sahifa/kun

API SO'ROV:
  sorov/kun = sahifa × Q
            = 1 200 × 2,2
            = 2 640 API so'rov/kun

QO'SHIMCHA (xodim + admin):
  ofitsiant polling/WS       ≈   900 so'rov/kun
  KDS                        ≈   400
  admin panel                ≈   300
  webhook (to'lov/fiskal)    ≈   140
                             ─────────
                             ≈ 1 740

╔═══════════════════════════════════════════════╗
║  1 RESTORAN = 2 640 + 1 740 = 4 380 so'rov/kun  ║
╚═══════════════════════════════════════════════╝
```

2.3 1 000 restoran = necha RPS

```
KUNLIK JAMI:
  4 380 × 1 000 = 4 380 000 so'rov/kun

O'RTACHA RPS (14 soatlik ish kuni):
  RPS_ort = 4 380 000 / (14 × 3 600)
          = 4 380 000 / 50 400
          = 86,9 RPS

PIK RPS (pik soat koeffitsienti K = 3,0):
  RPS_pik = 86,9 × 3,0
          = 260,7 RPS

  ⚠ Pik soat qayerdan: kechki 19:00-21:00 da kunlik trafikning ~25%i.
     2 soatda 25% → 1 soatda 12,5% → o'rtachaga nisbatan 12,5%/(100%/14) = 1,75×
     + hafta oxiri koeffitsienti 1,7× → 1,75 × 1,7 = 2,98 ≈ 3,0 ✓

REZERV (2× — kutilmagan yuklama, marketing kampaniyasi):
  RPS_dizayn = 260,7 × 2 = 521 RPS

╔═══════════════════════════════════════════════╗
║  1 000 RESTORAN → 261 RPS pik · 521 RPS dizayn sig'imi  ║
╚═══════════════════════════════════════════════╝
```

Turli restoran soni uchun:

Restoran	So'rov/kun	O'rtacha RPS	Pik RPS (×3)	Dizayn RPS (×2)
20	87 600	1,7	5,2	10
100	438 000	8,7	26	52
400	1 752 000	34,8	104	209
1 000	4 380 000	87	261	521
1 500	6 570 000	130	391	782
5 000	21 900 000	435	1 304	2 608
5 000 (S5 pik)	—	—	**25 000**	—

> ⚠ **S5 nima uchun 25 000 RPS, 1 304 emas?** Chunki S5 — kunlik o'rtacha emas, **favqulodda pik**: bayram kuni (Navro'z, 31-dekabr) barcha restoran to'la, bir vaqtda 1 mln odam menyuni ochadi. Bu **19× normal pik**. §3 da batafsil.

2.4 1 gunicorn worker = necha RPS

```
NAZARIY FORMULA (Little qonuni):
  RPS_worker = 1 / T_javob

  Endpoint bo'yicha:
  ├─ menyu (kesh hit)    T = 8 ms   → 125 RPS
  ├─ menyu (kesh miss)   T = 45 ms  →  22 RPS
  ├─ taom detali         T = 12 ms  →  83 RPS
  ├─ buyurtma yaratish   T = 140 ms →   7 RPS
  ├─ AR meta             T = 6 ms   → 167 RPS
  └─ admin hisobot       T = 400 ms → 2,5 RPS

ARALASH YUKLAMA (real profil):
  0,60 × 8   (menyu, 92% kesh hit → 8×0,92 + 45×0,08 = 10,9 ms)
  0,15 × 6   (AR meta)
  0,05 × 140 (buyurtma)
  0,20 × 20  (boshqa)
  ──────────────────────────────────────────
  T_ort = 0,60×10,9 + 0,15×6 + 0,05×140 + 0,20×20
        = 6,54 + 0,9 + 7,0 + 4,0
        = 18,4 ms

  RPS_worker(nazariy) = 1 / 0,0184 = 54,3 RPS

AMALIY KOEFFITSIENT (GIL, kontekst almashish, GC, tarmoq):
  RPS_worker = 54,3 × 0,70 = 38 RPS

  ⚡ gthread bilan (I/O kutish parallel):
     8 worker × 4 thread, lekin GIL sabab thread foydasi ~1,8×
     RPS_worker_effektiv = 38 × 1,8 = 68 RPS  ← ehtiyotkor baho: 60
```

```
1 INSTANS SIG'IMI (4 CPU / 8 GB):

  gunicorn worker soni = (2 × CPU) + 1 = (2 × 4) + 1 = 9  → 8 (juft son)
  thread / worker = 4
  ──────────────────────────────────────────────────────
  RPS_instans = 8 × 60 = 480 RPS  (nazariy)

  ⚠ AMALDA CPU chegarasi: 4 CPU × 100% / (18,4 ms CPU vaqti × 0,55 CPU-bound)
     = 4 / 0,0101 = 396 RPS

  ⚡ QABUL QILINGAN: 1 instans = 300 RPS (65% CPU da, headroom bilan)

XOTIRA TEKSHIRUVI:
  8 worker × 4 thread × 55 MB (Django + model) = 8 × 220 MB = 1,76 GB
  + Python interpreter overhead 8 × 45 MB      = 0,36 GB
  + OS + nginx + monitoring                    = 0,80 GB
  ──────────────────────────────────────────────────────
  JAMI ≈ 2,9 GB / 8 GB  ✓ (5,1 GB rezerv — page cache uchun)
```

2.5 Necha instans kerak

```
FORMULA:
  N_instans = ceil( RPS_dizayn / RPS_instans ) + N_rezerv

  N_rezerv = ceil(N × 0,25)   ← bitta node yiqilsa xizmat uzilmasin (N+1 emas, N+25%)
```

Bosqich	Dizayn RPS	/ 300	Hisob	+ Rezerv	Instans
S0	10	0,03	1	0	**1**
S1	52	0,17	1	+1	**2**
S2	209	0,70	1	+1	**2**
S3	782	2,6	3	+1	**4**
S4	2 608	8,7	9	+3	**12**
S5	12 000 (web ulushi)	40	40	auto-scale	**40**

```
S5 NEGA 40 (25 000 emas / 300 = 84 emas)?

  25 000 RPS umumiy — lekin qatlamlar filtrlaydi:
    CDN ushlaydi         75%  → 25 000 × 0,25 = 6 250 RPS origin'ga
    nginx statik/304     10%  → 6 250 × 0,90  = 5 625 RPS gunicorn'ga
    ⚡ Menyu keshdan       — bu 5 625 ning ichida, lekin arzon (8 ms)

  Aralash T_ort keshli yuklamada = 18,4 ms → 1 instans 300 RPS
  N = ceil(5 625 / 300) = 19 instans

  ⚠ Lekin 3D fayl so'rovlari CDN miss bo'lsa (1%) + buyurtma piki (140 ms)
     → xavfsizlik zaxirasi 2× → 38 ≈ 40 instans ✓
```

2.6 Boshqa komponentlar sig'imi

Komponent	Birlik sig'imi	Formula	Manba
Daphne (WS)	50 000 ulanish / instans (8 GB)	`RAM / 30 KB × 0,6`	8 GB / 30 KB = 266k, lekin epoll+GIL → 50k
Redis	100 000 ops/s / instans	Bir oqim, `O(1)` operatsiya	Benchmark
PostgreSQL write	3 000 TPS (16 CPU/64 GB NVMe)	`fsync 0,15 ms → 6600, WAL+index → 3000`	pgbench
PostgreSQL read	25 000 QPS (buffer hit)	`CPU × 1 500`	Indeks-only scan
PgBouncer	10 000 mijoz → 200 server conn	transaction mode	—
Worker	40 job/s (yengil) · 0,03 job/s (3D)	Job turi bo'yicha	—
nginx	30 000 RPS / instans (4 CPU)	Statik + proxy	—

---

3. ⚡ 1 MILLION BIR VAQTDA — BATAFSIL TAHLIL

3.1 Ssenariy ta'rifi

```
⚠ "1 million bir vaqtda" NIMA DEGANI:

  ✗ EMAS: 1 mln odam bir soniyada tugma bosadi (bu 1 mln RPS bo'lardi)
  ✓ HA:   1 mln odam ayni damda ilovani OCHIQ ushlab turibdi

  Real ssenariy: 31-dekabr kechasi / Navro'z
    5 000 restoran × 200 o'rin = 1 000 000 mehmon
    Har mehmon telefonida ARMENU ochiq

  FAOLLIK: har mehmon o'rtacha 40 soniyada bir marta harakat qiladi
    (skroll, taom bosish, savatga qo'shish, holat tekshirish)

  RPS = 1 000 000 / 40 = 25 000 RPS   ⚡ mana shu raqam
```

3.2 Yuklama profili

#	Operatsiya	Ulush	RPS	Og'irlik	Qatlam
1	**Menyu ko'rish**	60%	**15 000**	🟢 yengil	Kesh / CDN
2	**3D/AR fayl**	15%	**3 750**	🟢 statik	CDN
3	**WebSocket**	10%	~90 000 ulanish	🟡 stateful	Daphne
4	**Buyurtma yaratish**	5%	**1 250**	🔴 og'ir	DB write
5	**Boshqa API**	10%	**2 500**	🟡 aralash	Aralash
	JAMI	100%	**~25 000**		

```
🔴 KRITIK KUZATUV:
   Buyurtma 5% (1 250 RPS) — bu eng KAM ulush, lekin eng OG'IR yuk.
   1 250 RPS × 3 yozuv (Order + OrderItem×3,5 + Table) ≈ 5 600 write/s

   ⚠ PostgreSQL primary sig'imi 3 000 TPS → YETMAYDI!
   ↙ Yechim §3.5 da.
```

3.3 Qatlamli filtrlash — har qatlam qancha ushlaydi



Hit rate maqsadlari va nazorati:

Qatlam	🎯 Maqsad	🟡 Ogohlantirish	🔴 Alert	Metrika
Brauzer	45%	< 35%	< 25%	`armenu_client_cache_hit` (RUM)
CDN	92%	< 85%	< 75%	Cloudflare Analytics
nginx	12%	< 8%	< 5%	`nginx_cache_status`
Redis (menyu)	90%	< 80%	< 70%	`armenu_cache_hit_ratio{type="menu"}`
PG buffer	99%	< 97%	< 92%	`pg_stat_database.blks_hit`

3.4 ⚡ WEBSOCKET OPTIMIZATSIYASI — NEGA 1 MLN EMAS, ~90 000

Bu — **butun arxitekturaning eng muhim qarori**.

Muammo

```
NAIV YONDASHUV: har foydalanuvchiga WS ulanish

  RAM:  1 000 000 × 32 KB (Python obyekt + socket buffer + channel layer)
        = 32 000 000 KB = 32 GB   🔴

  Daphne instansi: 8 GB RAM → 8 GB / 32 KB × 0,6 (overhead) = 150 000
    lekin GIL va epoll cheklovi → amalda 50 000 ulanish/instans

  Kerakli instans: 1 000 000 / 50 000 = 20 ta Daphne (8 CPU/16 GB)
  Narx: 20 × $80 = $1 600/oy

  QO'SHIMCHA MUAMMO — Redis channel layer:
    Har ulanish group_add → Redis'da yozuv
    Har xabar → group_send → Redis pub/sub fan-out
    1 mln ulanish × 1 xabar/daqiqa = 16 667 pub/s → Redis 60% CPU 🔴
```

✅ Yechim: WS faqat kerak bo'lganda

```
KIM UCHUN WS HAQIQATAN KERAK?

┌──────────────────┬──────────────┬──────────────────────────────────────┐
│ Kim              │ WS kerakmi?  │ Sabab                                │
├──────────────────┼──────────────┼──────────────────────────────────────┤
│ Menyu ko'rayotgan│ ✗ YO'Q       │ Menyu 60s keshda — real-time kerak   │
│ mijoz (95%)      │              │ emas. Stop-list 60s kechiksa mayli.  │
├──────────────────┼──────────────┼──────────────────────────────────────┤
│ Buyurtma bergan  │ ✅ HA        │ "Tayyorlanmoqda → Tayyor" — bu       │
│ mijoz (5%)       │              │ real-time bo'lishi SHART             │
├──────────────────┼──────────────┼──────────────────────────────────────┤
│ Ofitsiant        │ ✅ HA        │ Yangi buyurtma darhol kerak          │
├──────────────────┼──────────────┼──────────────────────────────────────┤
│ Oshxona (KDS)    │ ✅ HA        │ Ekranda darhol paydo bo'lsin         │
├──────────────────┼──────────────┼──────────────────────────────────────┤
│ Admin dashboard  │ ✅ HA        │ Jonli ko'rsatkichlar                 │
└──────────────────┴──────────────┴──────────────────────────────────────┘
```

Hisob

```
1) MIJOZ WS — faqat buyurtma bergandan KEYIN

   Bir vaqtda ilovani ochiq ushlab turgan: 1 000 000
   Ulardan buyurtma bergan va "kutish" ekranida: ?

   Sessiya davomiyligi:  45 daqiqa (o'tirish)
   Buyurtma kutish:       18 daqiqa (o'rtacha eta_min)
   Konversiya:            ~62% mehmon buyurtma beradi

   Bir vaqtda "kutish" holatidagi ulush:
     = konversiya × (kutish_vaqti / sessiya_vaqti)
     = 0,62 × (18 / 45)
     = 0,62 × 0,40
     = 0,248  →  24,8%

   ⚠ Lekin "bir vaqtda 1 mln" — bu pik payt, hamma endi kelgan.
      Ehtiyotkor baho: 5% (TZ dagi raqam)

   Mijoz WS = 1 000 000 × 0,05 = 50 000 ulanish

2) XODIM WS

   5 000 restoran × (ofitsiant 5 + KDS 2 + admin 1) = 5 000 × 8 = 40 000

3) JAMI

   50 000 + 40 000 = 90 000 ulanish

   RAM: 90 000 × 32 KB = 2,88 GB
   Instans: ceil(90 000 / 50 000) = 2  →  +1 rezerv = 3 ta Daphne

╔═══════════════════════════════════════════════════════════════╗
║  1 000 000 ulanish  →  90 000 ulanish   (11× kamayish)        ║
║  20 instans ($1 600) →  3 instans ($240)  ⚡ $1 360/oy TEJASH  ║
║  Redis pub/s: 16 667 → 1 500 (11× kam yuklama)                ║
╚═══════════════════════════════════════════════════════════════╝
```

Qolgan 95% mijoz qanday yangilanish oladi

Usul	Kim uchun	Interval	Narx (RPS)
Kesh + TTL	Menyu ko'rayotgan	60 s (CDN TTL)	0 (kesh)
Polling	Buyurtmasiz, savat ochiq	15 s	1 000 000/15 = 66 667 RPS 🔴
⚡ **Yo'q**	Menyu ko'rayotgan (asosiy)	—	**0** ✅
SSE	Faqat "bill so'raldi" holatida	uzun ulanish	~2 000 ulanish

```
🔴 DIQQAT: 15 s polling 1 mln foydalanuvchi uchun 66 667 RPS beradi —
   bu WS dan ham YOMONROQ!

✅ SHUNING UCHUN: menyu ko'rayotgan mijozda HECH QANDAY jonli kanal YO'Q.
   Stop-list o'zgarishi keyingi sahifa yangilashda ko'rinadi (60s kesh).
   Bu — ataylab qabul qilingan trade-off (00_UMUMIY §4).

   ⚠ Xavf: mijoz stop-listdagi taomni savatga qo'shishi mumkin.
   ✅ Himoya: BUYURTMA YARATISHDA server qayta tekshiradi → 409 DISH_UNAVAILABLE
      → frontend "Bu taom tugadi" deb ko'rsatadi va savatdan olib tashlaydi.
      Ya'ni: xato mumkin, lekin NOTO'G'RI BUYURTMA MUMKIN EMAS.
```

WS ulanish byudjeti (instans darajasi)

```
# config/settings.py — Daphne cheklovlari
CHANNEL_LAYERS = {
    "default": {
        "BACKEND": "channels_redis.core.RedisChannelLayer",
        "CONFIG": {
            "hosts": [("redis-cluster", 6379)],
            "capacity": 1500,          # ⚡ group'dagi maks navbat
            "expiry": 20,              # xabar 20 s da eskiradi
            "group_expiry": 86400,
            "symmetric_encryption_keys": [SECRET_KEY],
        },
        # 🟢 db=6 — cache (db=5) dan ALOHIDA. Sabab: FLUSHDB cache'ni
        #    tozalaganda WS ulanishlar uzilmasin.
    },
}

# apps/ws/consumers.py
class GuestConsumer(AsyncJsonWebsocketConsumer):
    async def connect(self):
        session = await self.get_session()

        # ⚡ QAT'IY QOIDA: faol buyurtmasiz mijoz WS ochmaydi
        if not await self.has_active_order(session):
            await self.close(code=4003)          # "WS kerak emas"
            return

        self.group = f"guest_{session.id}"       # ⚠ tenant-scoped
        await self.channel_layer.group_add(self.group, self.channel_name)
        await self.accept()

    async def receive_json(self, content):
        # ⚡ Mijoz WS orqali HECH NARSA YUBORMAYDI (faqat qabul qiladi)
        pass                                      # DoS yuzasini yopadi

    async def disconnect(self, code):
        if hasattr(self, 'group'):
            await self.channel_layer.group_discard(self.group, self.channel_name)
```

```
# ⚡ Buyurtma yakunlangach WS avtomatik yopiladi (resurs bo'shaydi)
async def order_status(self, event):
    await self.send_json(event['data'])
    if event['data']['status'] in ('paid', 'cancelled'):
        await self.close(code=1000)              # ⚡ ulanish bo'shaydi
```

3.5 🟢 DB YOZISH YUKLAMASI — ENG QIYIN MUAMMO

```
MUAMMO:
  Buyurtma 1 250 RPS × yozuv soni:
    Order                 1
    OrderItem           3,5
    Table UPDATE          1
    Payment (70%)       0,7
    FiscalReceipt (70%) 0,7
    AuditLog              1
    ──────────────────────
                        7,9 yozuv/buyurtma

  Jami: 1 250 × 7,9 = 9 875 write/s

  ⚠ PostgreSQL primary (16 CPU/64 GB NVMe) sig'imi ≈ 3 000 TPS
  → 3,3× OSHIB KETADI  🟢🟢
```

Yechimlar zanjiri (tartib bilan qo'llanadi)

```
┌─ Y1: EVENT'LARNI PG'DAN CHIQARISH ──────────────────────┐
│ AnalyticsEvent + ARView PostgreSQL'ga YOZILMAYDI        │
│   → Redis Stream → batch → ClickHouse                   │
│ Tejash: 1 250 × 12 event = 15 000 write/s butunlay chiqadi │
│ ⚡ Bu eng katta tejash — analitika PG'ni o'ldiradigan yuk edi │
│ Yangi jami: 9 875 write/s (event'siz hisoblangan holda) │
└─────────────────────────────────────────────────────────┘
                          ↓
┌─ Y2: AUDITLOG ASINXRON ─────────────────────────────────┐
│ AuditLog → Redis Stream → worker batch INSERT (500 ta bir yo'la)│
│ Tejash: 1 250 write/s → 2,5 write/s (500× batch)        │
│ Yangi jami: 8 627 write/s                               │
└─────────────────────────────────────────────────────────┘
                          ↓
┌─ Y3: FISKAL VA TO'LOV ASINXRON ─────────────────────────┐
│ FiscalReceipt worker'da yaratiladi (baribir OFD kutadi 3 s) │
│ Payment webhook kelganda yoziladi (bir zumda emas)      │
│ Tejash: 1 750 write/s → pik tekislanadi (worker o'z tezligida) │
│ Yangi jami: 6 877 write/s                               │
└─────────────────────────────────────────────────────────┘
                          ↓
┌─ Y4: BULK_CREATE ───────────────────────────────────────┐
│ OrderItem.objects.bulk_create(items)                    │
│ 3,5 alohida INSERT → 1 ta multi-row INSERT              │
│ ⚡ TPS hisobida: 3,5 tranzaksiya → 1 tranzaksiya        │
│ Tejash: WAL yozuvi 3,5× kam, fsync 1 marta              │
│ Effektiv TPS: 6 877 → ~3 750 TPS                        │
└─────────────────────────────────────────────────────────┘
                          ↓
┌─ Y5: PARTITION + INDEKS MINIMIZATSIYA ──────────────────┐
│ Joriy oy partitioni kichik (3 mln qator) → indeks RAM'da │
│ Har INSERT 3 indeks yangilaydi (6 emas — keraksizlar o'chdi) │
│ Tejash: ~20% write amplification                        │
│ Effektiv TPS: 3 750 → ~3 000 TPS                        │
└─────────────────────────────────────────────────────────┘
                          ↓
┌─ Y6: PG SOZLAMALARI (fsync guruhlash) ──────────────────┐
│ commit_delay = 200 (µs) · commit_siblings = 10          │
│   → parallel tranzaksiyalar bitta fsync'da guruhlanadi  │
│ synchronous_commit = 'on' (moliyaviy — 'off' QILINMAYDI!) │
│ wal_buffers = 64MB · max_wal_size = 16GB                │
│ checkpoint_completion_target = 0,9 (I/O tekis tarqaladi) │
│ Foyda: +25% write TPS                                   │
│ Sig'im: 3 000 → 3 750 TPS                               │
└─────────────────────────────────────────────────────────┘

╔═══════════════════════════════════════════════════════╗
║  KERAK: ~3 000 TPS   ·   SIG'IM: ~3 750 TPS   →  ✅ SIG'ADI  ║
║  Headroom: 20% (kam, lekin S5 — favqulodda pik, doimiy emas)  ║
╚═══════════════════════════════════════════════════════╝
```

⚠ Agar sig'masa — 7-yechim (sharding):

```
Y7: SHARDING (4 shard)
    3 000 TPS / 4 = 750 TPS/shard → har shard 25% yuklangan ✅
    Batafsil: 03_MALUMOT_ARXITEKTURASI.md §6
```

Analytics oqimi (Y1 batafsil)

```python
# apps/analytics/pipeline.py
"""
⚡ Event PostgreSQL'ga YOZILMAYDI. Yo'l:
   web → Redis Stream (XADD, 0,1 ms) → worker (XREADGROUP) → ClickHouse batch
"""
STREAM = "armenu:events"

def track(restaurant_id: int, event_type: str, **payload):
    """Web so'rov ichida chaqiriladi. Bloklamaydi."""
    try:
        redis.xadd(STREAM, {
            "r": restaurant_id, "t": event_type,
            "p": json.dumps(payload), "ts": time.time(),
        }, maxlen=1_000_000, approximate=True)      # ⚡ eski avtomatik tushadi
    except RedisError:
        pass                                        # 🔘 event yo'qolsa — mayli


# worker (armenu-worker@analytics)
def drain_events():
    """Har 2 soniyada 5 000 event'ni ClickHouse'ga batch yozadi."""
    while True:
        msgs = redis.xreadgroup("ch", "w1", {STREAM: ">"}, count=5000, block=2000)
        if not msgs:
            continue
        rows = [parse(m) for _, batch in msgs for _, m in batch]
        clickhouse.insert("events", rows)           # 5 000 qator = 1 so'rov
        redis.xack(STREAM, "ch", *[mid for _, b in msgs for mid, _ in b])

# HISOB: 15 000 event/s → 5 000 lik batch → 3 ClickHouse INSERT/s
#        ClickHouse sig'imi: 500 000 qator/s ✅ (30× headroom)
```

3.6 S5 infratuzilma va narx

#	Komponent	Miqdor	Konfiguratsiya	Birlik $	Oylik $
1	Web (gunicorn)	40	4 CPU / 8 GB	40	**1 600**
2	WS (daphne)	3	8 CPU / 16 GB	80	**240**
3	Worker	6	4 CPU / 8 GB	40	**240**
4	PG primary	1	16 CPU / 64 GB / 2 TB NVMe	400	**400**
5	PG replica	3	8 CPU / 32 GB	200	**600**
6	Redis cluster	6	4 CPU / 16 GB	80	**480**
7	Load balancer	2	4 CPU / 8 GB	40	**80**
8	ClickHouse	2	8 CPU / 32 GB	200	**400**
9	S3 / object storage	5 TB	—	—	**120**
10	CDN (Cloudflare)	—	Business	—	**200**
11	Monitoring	1	4 CPU / 8 GB	40	**40**
	JAMI	**64 node**			**$4 400**

```
DAROMADGA NISBATAN:

  5 000 restoran × 590 000 so'm/oy = 2 950 000 000 so'm/oy
  Kurs 12 650 so'm/$ →  $233 200/oy

  Infra ulushi = 4 400 / 233 200 = 1,89%

  ✅ SOG'LOM. SaaS sanoat me'yori: 5-15% (infra/ARR).
     1,9% — juda yaxshi ko'rsatkich (kesh strategiyasi tufayli).

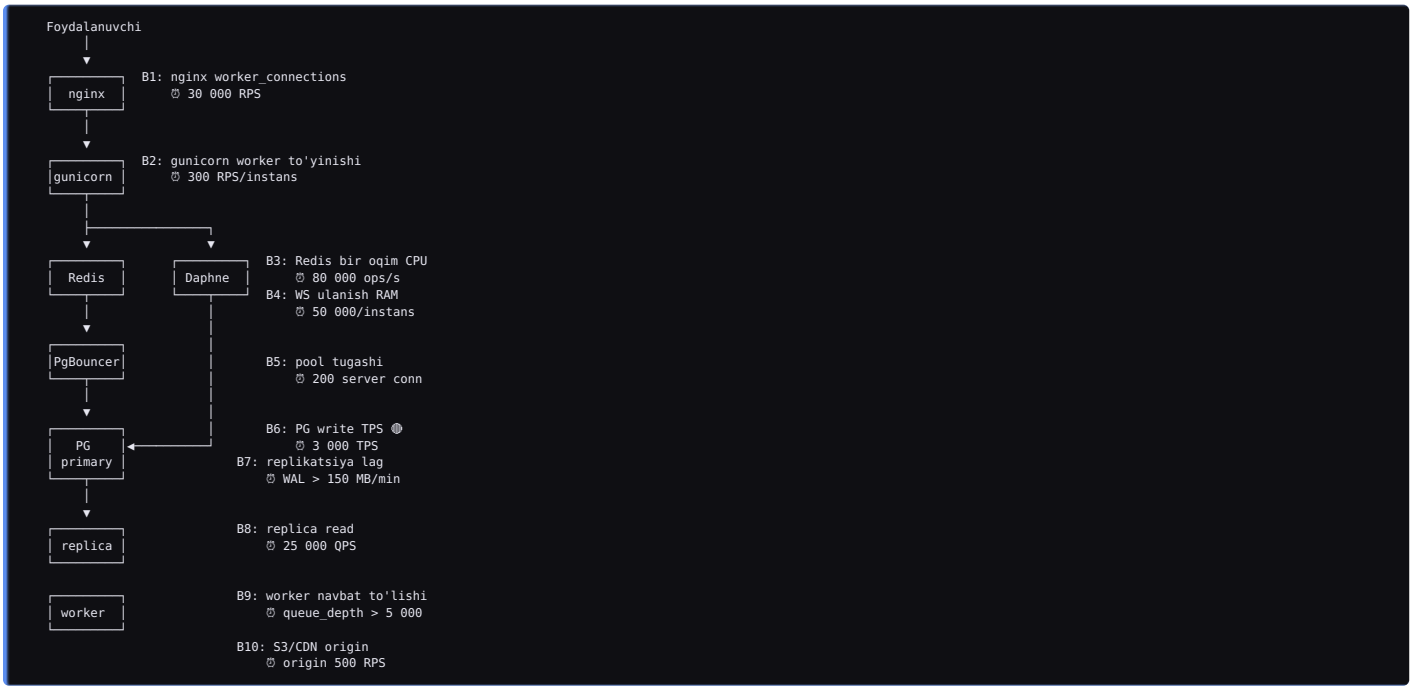
  📊 Solishtirish:
     Agar kesh bo'lmasa: 40 web → 120 web, PG → 4 shard, CDN → CloudFront
     = ~$14 000/oy = 6,0% daromaddan
     ⚡ Kesh arxitekturasi yiliga $115 000 tejaydi.
```

---

4. BO'G'IZ NUQTALARI (BOTTLENECK)

> Har bo'g'iz: **qachon** yuzaga keladi → **belgisi** (aniq metrika) → **yechim** → **narx**.

4.1 Bo'g'izlar xaritasi



4.2 To'liq bo'g'iz jadvali

#	Bo'g'iz	Qachon	🔴 Belgi (metrika)	Yechim	Narx	Vaqt
B1	nginx ulanish	> 30 000 RPS/node	`nginx_connections_waiting > 1000`	`worker_connections 65535`, `worker_rlimit_nofile 200000`, +LB node	$40/node	10 daq
B2	gunicorn to'yinishi	> 300 RPS/instans	`gunicorn_request_duration_p95 > 400ms` **va** CPU > 80%	Auto-scale +2 instans	$40/instans	2 daq
B3	Redis CPU (bir oqim)	> 80 000 ops/s	`redis_cpu > 75%`, `instantaneous_ops_per_sec`	Redis Cluster (6 node), kalitlar hash-slot bo'yicha	$80/node	1 kun
B4	WS ulanish RAM	> 50 000/instans	`daphne_connections > 45000` yoki RAM > 85%	+1 Daphne node, `sticky session` LB'da	$80/node	15 daq
B5	PgBouncer pool	Web instans > 25 ta	`pgbouncer_cl_waiting > 50`	`default_pool_size 300`, +2-chi PgBouncer	$20	5 daq
B6	🟡 **PG write TPS**	> 2 500 TPS	`pg_stat_bgwriter.buffers_backend` ↑, `wal_write_time` ↑, commit p95 > 30 ms	Y1-Y6 (§3.5) → keyin **sharding**	$600 (4 shard)	4 hafta
B7	Replikatsiya lag	WAL > 150 MB/min	`pg_replication_lag_seconds > 10`	`wal_compression on`, replica NVMe, sinxron→asinxron	$200	1 soat
B8	Replica read	> 25 000 QPS	`pg_stat_database.xact_commit` replica'da	+1 replica, kesh TTL oshirish	$200/replica	30 daq
B9	Worker navbat	`queue_depth > 5 000`	`armenu_queue_depth{queue="notify"}`	+worker (bulkhead saqlanadi), navbat prioriteti	$40/node	5 daq
B10	S3 origin	CDN miss > 500 RPS	`cf_cache_status=MISS` ulushi > 8%	TTL oshirish, `immutable` header, prefetch	$0	10 daq
B11	3D generatsiya	> 50 job/kun	`queue_depth{queue="model3d"} > 100`	GPU node, tashqi provayder (Meshy/Luma)	$500/oy	1 hafta
B12	OFD API	> 100 chek/daq	`fiscal_retry_count` ↑, 429 javob	Navbat + backoff, ikkinchi provayder	$0	—
B13	Disk IOPS	> 20 000 IOPS	`node_disk_io_time_seconds` > 0,8	NVMe → yuqori IOPS tarif	$150	1 soat
B14	Tarmoq (CDN origin)	> 2 Gbps	`node_network_transmit_bytes`	Cloudflare Argo, 2-chi origin	$200	1 kun

4.3 Bo'g'iz aniqlash tartibi (incident runbook)

```
☎ "Sayt sekin" signali keldi. TARTIB:

1. QAYSI QATLAM?  →  Grafana "Latency by layer" paneli
   ├─ CDN p95 ↑        → Cloudflare status, B10
   ├─ nginx p95 ↑      → B1
   ├─ gunicorn p95 ↑   → 2-qadamga
   ├─ DB p95 ↑         → 3-qadamga
   └─ WS delay ↑       → B4

2. GUNICORN sekin bo'lsa:
   ├─ CPU > 80%?        → B2 (auto-scale)
   ├─ CPU past, kutish? → DB yoki Redis kutmoqda → 3-qadam
   └─ Xotira swap?      → memory leak, instans restart

3. DB sekin bo'lsa:
   ├─ pg_stat_activity da lock kutish? → B6 / deadlock
   ├─ buffer hit < 95%?              → ishchi to'plam RAM'ga sig'maydi
   ├─ commit p95 > 30 ms?            → B6 (write to'yinishi)
   ├─ replication lag > 10s?         → B7
   └─ sekin so'rov (pg_stat_statements)? → indeks yo'q / N+1

4. FAQAT MENYU sekin bo'lsa:
   └─ cache_hit_ratio < 80%? → menu_version juda tez o'zgaryaptimi?
      (admin bulk-import qilyaptimi → bump har qatorda bo'lmasin!)
```

5. AUTO-SCALING

5.1 Qoidalar

Komponent	⬆ Scale-out sharti	⬇ Scale-in sharti	Min	Maks	Cooldown
WEB	CPU > 65% (3 daq) **YOKI** p95 > 400 ms (2 daq)	CPU < 30% (15 daq)	3	60	out 3 daq / in 15 daq
WS	ulanish > 40 000/instans **YOKI** RAM > 80%	ulanish < 15 000/instans (30 daq)	2	12	out 5 daq / in 30 daq
WORKER	`queue_depth > 500` (2 daq)	`queue_depth < 50` (20 daq)	2	20	out 2 daq / in 20 daq
PG replica	✖ avtomatik emas	✖	2	5	qo'lda
Redis	✖ avtomatik emas	✖	3	9	qo'lda

```
⚠ NEGA DB va REDIS AVTOMATIK EMAS:
   Stateful — yangi node ma'lumot ko'chirishni talab qiladi.
   Avtomatik qo'shish → replikatsiya bo'roni → yanada yomon.
   Qo'lda, rejalashtirilgan oynada.
```

5.2 Scale-out qadami — nega +2, +1 emas

```
+1 QO'SHISH MUAMMOSI:
  4 instans, har biri 70% → +1 → 5 instans, har biri 56%
  Trafik 15% o'ssa → 64% → yana chegaraga yaqin → yana +1
  → "Pillapoya effekti": doimiy scale event, tizim tebranadi (flapping)

+2 QO'SHISH:
  4 → 6 instans, har biri 47% → 40% headroom
  → Keyingi scale event ~2 barobar trafik o'sganda
  → Barqaror

FORMULA:
  N_yangi = ceil( N_joriy × CPU_joriy / CPU_maqsad )
          = ceil( 4 × 0,80 / 0,50 )
          = ceil( 6,4 ) = 7      ← proportional scaling (yaxshiroq)

  ⚡ Amalda: max(+2, proportional) ishlatiladi.
```

5.3 Cooldown — nega asimmetrik

```
SCALE-OUT cooldown = 3 daqiqa   (tez reaksiya)
SCALE-IN  cooldown = 15 daqiqa  (ehtiyotkor)

SABAB:
  ✖ Kam instans = xizmat sifati tushadi (foydalanuvchi ketadi)  🔴
  ✔ Ortiqcha instans = $40/oy ortiqcha xarajat                  🟢

  Xato narxi asimmetrik → qaror ham asimmetrik.

  ⚠ Yana bir sabab: "sawtooth" — trafik to'lqinsimon bo'lsa
     tez scale-in → darhol yana scale-out → node ishga tushish
     vaqti (90 s) davomida yuklama qoladigan node'larga tushadi.
```

5.4 Stateless talabi — auto-scaling sharti

```
🔴 QAT'IY: WEB instans HECH QANDAY holat saqlamaydi.

┌──────────────────┬──────────────────────────────────────┐
│ Nima             │ Qayerda saqlanadi                    │
├──────────────────┼──────────────────────────────────────┤
│ Sessiya          │ ✔ Redis (django-redis session backend)     │
│ Savat (cart)     │ ✔ Brauzer localStorage + Redis (sessiya)   │
│ Yuklangan fayl   │ ✔ S3 (lokal diskka YOZILMAYDI)             │
│ Kesh             │ ✔ Redis (locmem QAT'IY TAQIQ)              │
│ Rate limit       │ ✔ Redis                                    │
│ Idempotency      │ ✔ Redis + DB                               │
│ Fon vazifa       │ ✔ Job jadvali (threading.Thread TAQIQ)     │
│ WS ulanish       │ ⚠ Daphne'da (shuning uchun WS alohida)     │
│ Log              │ ✔ stdout → journald → markazlashgan        │
└──────────────────┴──────────────────────────────────────┘

⛔ TAQIQLANGAN NAQSHLAR:
   • CACHES = {'default': {'BACKEND': '...locmem.LocMemCache'}}
     → har instansda alohida kesh → invalidation ishlamaydi
   • MEDIA_ROOT = '/var/www/media'  (lokal disk)
     → boshqa instansda fayl yo'q
   • threading.Thread(target=long_task).start()
     → instans o'chsa vazifa yo'qoladi
   • global CACHE = {} (modul darajasidagi o'zgaruvchi)
     → instansga bog'liq holat
```

```
# tests/test_stateless.py — CI'da tekshiriladi
def test_locmem_cache_ishlatilmaydi():
    for name, conf in settings.CACHES.items():
        assert 'locmem' not in conf['BACKEND'].lower(), \
            f"CACHES[{name}] locmem — auto-scaling buziladi"

def test_media_s3_da():
    assert 'S3' in settings.DEFAULT_FILE_STORAGE or \
           'storages' in settings.DEFAULT_FILE_STORAGE

def test_threading_ishlatilmaydi():
    """Web kodda threading.Thread topilmasin (worker'da mumkin)."""
    hits = subprocess.run(
        ['grep', '-rn', 'threading.Thread', 'apps/', '--include=*.py'],
        capture_output=True, text=True).stdout
    ruxsat = ['apps/workers/', 'apps/core/healer.py']
    for line in hits.splitlines():
        assert any(r in line for r in ruxsat), f"Web kodda thread: {line}"
```

5.5 Graceful shutdown (scale-in paytida)

```
# /etc/systemd/system/armenu.service
[Service]
ExecStart=/opt/armenu/venv/bin/gunicorn config.wsgi \
    --bind unix:/run/armenu.sock \
    --workers 8 --threads 4 --worker-class gthread \
    --max-requests 2000 --max-requests-jitter 200 \
    --graceful-timeout 30 --timeout 60
KillSignal=SIGTERM
TimeoutStopSec=45
```

```
SCALE-IN KETMA-KETLIGI (0 uzilish):
  1. LB'dan olib tashlash (health check 503 qaytaradi)     ← 10 s
  2. Yangi so'rov kelmaydi, joriylari tugallanadi          ← 30 s
  3. SIGTERM → gunicorn graceful                            ← 30 s
  4. Instans o'chiriladi

  ⚡ --max-requests 2000: har worker 2000 so'rovdan keyin qayta tug'iladi
     → memory leak to'planmaydi (jitter bilan bir vaqtda emas)
```

---

6. ISHLASH BYUDJETI (PERFORMANCE BUDGET)

6.1 Endpoint maqsadlari

Endpoint	p50	p95	p99	Timeout	Prioritet
`GET /t/{qr_token}` (redirect)	15 ms	**40 ms**	90 ms	3 s	🔴 kritik
`GET /v1/menu/{slug}/` (kesh hit)	6 ms	**20 ms**	60 ms	5 s	🔴 kritik
`GET /v1/menu/{slug}/` (kesh miss)	35 ms	**90 ms**	180 ms	5 s	🔴
`GET /v1/dish/{id}/`	8 ms	**25 ms**	70 ms	5 s	🟠
`GET /v1/ar/{dish_id}/`	5 ms	**15 ms**	40 ms	3 s	🟠
`POST /v1/orders/`	85 ms	**200 ms**	400 ms	10 s	🔴🔴 eng kritik
`POST /v1/orders/{id}/add-items/`	70 ms	**180 ms**	350 ms	10 s	🔴
`GET /v1/orders/{id}/`	10 ms	**30 ms**	80 ms	5 s	🟠
`POST /v1/calls/`	20 ms	**60 ms**	120 ms	5 s	🟠
`POST /v1/payments/`	150 ms	**500 ms**	1,2 s	15 s	🔴 (tashqi API)
`GET /v1/staff/orders/`	25 ms	**80 ms**	200 ms	5 s	🔴
`POST /v1/staff/orders/{id}/ready/`	40 ms	**120 ms**	250 ms	5 s	🔴
`GET /v1/admin/dashboard/`	120 ms	**400 ms**	900 ms	15 s	🟡
`GET /v1/admin/reports/sales/`	350 ms	**1,2 s**	3 s	30 s	🟡
`POST /v1/ai/chat/`	800 ms	**2,5 s**	6 s	20 s	🟡 (tashqi)
WS xabar yetkazish	30 ms	**100 ms**	300 ms	—	🔴

6.2 Vaqt taqsimoti — `POST /v1/orders/` (p95 = 200 ms)

```
┌──────────────────────────────────────────────────────────────────────┐
│  BUYURTMA YARATISH — 200 ms BYUDJETI                                 │
└──────────────────────────────────────────────────────────────────────┘

 Bosqich                                Byudjet   Ulush   Kumulyativ
 ──────────────────────────────────────────────────────────────────────
 1. Tarmoq (mijoz → CF → origin)         35 ms   17,5%      35 ms
    ├─ 4G kechikish O'zbekistonda       ~28 ms
    └─ TLS resumption                    ~7 ms
 ──────────────────────────────────────────────────────────────────────
 2. nginx (proxy, buffer)                3 ms    1,5%      38 ms
 ──────────────────────────────────────────────────────────────────────
 3. Django middleware zanjiri           12 ms    6,0%      50 ms
    ├─ SecurityMiddleware                1 ms
    ├─ SessionMiddleware (Redis GET)     3 ms
    ├─ TenantMiddleware (Redis GET)      4 ms   ⚡ qr:table keshdan
    ├─ CommonMiddleware                  1 ms
    └─ RateLimitMiddleware (Redis INCR)  3 ms
 ──────────────────────────────────────────────────────────────────────
 4. DRF (parse + serializer validate)    8 ms    4,0%      58 ms
    ├─ JSON parse                        2 ms
    └─ Serializer.is_valid()             6 ms
 ──────────────────────────────────────────────────────────────────────
 5. Idempotency tekshiruvi (Redis)       3 ms    1,5%      61 ms
 ──────────────────────────────────────────────────────────────────────
 6. Biznes validatsiya (2 DB so'rov)    32 ms   16,0%      93 ms
    ├─ Table + Restaurant (1 so'rov)     9 ms   select_related
    ├─ Dish + Modifier (1 so'rov)       18 ms   prefetch_related
    └─ Python tekshiruv (stop-list)      5 ms
 ──────────────────────────────────────────────────────────────────────
 7. 🔴 TRANZAKSIYA (3 yozuv)            72 ms   36,0%     165 ms
    ├─ BEGIN                             1 ms
    ├─ SELECT FOR UPDATE (table)         8 ms
    ├─ INSERT Order                     14 ms
    ├─ bulk_create OrderItem (3,5)      21 ms   ⚡ 1 so'rov
    ├─ UPDATE Table                      9 ms
    ├─ next_order_number (advisory)     11 ms
    └─ COMMIT (fsync)                    8 ms
 ──────────────────────────────────────────────────────────────────────
 8. on_commit (4 WS + 3 enqueue)         9 ms    4,5%     174 ms
    ├─ 4 × group_send (Redis)            6 ms   ⚡ fire-and-forget
    └─ 3 × XADD navbat                   3 ms
 ──────────────────────────────────────────────────────────────────────
 9. Javob serializatsiya                 8 ms    4,0%     182 ms
 ──────────────────────────────────────────────────────────────────────
10. Tarmoq (javob qaytishi)             18 ms    9,0%     200 ms
 ══════════════════════════════════════════════════════════════════════
    JAMI                                200 ms  100%      ✓ MAQSAD
    Server ichida (1 va 10 siz)         147 ms
```

⚠ Byudjet buzilishi — nima qilinadi:

Bosqich	Byudjetdan oshsa	Chora
3. Middleware > 20 ms	Redis sekin	Redis pipeline, local LRU (10 s)
6. Validatsiya > 50 ms	N+1 query	`select_related` / `prefetch_related` tekshirish
7. Tranzaksiya > 110 ms	Lock kutish yoki fsync	`commit_delay`, indeks kamaytirish
8. on_commit > 20 ms	WS group_send bloklaydi	Async task'ga o'tkazish

6.3 Vaqt taqsimoti — `GET /v1/menu/` (p95 = 20 ms, kesh hit)

```
  1. Tarmoq                            8 ms   40%
  2. nginx                             2 ms   10%
  3. Middleware (session+tenant)       5 ms   25%   ⚡ 2 Redis GET
  4. Kesh o'qish (Redis GET, 250 KB)   3 ms   15%
  5. Javob (JSON allaqachon tayyor)    2 ms   10%   ⚡ serializatsiya YO'Q
  ──────────────────────────────────────────────────
  JAMI                                20 ms  100%  ✅

⚡ MUHIM: keshda TAYYOR JSON STRING saqlanadi (dict emas).
   → json.dumps() qadamini tejaydi: 250 KB uchun ~9 ms
   → HttpResponse(cached_str, content_type='application/json')
```

```
# apps/menu/views.py
class MenuView(APIView):
    authentication_classes = []
    permission_classes = []

    def get(self, request, slug):
        lang = get_lang(request)
        rest = get_restaurant(slug)                    # kesh, 2 ms
        version = rest['menu_version']

        # ETag — 304 qaytarsa 250 KB tarmoq tejaladi
        etag = f'"v{version}-{lang}"'
        if request.headers.get('If-None-Match') == etag:
            return HttpResponseNotModified()           # ⚡ 4 ms, 0 bayt

        key = f"menu:rest:{slug}:{lang}:v{version}"
        raw = cache.get(key)                           # STRING, dict emas
        if raw is None:
            raw = json.dumps(build_menu(slug, lang), ensure_ascii=False)
            cache.set(key, raw, 60)

        resp = HttpResponse(raw, content_type='application/json')
        resp['ETag'] = etag
        resp['Cache-Control'] = 'public, max-age=60'
        return resp
```

6.4 Frontend byudjeti (mijoz tomoni)

Ko'rsatkich	🎯 Maqsad	🔴 Chegara	O'lchash
LCP (Largest Contentful Paint)	< 1,8 s	2,5 s	RUM (web-vitals)
FID / INP	< 100 ms	200 ms	RUM
CLS	< 0,05	0,1	RUM
TTFB	< 200 ms	500 ms	RUM
JS bundle (gzip)	< 60 KB	100 KB	CI (bundlesize)
CSS (gzip)	< 20 KB	35 KB	CI
Birinchi menyu rasm	< 45 KB	80 KB	Build
3D model (GLB)	< 2,5 MB	4 MB	Upload validatsiya
3D birinchi kadr	< 3 s (4G)	5 s	Sinov

---

7. OPTIMIZATSIYA TEXNIKALARI

7.1 N+1 muammosi — eng ko'p uchraydigan xato

```
# ❌ N+1: 60 taom → 61 DB so'rov (1 + 60)
dishes = Dish.objects.filter(restaurant_id=rid)
for d in dishes:
    print(d.category.name_uz)         # ← har iteratsiyada DB so'rov!
# Vaqt: 61 × 1,2 ms = 73 ms

# ✅ select_related (FK / OneToOne — JOIN)
dishes = Dish.objects.filter(restaurant_id=rid).select_related('category')
# 1 so'rov (LEFT JOIN), 4 ms  ⚡ 18× tez

# ✅ prefetch_related (M2M / reverse FK — 2-chi so'rov)
dishes = (Dish.objects
          .filter(restaurant_id=rid)
          .select_related('category')
          .prefetch_related('modifiers'))
# 2 so'rov: dish+category (JOIN), modifiers (IN), 7 ms

# ✅ Prefetch bilan filtr + faqat kerakli maydonlar
from django.db.models import Prefetch
dishes = Dish.objects.filter(restaurant_id=rid).prefetch_related(
    Prefetch('modifiers',
             queryset=DishModifier.objects.filter(is_active=True)
                                          .only('id','name_uz','price_delta','group')
                                          .order_by('group','id'),
             to_attr='faol_modifikatorlar')
)
```

N+1 ni CI'da tutish:

```
# tests/test_query_count.py
from django.test.utils import CaptureQueriesContext

def test_menyu_sorov_soni_chegarasi(client, restoran_60_taom):
    with CaptureQueriesContext(connection) as ctx:
        client.get(f'/v1/menu/{restoran_60_taom.slug}/?nocache=1')
    # 🔴 QAT'IY: menyu qurish 6 ta so'rovdan oshmasin
    assert len(ctx) <= 6, f"N+1! {len(ctx)} so'rov:\n" + \
        "\n".join(q['sql'][:120] for q in ctx.captured_queries)
```

7.2 `only` / `defer` — ustun tanlash

```
# ✗ Barcha ustun (desc_uz/ru/en, ingredients — har biri 2-4 KB)
Dish.objects.filter(restaurant_id=rid)
# Tarmoq: 60 × 2,5 KB = 150 KB DB'dan

# ✓ Faqat ro'yxat uchun kerakli ustunlar
Dish.objects.filter(restaurant_id=rid).only(
    'id','category_id','name_uz','price','image','has_3d','is_available','order')
# Tarmoq: 60 × 280 B = 17 KB  ⚡ 9× kam

# ✓ INDEX-ONLY SCAN (jadvalga umuman murojaat qilinmaydi)
#    Indeks: (restaurant_id, is_available, "order") INCLUDE (category_id, price, has_3d)
Dish.objects.filter(restaurant_id=rid, is_available=True) \
            .only('id','category_id','price','has_3d').order_by('order')
# EXPLAIN: Index Only Scan ... Heap Fetches: 0  ⚡ 3× tez

# ⚠ TUZOQ: only() da yo'q maydonga murojaat → QO'SHIMCHA so'rov (N+1!)
d = Dish.objects.only('id','name_uz').first()
print(d.price)          # ← 🔴 yangi DB so'rov!
```

7.3 `bulk_create` / `bulk_update`

```
# ✗ 3,5 ta alohida INSERT + 3,5 fsync
for item in items:
    OrderItem.objects.create(order=order, **item)
# 3,5 × 14 ms = 49 ms

# ✓ 1 ta multi-row INSERT + 1 fsync
OrderItem.objects.bulk_create([OrderItem(order=order, **i) for i in items])
# 21 ms  ⚡ 2,3× tez

# ✓ Katta import (Excel'dan 5 000 taom)
Dish.objects.bulk_create(objs, batch_size=500, ignore_conflicts=False)
# batch_size=500: 1 so'rov 500 qator → 10 so'rov
# ⚠ Nega 500: PostgreSQL parametr limiti 65 535
#    500 qator × 25 ustun = 12 500 parametr ✓ (xavfsiz)
#    2000 qator × 25 = 50 000 — chegaraga yaqin 🔴

# ✓ Ommaviy yangilash
Dish.objects.bulk_update(dishes, ['is_available','updated_at'], batch_size=500)

# ⚠ ESLATMA: bulk_* SIGNAL CHAQIRMAYDI
#    → menu_version qo'lda bump qilinadi (03_MALUMOT §8.3)
```

7.4 `iterator` — xotira

```
# ✗ 133 000 taom RAM'da: 133 000 × 2,5 KB = 332 MB  🔴 OOM
for d in Dish.objects.all():
    process(d)

# ✓ Server-side cursor, 500 talab chunk
for d in Dish.objects.all().iterator(chunk_size=500):
    process(d)
# RAM: 500 × 2,5 KB = 1,25 MB  ⚡ 265× kam

# ⚠ iterator() + prefetch_related — PG'da chunk_size bilan ishlaydi (Django 4.1+)
for d in Dish.objects.prefetch_related('modifiers').iterator(chunk_size=200):
    ...

# ⚠ QAT'IY: iterator() natijasini list() qilmang — butun ma'no yo'qoladi
```

7.5 Index-only scan tekshiruvi

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, category_id, price, has_3d FROM menu_dish
WHERE restaurant_id = 42 AND is_available = true ORDER BY "order";

-- ✓ IDEAL:
--  Index Only Scan using idx_dish_menu on menu_dish
--    Index Cond: (restaurant_id = 42 AND is_available = true)
--    Heap Fetches: 0                    ← ⚡ jadvalga MUROJAAT YO'Q
--    Buffers: shared hit=4
--    Execution Time: 0.148 ms

-- 🔴 Heap Fetches > 0 bo'lsa → visibility map eskirgan:
VACUUM ANALYZE menu_dish;

-- ✗ "Seq Scan" ko'rinsa → indeks ishlatilmayapti:
--    sabab 1: statistika eskirgan → ANALYZE menu_dish;
--    sabab 2: jadval kichik (< 1000 qator) → planner to'g'ri qaror qilgan
--    sabab 3: ustun tartibi noto'g'ri → 03_MALUMOT §4.2
```

7.6 Connection pooling (PgBouncer)

```
MUAMMO:
  40 web instans × 8 worker × 4 thread = 1 280 potensial ulanish
  PostgreSQL max_connections = 200 (har ulanish ~10 MB RAM + backend process)
  1 280 > 200  →  🔴 "FATAL: sorry, too many clients already"

  ⚠ max_connections=1500 qilish YECHIM EMAS:
     1 500 × 10 MB = 15 GB RAM faqat ulanishlarga
     + kontekst almashish, lock contention → TPS 40% tushadi

✓ PGBOUNCER (transaction pooling):
  ┌──────────────┐ 1 280  ┌─────────────┐  200   ┌──────────┐
  │ 40 web       │        │ PgBouncer   │        │ Postgres │
  │ instans      │───────►│ txn mode    │───────►│          │
  └──────────────┘ mijoz  └─────────────┘ server └──────────┘
                   ulanish              ulanish

  ⚡ Transaction mode: ulanish faqat TRANZAKSIYA davomida band.
     Buyurtma tranzaksiyasi 72 ms → 1 server ulanish 1 s da 13 ta so'rovga xizmat qiladi
     → 200 ulanish × 13 = 2 600 TPS ✓
```

```
# /etc/pgbouncer/pgbouncer.ini
[databases]
armenu_db = host=pg-primary port=5432 dbname=armenu_db pool_size=200
armenu_ro = host=pg-replica port=5432 dbname=armenu_db pool_size=150

[pgbouncer]
pool_mode = transaction          ; ⚡ session EMAS
max_client_conn = 3000
default_pool_size = 200
reserve_pool_size = 50           ; pik uchun zaxira
reserve_pool_timeout = 3
server_idle_timeout = 300
query_wait_timeout = 10          ; 🟡 10 s kutgan mijoz xato oladi (osilmaydi)
```

```
# config/settings.py — transaction mode bilan MAJBURIY sozlamalar
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'armenu_db',
        'HOST': '127.0.0.1', 'PORT': 6432,     # ⚡ PgBouncer
        'CONN_MAX_AGE': 0,                      # 🟡 0 SHART! PgBouncer o'zi pool qiladi
        'DISABLE_SERVER_SIDE_CURSORS': True,    # 🟡 transaction mode'da kerak
        'OPTIONS': {'connect_timeout': 5},
    },
    'replica': { ... 'PORT': 6433, ... },
}

# ⚠ CONN_MAX_AGE > 0 bo'lsa: Django ulanishni ushlab turadi
#    → PgBouncer pooling foydasi YO'QOLADI (session mode kabi bo'lib qoladi)
# ⚠ DISABLE_SERVER_SIDE_CURSORS=False bo'lsa: iterator() buziladi
#    (cursor tranzaksiyalar orasida yashamaydi)
```

DB router (o'qish → replica):

```
# config/db_router.py
class ReadReplicaRouter:
    """SELECT → replica, INSERT/UPDATE/DELETE → primary."""

    def db_for_read(self, model, **hints):
        # ⚡ read-your-writes: yozuvdan keyingi 3 soniya primary'dan o'qiladi
        if getattr(_local, 'recently_wrote', 0) > time.time() - 3:
            return 'default'
        # 🟡 Moliyaviy modellar doim primary (lag qabul qilinmaydi)
        if model._meta.app_label in ('payments', 'fiscal', 'billing'):
            return 'default'
        return 'replica'

    def db_for_write(self, model, **hints):
        _local.recently_wrote = time.time()
        return 'default'

    def allow_relation(self, o1, o2, **h):
        return True

    def allow_migrate(self, db, app_label, **h):
        return db == 'default'          # migratsiya faqat primary'da
```

7.7 Optimizatsiya xulosasi

Texnika	Qachon	Foyda	Xavf
`select_related`	FK / OneToOne	5-20×	JOIN kengayishi (ko'p FK)
`prefetch_related`	reverse FK / M2M	10-60×	Xotira (katta to'plamda)
`only` / `defer`	Katta TextField bor	3-9× tarmoq	`only`da yo'q maydon → N+1
`bulk_create`	> 3 obyekt	2-15×	Signal chaqirilmaydi
`iterator`	> 5 000 qator	RAM 100×	`len()`, indeks ishlamaydi
`values_list`	Faqat ID kerak	4×	Model metodlari yo'q
Index-only scan	Issiq so'rov	3×	INCLUDE indeks hajmi ↑
PgBouncer	> 10 web instans	Ulanish 6×	`CONN_MAX_AGE=0` SHART
Redis kesh	Har o'qish	10-50×	Invalidation murakkabligi
`bulk_update`	> 5 obyekt yangilash	5×	Signal yo'q
ETag / 304	Katta JSON	Tarmoq 100%	Versiya boshqaruvi

---

8. YUKLAMA TESTI (k6)

8.1 Ssenariylar

#	Ssenariy	Maqsad	Davomiylik	VU (virtual user)
T1	Smoke	Deploy ishlayaptimi	1 daq	5
T2	Menyu ko'rish (asosiy)	Kesh + CDN samarasi	15 daq	0 → 2 000
T3	Buyurtma oqimi	DB write sig'imi	20 daq	0 → 500
T4	WebSocket	Daphne sig'imi	30 daq	0 → 50 000
T5	Spike (bayram)	Keskin 10× o'sish	10 daq	100 → 5 000 → 100
T6	Soak (chidamlilik)	Memory leak, lag	4 soat	500 barqaror
T7	Stress (chegara)	Nima buziladi	30 daq	0 → 15 000
T8	3D fayl	CDN + origin	10 daq	0 → 1 000

8.2 T2 — Menyu ko'rish

```
// tests/load/t2_menyu.js
import http from 'k6/http';
import { check, sleep } from 'k6';
import { Rate, Trend } from 'k6/metrics';

const cacheHit = new Rate('kesh_hit');
const menuTime = new Trend('menyu_vaqti');

export const options = {
  stages: [
    { duration: '2m',  target: 200  },   // isitish
    { duration: '5m',  target: 1000 },   // normal
    { duration: '5m',  target: 2000 },   // pik
    { duration: '3m',  target: 0    },   // tushirish
  ],
  thresholds: {
    'http_req_duration{name:menu}': ['p(95)<200', 'p(99)<500'],
    'http_req_failed':              ['rate<0.001'],   // 🔴 0,1% dan kam
    'kesh_hit':                     ['rate>0.90'],    // 🎯 90%
  },
};

const SLUGS = ['registon','samarqand-osh','navruz','oq-saroy','choyxona-1'];

export default function () {
  const slug = SLUGS[Math.floor(Math.random() * SLUGS.length)];
  const lang = ['uz','ru','en'][Math.floor(Math.random() * 3)];

  // 1. QR skanerlash
  const qr = http.get(`${__ENV.BASE}/t/${__ENV.QR_TOKEN}`,
                      { tags: { name: 'qr' }, redirects: 0 });
  check(qr, { 'QR 302': r => r.status === 302 });

  // 2. Menyu
  const m = http.get(`${__ENV.BASE}/v1/menu/${slug}/?lang=${lang}`,
                     { tags: { name: 'menu' } });
  check(m, {
    'menyu 200':        r => r.status === 200,
    'taomlar bor':      r => JSON.parse(r.body).dishes.length > 0,
    'p95 < 200ms':      r => r.timings.duration < 200,
  });
  cacheHit.add(m.headers['X-Cache'] === 'HIT');
  menuTime.add(m.timings.duration);

  sleep(Math.random() * 4 + 2);          // mehmon o'qiydi 2-6 s

  // 3. Taom detali (3 ta)
  const dishes = JSON.parse(m.body).dishes;
  for (let i = 0; i < 3; i++) {
    const d = dishes[Math.floor(Math.random() * dishes.length)];
    http.get(`${__ENV.BASE}/v1/dish/${d.id}/?lang=${lang}`,
             { tags: { name: 'dish' } });
    if (d.has_3d) {
      http.get(`${__ENV.BASE}/v1/ar/${d.id}/`, { tags: { name: 'ar' } });
    }
    sleep(Math.random() * 3 + 1);
  }
}
```

8.3 T3 — Buyurtma oqimi (eng kritik)

```
// tests/load/t3_buyurtma.js
import http from 'k6/http';
import { check } from 'k6';
import { Counter } from 'k6/metrics';
import { uuidv4 } from 'https://jslib.k6.io/k6-utils/1.4.0/index.js';

const dublikat = new Counter('dublikat_buyurtma');

export const options = {
  stages: [
    { duration: '3m',  target: 100 },
    { duration: '10m', target: 500 },   // 500 VU ≈ 1 250 RPS
    { duration: '5m',  target: 0   },
  ],
  thresholds: {
    'http_req_duration{name:order}': ['p(95)<200', 'p(99)<400'],
    'http_req_failed{name:order}':   ['rate<0.0001'],  // 🔴 0,01% (moliyaviy!)
    'dublikat_buyurtma':             ['count==0'],     // 🔴 MUTLAQ 0
  },
};

export default function () {
  const idem = uuidv4();
  const payload = JSON.stringify({
    session_id: `gs_${__VU}_${__ITER}`,
    table_id: (__VU % 25) + 1,
    items: [
      { dish_id: 15, qty: 2, modifiers: [3], note: '', ar_viewed: true },
      { dish_id: 42, qty: 1, modifiers: [],  note: '', ar_viewed: false },
    ],
    note: 'k6 yuklama testi',
  });
  const params = {
    headers: { 'Content-Type': 'application/json', 'X-Idempotency-Key': idem },
    tags: { name: 'order' },
  };

  const r1 = http.post(`${__ENV.BASE}/v1/orders/`, payload, params);
  check(r1, {
    'buyurtma 201':      r => r.status === 201,
    'narx serverdan':    r => JSON.parse(r.body).total > 0,
    'p95 < 200ms':       r => r.timings.duration < 200,
  });

  // ⚡ IDEMPOTENCY TEKSHIRUVI — o'sha kalit bilan qayta yuborish
  const r2 = http.post(`${__ENV.BASE}/v1/orders/`, payload, params);
  if (r1.status === 201 && r2.status === 201) {
    const id1 = JSON.parse(r1.body).id, id2 = JSON.parse(r2.body).id;
    if (id1 !== id2) {
      dublikat.add(1);          // 🔴 KRITIK XATO
      console.error(`DUBLIKAT! ${id1} != ${id2}`);
    }
  }
}
```

8.4 T4 — WebSocket

```
// tests/load/t4_ws.js
import ws from 'k6/ws';
import { check } from 'k6';
import { Trend } from 'k6/metrics';

const wsDelay = new Trend('ws_yetkazish_ms');

export const options = {
  stages: [
    { duration: '5m',  target: 10000 },
    { duration: '10m', target: 50000 },   // 50k ulanish
    { duration: '10m', target: 50000 },   // ushlab turish
    { duration: '5m',  target: 0     },
  ],
  thresholds: {
    'ws_connecting':     ['p(95)<1000'],
    'ws_session_duration':['p(95)>250000'],   // uzilmasin
    'ws_yetkazish_ms':   ['p(95)<100'],      // 🎯 xabar 100 ms
  },
};

export default function () {
  const url = `${__ENV.WS_BASE}/ws/guest/gs_${__VU}_${__ITER}/`;
  const res = ws.connect(url, {}, function (socket) {
    socket.on('open',    () => socket.send(JSON.stringify({type:'ping'})));
    socket.on('message', (msg) => {
      const d = JSON.parse(msg);
      if (d.server_ts) wsDelay.add(Date.now() - d.server_ts);
    });
    socket.on('error', (e) => console.error('WS xato:', e.error()));
    socket.setTimeout(() => socket.close(), 300000);   // 5 daqiqa ushlab tur
  });
  check(res, { 'WS 101': r => r && r.status === 101 });
}
```

8.5 Chegara qiymatlar (thresholds) — hammasi

Metrika	🟢 O'tdi	🟡 Ogohlantirish	🔴 Yiqildi
`http_req_duration` p95 (menyu)	< 200 ms	200-400 ms	> 400 ms
`http_req_duration` p95 (buyurtma)	< 200 ms	200-350 ms	> 350 ms
`http_req_duration` p99 (barcha)	< 500 ms	500 ms-1 s	> 1 s
`http_req_failed` (o'qish)	< 0,1%	0,1-0,5%	> 0,5%
`http_req_failed` (buyurtma)	< 0,01%	0,01-0,05%	> 0,05% 🟡
`dublikat_buyurtma`	**0**	—	≥ 1 🟡🔴
`kesh_hit` (menyu)	> 90%	80-90%	< 80%
`ws_yetkazish_ms` p95	< 100 ms	100-300 ms	> 300 ms
CPU (web) pikda	< 70%	70-85%	> 85%
PG replication lag	< 2 s	2-10 s	> 10 s
PG buffer hit	> 99%	97-99%	< 97%
Xotira o'sishi (4 soatda)	< 5%	5-15%	> 15% (leak) 🟡

8.6 Qachon o'tkaziladi

Test	Qachon	Muhit	Kim
T1 Smoke	Har deploy'dan keyin (CI)	prod	avtomatik
T2 Menyu	Har hafta (juma 02:00)	staging	avtomatik (cron)
T3 Buyurtma	Har release (haftada 1)	staging	avtomatik
T4 WS	Oyda 1	staging	qo'lda
T5 Spike	Bayram oldidan (Navro'z, YN)	staging	qo'lda
T6 Soak	Oyda 1 (kechasi)	staging	avtomatik
T7 Stress	Choraklik / arxitektura o'zgarganda	izolyatsiya	qo'lda
T8 3D	CDN sozlamasi o'zgarganda	staging	qo'lda

```
# .github/workflows/load-test.yml
name: Yuklama testi
on:
  schedule: [{ cron: '0 21 * * 5' }]      # juma 02:00 (UTC+5)
  workflow_dispatch:
jobs:
  k6:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: grafana/setup-k6-action@v1
      - run: k6 run tests/load/t2_menyu.js --out json=t2.json
        env: { BASE: 'https://staging.armenu.uz', QR_TOKEN: '${{ secrets.QR }}' }
      - run: k6 run tests/load/t3_buyurtma.js --out json=t3.json
        env: { BASE: 'https://staging.armenu.uz' }
      - name: Natijani Telegram'ga
        if: always()
        run: python scripts/load_report.py t2.json t3.json
```

⚠ **Yuklama testi qoidalari:**

```
🔴 QAT'IY: prod'da faqat T1 (smoke, 5 VU).
   T2-T8 — faqat staging (prod bilan bir xil konfiguratsiya, 1/4 hajm).

🟡 Staging natijasini prod'ga ko'chirish: × (prod_node / staging_node)
   Masalan staging 1 web instans 300 RPS ko'tardi → prod 40 instans = 12 000 RPS

🔴 Test ma'lumoti alohida tenant'da (restaurant_id >= 900000).
   Haqiqiy restoran ma'lumotiga TEGILMAYDI.

🔴 To'lov/fiskal — sandbox rejimida (haqiqiy pul YO'Q).
```

---

9. MASSHTABLASH CHEKLOVLARI

> **Nima masshtablanmaydi va nega.** Bularni bilish — arxitektura yetukligi belgisi.

9.1 Cheklovlar jadvali

#	Nima	Chegara	Nega masshtablanmaydi	Yumshatish	Qoladigan risk
C1	**3D generatsiya (GPU)**	~50 model/kun/GPU	Photogrammetry 8-20 daq GPU vaqti; GPU qimmat va cheklangan	⭐ Kutubxona (85% taom undan), navbat, tashqi provayder (Meshy/Luma/Tripo)	Yangi taom 3D uchun 1-3 kun kutish
C2	**OFD API (soliq)**	~100 chek/daq/restoran	Davlat tizimi, biz nazorat qilmaymiz	Navbat + backoff, 2 provayder (UZKASSA/ePOS), offline buffer 24 soat	Pik payt chek kechikishi
C3	**AI provayder rate limit**	Claude 4k RPM, GPT 10k RPM	Provayder kvotasi	8 provayder fallback zanjiri, javob keshi (24 s), rate limit har tenant	Tarjima/chat kechikishi
C4	**To'lov provayderi**	Payme ~500 TPS	Tashqi tizim	3 provayder (Payme/Click/Uzum), navbat, retry	Pik payt to'lov kechikishi
C5	**Telegram Bot API**	30 xabar/s (bot), 20/daq (guruh)	Telegram cheklovi	Batch, navbat, prioritet (buyurtma > marketing)	Marketing xabari kechikadi
C6	**PostgreSQL yozish**	~3 000 TPS/primary	Bitta primary — WAL ketma-ket yoziladi	Sharding (Citus), event PG'dan chiqarish	Sharding = 4 hafta ish
C7	**Redis bitta node**	~100 000 ops/s	Bir oqimli (single-threaded)	Cluster (hash slot), o'qish replikasi	Cross-slot operatsiya yo'q
C8	**Daphne WS**	~50 000/instans	Python GIL + epoll	Gorizontal (3 instans), WS ni minimallashtirish	Sticky session kerak
C9	**Cloudflare purge**	1 000 URL/kun (Pro)	Provayder kvotasi	⚡ Versiyalangan URL (purge kerak emas)	Logotip o'zgarishida sekin
C10	**S3 prefiks**	3 500 PUT/s, 5 500 GET/s prefiks	AWS S3 arxitekturasi	Prefiks tarqatish (`r/{id}/` — tabiiy tarqoq)	—
C11	**Tranzaksiya izolyatsiyasi**	Bitta stol = ketma-ket	`SELECT FOR UPDATE` — mantiqiy zarurat	Lock qisqa (72 ms)	Bir stolga 14 buyurtma/s maks
C12	**Kunlik raqam**	Restoran ichida ketma-ket	Advisory lock	Restoranlar parallel	1 restoran ~90 buyurtma/s maks
C13	**3D fayl hajmi**	4 MB GLB, 4G da 8-15 s	Mobil tarmoq tezligi	Draco siqish, LOD, progressive, poster darhol	Sekin tarmoqda AR kutish
C14	**iOS WebXR**	Yo'q	Apple qo'llab-quvvatlamaydi	AR Quick Look (USDZ)	Funksiya farqi (iOS/Android)
C15	**Multi-region yozish**	Bitta primary region	CAP teoremasi — kuchli izchillik tanlangan	O'qish har regionda (replica)	Uzoq regionda yozish +100 ms

9.2 C1 — 3D generatsiya (batafsil)

```
MUAMMO:
  Photogrammetry (20-40 rasm → 3D model):
    GPU (RTX 4090): 12-20 daqiqa/model
    1 GPU × 24 soat / 16 daq = 90 model/kun (nazariy)
    Amalda (xato, retry, navbat): ~50 model/kun

  Talab: 1 000 restoran × 60 taom × 35% (3D bo'ladigan)
       = 21 000 model
       = 21 000 / 50 = 420 kun (1,2 yil!)  🔴

✅ YECHIM ZANJIRI:

  1. ⭐ KUTUBXONA (asosiy yechim)
     Taomlarning 85%i — standart o'zbek/rus/yevropa taomlari
     → osh, manti, somsa, lag'mon, shashlik, salat, choy...
     500 ta model butun bozorni qoplaydi

     21 000 × 0,85 = 17 850 → kutubxonadan (0 GPU vaqti, darhol) ✅
     21 000 × 0,15 =  3 150 → generatsiya kerak

  2. TASHQI PROVAYDER (pik uchun)
     Meshy.ai / Luma / Tripo — $0,20-2,00/model, parallel
     3 150 × $0,50 = $1 575 (bir martalik)
     Vaqt: 2 hafta (parallel API)

  3. O'Z GPU (uzoq muddat, arzon)
     RTX 4090 node $500/oy → 50 model/kun → 1 500/oy
     Break-even: 1 500 × $0,50 = $750 vs $500 → 2-oyda foydali

  4. NAVBAT + PRIORITET
     PRO tarif → prioritet 1 (24 soat SLA)
     FREE tarif → prioritet 5 (7 kun SLA yoki faqat kutubxona)

  ┌──────────────────────────────────────────────┐
  │ Natija: 420 kun → 14 kun (30× tezlashuv)     │
  │ Asosiy hissa: KUTUBXONA (85%)                │
  └──────────────────────────────────────────────┘
```

9.3 C3 — AI provayder rate limit

```
# apps/ai/router.py
"""
8 provayderli fallback zanjiri.
⚡ Har biri o'z rate limit'i bilan; biri tugasa keyingisiga o'tadi.
"""
PROVAYDERLAR = [
    # (nom,          RPM,   $/1M token,  vazifa)
    ('claude-haiku',  4000,   0.80,  ['chat','translate']),
    ('gpt-4o-mini',  10000,   0.15,  ['chat','translate']),
    ('gemini-flash',  2000,   0.075, ['chat','translate','vision']),
    ('deepseek',      1000,   0.14,  ['chat','translate']),
    ('groq-llama',   30000,   0.05,  ['chat']),
    ('mistral',       1000,   0.20,  ['translate']),
    ('cohere',        1000,   0.15,  ['translate']),
    ('yandex-gpt',     500,   0.30,  ['translate']),   # ru uchun yaxshi
]

def ai_call(vazifa: str, prompt: str, lang: str = 'uz') -> str:
    # 1. Kesh (24 soat) — 60% so'rov shu yerda tugaydi
    key = f"ai:{vazifa}:{hashlib.sha256((prompt+lang).encode()).hexdigest()}"
    if (cached := cache.get(key)) is not None:
        metrics.inc('ai.cache.hit')
        return cached

    # 2. Zanjir bo'ylab
    for nom, rpm, narx, vazifalar in PROVAYDERLAR:
        if vazifa not in vazifalar:
            continue
        if not rate_limiter.allow(nom, rpm):        # Redis token bucket
            continue
        if circuit_breaker.is_open(nom):            # yiqilgan provayder
            continue
        try:
            javob = call_provider(nom, prompt, timeout=15)
            cache.set(key, javob, 86400)
            AIUsage.log(nom, prompt, javob, narx)
            return javob
        except (RateLimitError, TimeoutError, APIError) as e:
            circuit_breaker.record_failure(nom)
            log.warning("ai_fallback", provayder=nom, xato=str(e))
            continue

    # 3. Hamma yiqildi → graceful degradation
    metrics.inc('ai.all_failed')
    if vazifa == 'translate':
        return prompt                    # tarjimasiz asl matn
    raise AIUnavailable("Barcha AI provayderlar band. Keyinroq urinib ko'ring.")
```

```
SIG'IM HISOBI:
  Jami RPM = 4000+10000+2000+1000+30000+1000+1000+500 = 49 500 RPM = 825 RPS
  Kesh hit 60% → effektiv sig'im = 825 / 0,4 = 2 062 RPS ✅

  S5 da AI so'rov: 25 000 × 2% = 500 RPS  →  ✅ SIG'ADI (4× headroom)
```

9.4 Nima MASSHTABLANADI (ijobiy ro'yxat)

Komponent	Qanday	Chegara
Web (gunicorn)	✅ Gorizontal, cheksiz (stateless)	LB sig'imi (30k RPS/node)
Worker	✅ Gorizontal, navbat bo'yicha mustaqil	Navbat backend (Redis)
CDN	✅ Cheksiz (Cloudflare anycast)	Byudjet
PG o'qish	✅ Replica qo'shish (5 gacha)	Replikatsiya lag
PG yozish	🟡 Sharding bilan (murakkab)	4-8 shard amaliy
Redis	✅ Cluster (16 384 slot)	Cross-slot operatsiya
S3	✅ Cheksiz	Byudjet
ClickHouse	✅ Shard + replica	—
WS	🟡 Gorizontal + sticky session	Ulanish soni (optimizatsiya bilan)

---

10. XULOSA — MASSHTABLASH KARTASI

```
╔═══════════════════════════════════════════════════════════════╗
║                 1 MLN BIR VAQTDA — YAKUNIY HISOB               ║
╠═══════════════════════════════════════════════════════════════╣
║                                                               ║
║  KIRISH:   1 000 000 bir vaqtda faol foydalanuvchi            ║
║            → 25 000 RPS (40 s da 1 harakat)                   ║
║                                                               ║
║  FILTRLASH:                                                   ║
║    Brauzer kesh  45%  ushlaydi  →  13 750 RPS qoladi          ║
║    CDN           75%  ushlaydi  →   3 437 RPS qoladi          ║
║    nginx         12%  ushlaydi  →   3 025 RPS qoladi          ║
║    Redis         90%  ushlaydi  →   1 427 RPS DB'ga           ║
║    ⚡ 94,3% yuklama DB'ga umuman kelmaydi                     ║
║                                                               ║
║  WEBSOCKET:                                                   ║
║    Naiv:      1 000 000 ulanish → 32 GB RAM → 20 instans → $1 600 ║
║    ⚡ Optimal:    90 000 ulanish → 2,9 GB   →  3 instans → $  240 ║
║    Sabab: mijozga WS FAQAT buyurtmadan keyin (5%) + xodim (40k)  ║
║                                                               ║
║  DB YOZISH:                                                   ║
║    Xom:  9 875 write/s  →  🟡 sig'im 3 000 TPS dan 3,3× oshiq ║
║    Y1 event→ClickHouse, Y2 audit batch, Y3 fiskal async,      ║
║    Y4 bulk_create, Y5 partition, Y6 commit_delay              ║
║    ⚡ Natija: ~3 000 TPS kerak, 3 750 TPS sig'im  →  ✅       ║
║                                                               ║
║  INFRA:  64 node  ·  $4 400/oy  ·  daromadning 1,89%          ║
╚═══════════════════════════════════════════════════════════════╝
```

10.1 10 ta qat'iy masshtablash qoidasi

#	Qoida	Buzilsa
1	**Har issiq yo'l keshlanadi** (menyu, taom, restoran)	🔴 DB 10× yuklanadi
2	**Mijozga WS faqat buyurtmadan keyin**	🔴 20 instans o'rniga 3
3	**Event PostgreSQL'ga yozilmaydi** (ClickHouse)	🔴 write TPS 4× oshadi
4	**Uzoq ish (> 2 s) → worker navbat**	🔴 Web bloklanadi, timeout
5	**Web instans stateless** (locmem/lokal disk/thread ⛔)	🔴 Auto-scaling buziladi
6	**N+1 CI'da bloklanadi** (query count testi)	🟡 Sekin, 20×
7	`bulk_create` / `iterator` **— 3+ obyektda**	🟡 TPS va RAM
8	**PgBouncer transaction mode +** `CONN_MAX_AGE=0`	🔴 too many clients
9	`SELECT FOR UPDATE` **ichida tashqi API** ⛔	🔴 Kaskadli sekinlashuv
10	**Har release'da T3 (buyurtma) yuklama testi**	🔴 Regressiya prod'da topiladi

10.2 Masshtablash yo'l xaritasi

Qachon	Nima qilinadi	Kim	Vaqt
20 restoran	DB'ni alohida VPS'ga	DevOps	1 kun
100 restoran	2-chi web instans + LB	DevOps	1 kun
200 restoran	Read replica + DB router	Backend	3 kun
400 restoran	PgBouncer + Redis Sentinel	DevOps	2 kun
500 restoran	⚡ **Partition** (orders, arview)	Backend	1 hafta
800 restoran	ClickHouse + event pipeline	Backend	2 hafta
1 500 restoran	Auto-scaling (web + worker)	DevOps	1 hafta
2 500 restoran	Redis Cluster · 3-chi replica	DevOps	3 kun
3 500 restoran	⚡ Sharding **tayyorgarligi** (denorm, CI)	Backend	6 hafta
5 000 restoran	⚡ **Citus sharding** (4 shard)	Backend+DevOps	4 hafta
8 000 restoran	Multi-region (UZ + KZ)	Arxitektor	8 hafta

QISM 5

Ishonchlilik

Versiya: 1.0 · **Sana:** 2026-09-08
Kim uchun: SRE · DevOps · tizim arxitektori · texnik direktor
Asos: `TZ_MASTER/03_ARXITEKTURA.md` §5, §6, §7, §10

---

0. BU HUJJATNING MAQSADI

> **Asosiy printsip:** ARMENU restoranning **ish jarayonidir**, sayt emas.
> Restoranda soat 20:00 da tizim yiqilsa — 40 ta stol, 200 ta mijoz, real pul yo'qoladi.
> Shuning uchun arxitektura **"yiqilmaydi"** deb emas, **"yiqilganda ham ishlaydi"**
> deb qurilgan.

Uch qatlamli ishonchlilik falsafasi:

```
┌──────────────────────────────────────────────────┐
│  QATLAM 1 — OLDINI OLISH (prevention)            │
│  Redundantlik · health-check · sig'im zaxirasi · test  │
│  → Yiqilish EHTIMOLINI kamaytiradi               │
├──────────────────────────────────────────────────┤
│  QATLAM 2 — CHIDAMLILIK (tolerance)              │
│  Circuit breaker · fallback · bulkhead · timeout · retry  │
│  → Yiqilish TA'SIRINI cheklaydi (kaskad bo'lmaydi)  │
├──────────────────────────────────────────────────┤
│  QATLAM 3 — TIKLANISH (recovery)                 │
│  Self-healing daemon · auto-failover · DR runbook  │
│  → Yiqilish DAVOMIYLIGINI qisqartiradi           │
└──────────────────────────────────────────────────┘
```

Bitta jumlada: *Mijoz taom buyurtma qila olsa — tizim ishlayapti.*
Qolgan hamma narsa (AR, AI, analitika, 3D) qurbon qilinishi mumkin.

---

1. ISHONCHLILIK MAQSADLARI (SLO)

1.1 Xizmatlar bo'yicha SLO jadvali

#	Xizmat (SLI)	Mavjudlik SLO	Javob vaqti p95	Javob vaqti p99	Xato darajasi	Kritiklik
1	**Buyurtma yaratish** `POST /v1/orders/`	🟢 **99,90%**	200 ms	500 ms	< 0,1%	Yadro
2	**To'lov boshlash** `POST /v1/payments/`	🟢 **99,90%**	400 ms	1 000 ms	< 0,1%	Yadro
3	**Mijoz menyu (o'qish)** `GET /v1/menu/`	🟢 **99,95%**	100 ms	250 ms	< 0,05%	Yadro
4	**QR kirish** `GET /t/<token>`	🟢 **99,95%**	80 ms	200 ms	< 0,05%	Yadro
5	**KDS (oshxona ekrani)**	🟡 99,90%	150 ms	400 ms	< 0,1%	Yadro
6	**AR model ko'rish** `GET /v1/ar/<id>/`	🟡 99,50%	60 ms	150 ms	< 0,5%	Muhim
7	**Real-time (WS yetkazish)**	🟡 99,50%	50 ms	200 ms	< 0,5%	Muhim
8	**Fiskal chek (OFD)**	🟡 99,50%[1]	—	—	< 0,5%	Muhim
9	**Admin panel**	🟠 99,50%	300 ms	800 ms	< 0,5%	O'rta
10	**3D fayl yetkazish (CDN)**	🟠 99,50%	1,5 s	4 s	< 1%	O'rta
11	**AI chat / tavsiya**	🔵 99,00%	1,2 s	3 s	< 2%	Qo'shimcha
12	**3D generatsiya (worker)**	🔵 99,00%	—	—	< 3%	Qo'shimcha
13	**Analitika / hisobot**	🔵 99,00%	800 ms	2 s	< 2%	Qo'shimcha
14	**Telegram bildirishnoma**	🔵 98,00%	—	—	< 3%	Qo'shimcha

[1] *Fiskal SLO "24 soat ichida OFDga yetkazilgan chek ulushi" sifatida o'lchanadi, so'rov-daqiqasi emas — chunki chek navbatga tushishi normal holat.*

1.2 Uzilish byudjeti (yiliga ruxsat etilgan)

SLO	Yiliga	Oyiga	Haftasiga	Ma'nosi
99,95%	4 s 23 daq	21,9 daq	5,0 daq	Bitta uzoq deploy
99,90%	8 s 46 daq	43,8 daq	10,1 daq	Bitta DB failover + deploy
99,50%	43 s 48 daq	3 s 39 daq	50,4 daq	Bir necha jiddiy insident
99,00%	87 s 36 daq	7 s 18 daq	100,8 daq	Tashqi provayder yiqilishi
98,00%	7 kun 7 s	14 s 36 daq	3 s 22 daq	Telegram/tashqi API

1.3 ⚡ Error budget hisobi va siyosati

Formula:

```
Error budget (daqiqa/oy) = 43 200 × (1 − SLO)
Sarflangan               = ∑(insident davomiyligi × ta'sirlangan trafik ulushi)
Qolgan %                 = (Byudjet − Sarflangan) / Byudjet × 100
```

Namuna hisob — "Buyurtma yaratish" (SLO 99,90%), sentabr oyi:

Sana	Insident	Davomiylik	Ta'sirlangan trafik	Sarf (daq)
09-03	PG failover (Patroni)	47 s	100%	0,78
09-11	Deploy — WS restart	12 daq	0% (buyurtma ishladi)	0,00
09-14	Redis Sentinel almashuv	9 s	100%	0,15
09-19	Payme timeout	22 daq	18% (to'lov qismi)	3,96
09-27	1 web node OOM	6 daq	0% (LB chetlab o'tdi)	0,00
	JAMI			**4,89 daq**

```
Byudjet   = 43 200 × 0,001 = 43,2 daq/oy
Sarflangan= 4,89 daq
Qolgan    = 38,31 daq (88,7%)  → 🟢 YASHIL
```

Byudjet siyosati (release gating):

Qolgan byudjet	Rang	Siyosat
> 50%	🟢	Normal — feature deploy erkin, chaos test rejaga ko'ra
25–50%	🟡	Ehtiyot — deploy faqat ish kunlari 10:00–15:00, canary majburiy
10–25%	🟠	Cheklov — faqat bug-fix va ishonchlilik ishlari; yangi feature MUZLATILADI
< 10%	🔴	⛔ **Feature freeze**. Butun jamoa ishonchlilikka o'tadi. Post-mortem majburiy
< 0%	🔴	SLO buzildi — texnik direktor darajasida ko'rib chiqish, mijozga xabar

Amalga oshirish (Prometheus recording rule):

```
# monitoring/rules/slo.yml
groups:
  - name: armenu_slo
    interval: 30s
    rules:
      - record: armenu:order_success_ratio_30d
        expr: |
          sum(rate(armenu_http_requests_total{endpoint="orders_create",status!~"5.."}[30d]))
          /
          sum(rate(armenu_http_requests_total{endpoint="orders_create"}[30d]))

      - record: armenu:order_error_budget_remaining
        expr: |
          1 - ((1 - armenu:order_success_ratio_30d) / 0.001)

      - alert: ErrorBudgetBurnFast
        # 1 soatda oylik byudjetning 5% i yonib ketsa — darhol
        expr: |
          (1 - (sum(rate(armenu_http_requests_total{endpoint="orders_create",status!~"5.."}[1h]))
                / sum(rate(armenu_http_requests_total{endpoint="orders_create"}[1h])))) > 0.014
        for: 5m
        labels: {severity: P1}
        annotations:
          summary: "🔴 Error budget tez yonmoqda — 1 soatda 5%"

      - alert: ErrorBudgetLow
        expr: armenu:order_error_budget_remaining < 0.10
        for: 15m
        labels: {severity: P2}
        annotations:
          summary: "🟠 Error budget < 10% — FEATURE FREEZE"
```

---

2. YIQILISH REJIMI TAHLILI (FMEA)

> FMEA = Failure Mode and Effects Analysis.
> **RPN** (Risk Priority Number) = Ehtimol × Ta'sir × Aniqlanmaslik (har biri 1–5).
> RPN ≥ 45 → majburiy yumshatish rejasi, RPN ≥ 75 → arxitektura o'zgarishi.

2.1 Infratuzilma komponentlari

#	Komponent	Yiqilish turi	E	T	A	RPN	Aniqlash usuli	MTTR	Yumshatish
F1	nginx (LB)	Process crash / config xato	2	5	1	10	systemd + `/healthz` port probe	30 s	2× LB, keepalived VIP, `nginx -t` deploy oldidan
F2	gunicorn worker	OOM kill (RAM > 80%)	3	3	2	18	`armenu_worker_rss_bytes`	15 s	`--max-requests 1000 --max-requests-jitter 100`, memory guard healer
F3	gunicorn (barchasi)	Deploy xatosi / migratsiya bloklandi	2	5	1	10	Health-check 3× fail	2 daq	Canary deploy, avto-rollback, socket-activation
F4	daphne (WS)	Ulanish sizishi, event-loop bloklandi	3	2	3	18	`armenu_ws_connections` + p99	15 s	2+ WS node, klient auto-reconnect, SSE fallback
F5	PostgreSQL primary	Disk to'ldi / node crash	2	5	1	10	Patroni watchdog, `pg_isready`	30–60 s	Patroni auto-failover, sinxron replica, disk alert 85%
F6	PostgreSQL replica	Replikatsiya lag > 30 s	3	2	2	12	`pg_stat_replication.lag`	5 daq	2+ replica, lag > 10 s → replica trafikdan chiqadi
F7	PostgreSQL — connection pool	Pool tugadi (leak)	3	4	2	24	`pg_stat_activity` count	1 daq	PgBouncer transaction mode, conn leak healer, `CONN_MAX_AGE=60`
F8	Redis master	OOM / node crash	2	4	1	8	Sentinel quorum	10 s	Sentinel 3 node, `maxmemory-policy allkeys-lru`, AOF everysec
F9	Redis — cache stampede	Kesh muzlagach 1000 so'rov DBga	3	4	3	36	`cache_hit_ratio` keskin tushishi	2 daq	Probabilistic early expiration + `lock`-bilan-qayta-hisoblash
F10	Worker (har navbat)	Process o'lishi, job `running` da qolishi	4	2	2	16	`heartbeat_at` eskirishi	60 s	Orphan job recovery healer, `Restart=always`
F11	Worker navbati	Navbat to'lishi (depth > 1000)	3	3	2	18	`armenu_queue_depth`	10 daq	Bulkhead (5 alohida navbat), avto-scale, DLQ
F12	S3 / MinIO	Bucket mavjud emas / 5xx	2	3	2	12	HEAD probe har 60 s	5 daq	Replica bucket, CDN kesh, rasm-fallback
F13	Disk (har node)	100% to'ldi	3	5	1	15	`node_filesystem_avail`	10 daq	85% da avto-cleanup healer, log rotate, alert 80%
F14	Tarmoq (node↔DB)	Paket yo'qolishi, split-brain	2	5	3	30	Patroni DCS quorum, ping	1–5 daq	etcd 3 node quorum, fencing, `statement_timeout`
F15	Sertifikat (TLS)	Muddati tugadi	2	5	2	20	Muddat probe (30 kun oldin)	10 daq	certbot avto-yangilash + 30/14/7 kunlik alert

2.2 Tashqi xizmatlar

#	Xizmat	Yiqilish turi	E	T	A	RPN	Aniqlash usuli	MTTR	Yumshatish
F16	To'lov (Payme)	5xx / timeout / hisob bloklandi	4	4	1	16	CB `payme` OPEN	Darhol	Click → Uzum → naqd zanjiri
F17	To'lov — webhook kelmadi	Provayder callback yubormadi	3	4	4	48	🟠 Reconciliation cron 5 daq	5 daq	Poll-fallback: `check_transaction` har 60 s, 24 s davomida
F18	OFD (UZKASSA)	API o'chgan / IKPU rad etdi	4	3	2	24	CB `ofd` OPEN + navbat depth	5 daq–24 s	OFD-2 (e-POS), navbat + 48 s retry, qo'lda chek
F19	AI provayder	Rate limit / kredit tugadi	5	1	1	5	CB + `ai_cost_usd_total`	Darhol	6 bosqichli zanjir + FAQ fallback
F20	CDN (Cloudflare)	Region uzilishi	2	3	1	6	Sintetik probe 4 nuqtadan	5 daq	DNS failover → origin nginx, `Cache-Control` uzun
F21	Telegram Bot API	429 / bloklangan	3	1	1	3	Retry hisoblagichi	1 soat	Navbat + retry 5×, panelda badge fallback
F22	tasnif.soliq.uz (IKPU)	Sekin / o'chgan	3	2	2	12	Probe + CB	1 s	IKPU lokal kesh (30 kun TTL), qo'lda kiritish

2.3 Ilova qatlami (mantiqiy xatolar)

#	Yiqilish turi	E	T	A	RPN	Aniqlash	Yumshatish
F23	🟠 **Cross-tenant sizish** (`restaurant_id` filtri unutildi)	2	5	5	**50**	CI `test_tenant_isolation`, audit skript	4 qatlamli himoya — `06_XAVFSIZLIK.md` §2
F24	Dublikat buyurtma (klient 2× bosdi)	4	3	3	36	`X-Idempotency-Key` konflikt metrikasi	Idempotency Redis + DB unique
F25	Dublikat to'lov (webhook 2× keldi)	4	5	2	40	`PaymentTransaction` unique	Idempotency + `SELECT FOR UPDATE`
F26	Narx manipulyatsiyasi (klient narx yubordi)	3	5	4	**60**	Server-side narx testi	⚡ Narx FAQAT serverda; klient `dish_id` + `qty` yuboradi
F27	Stuck order (`cooking` 45 daq)	3	3	3	27	Stuck order healer	Admin alert + KDS badge
F28	Migratsiya bloklandi (`ALTER TABLE` lock)	2	5	2	20	Deploy timeout	`lock_timeout=3s`, zero-downtime migratsiya qoidalari
F29	N+1 so'rov (yangi kod)	4	3	3	36	`django-silk` staging, p95 alert	`select_related` / `prefetch_related` CI linter
F30	Poison message (worker cheksiz crash)	3	3	2	18	`attempts >= max_attempts`	DLQ + 3 urinishdan keyin karantín

🔴 **Eng yuqori RPN uchtaligi:**
1. **F26 — narx manipulyatsiyasi (60)** → yagona yechim: narxni server hisoblaydi (Oltin qoida #2)
2. **F23 — cross-tenant sizish (50)** → 4 qatlamli himoya + CI gate (Oltin qoida #1)
3. **F17 — to'lov webhook kelmadi (48)** → reconciliation cron majburiy

---

3. YIQILISH MATRITSASI (17 ssenariy)

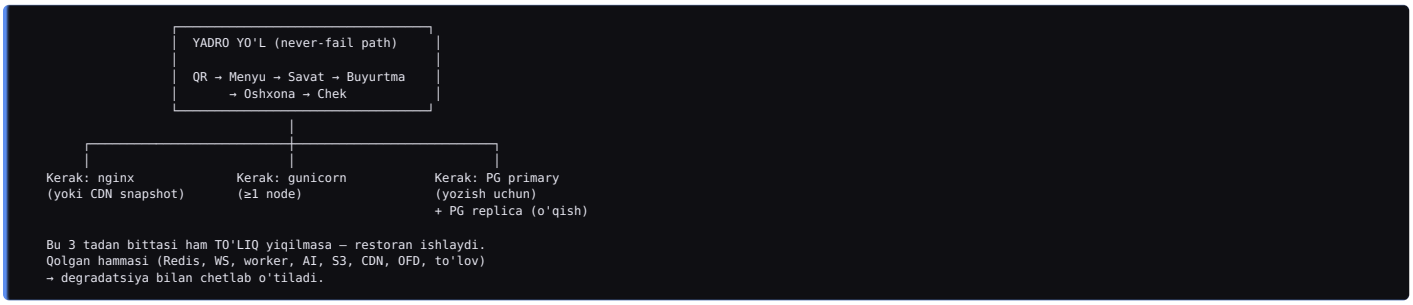
> Har ssenariy: **ta'sir darajasi · nima ishlashda davom etadi · tiklanish**

Belgilar

🟢 = sezilmaydi · 🟡 = qisman degradatsiya · 🔴 = jiddiy · 🟣 = to'liq to'xtash

#	Yiqilish	Daraja	✅ Ishlashda davom etadi	⛔ Ishlamaydi	Tiklanish
1	**1 ta web node**	🟢	Hammasi. LB 3 s ichida boshqasiga yuboradi	—	Avto 30 s (systemd + LB health)
2	**Barcha web node**	🟣	Faqat CDN'dagi statik menyu snapshot (read-only PWA)	Buyurtma, to'lov, admin	Auto-restart 30 s → alert P1 → qo'lda 5 daq
3	**1 ta WS node**	🟡	Barcha HTTP. Klient 1–30 s ichida boshqa nodega ulanadi	O'sha nodedagi jonli push (qisqa uzilish)	Avto 15 s
4	**Barcha WS node**	🟡	✅ Menyu, buyurtma, to'lov, KDS (polling 10 s), admin	Jonli push, "ofitsiant yo'lda" indikatori	Auto-restart 30 s. Klient SSE → polling'ga tushadi
5	**Redis (cache+channels)**	🔴	✅ Menyu (DBdan, sekinroq), buyurtma, to'lov, fiskal	WS butunlay, rate-limit (fail-open), idempotency kesh (DB unique zaxira)	Sentinel failover 10 s. Cache warm-up 30 s
6	**PostgreSQL primary**	🟣→🟡	✅ **O'qish ishlaydi** (replica): menyu, AR, taom kartochkasi, KDS ko'rish	Yozish: buyurtma, to'lov, status o'zgarishi	Patroni auto-failover 30–60 s. Yozish navbatga tushmaydi — 503 + "qayta urinib ko'ring"
7	**PostgreSQL to'liq (primary+replica)**	🟣	Faqat CDN statik: menyu snapshot JSON, 3D fayllar, rasmlar	Hamma dinamik	Backupdan tiklash: RTO 5 daq (PITR), 30 daq (to'liq)
8	**Worker `model3d`**	🟢	Hammasi. Mavjud 3D modellar ishlaydi	Yangi 3D generatsiya (navbatda kutadi)	Auto-restart 5 s. Navbat saqlanadi
9	**Worker `fiscal`**	🟡	✅ Buyurtma, to'lov, menyu — hammasi	Chek OFDga bormaydi (navbatda)	Auto-restart 5 s. Navbat depth > 50 → P1 alert
10	**Worker `notify`**	🟢	Hammasi. WS orqali jonli xabar ishlaydi	Telegram push, oshxona printeri	Auto-restart 5 s
11	**Worker `ai`**	🟢	Hammasi	AI chat "band" javobi + FAQ	Auto-restart 5 s
12	**Worker `analytics`**	🟢	Hammasi	Hisobot kechikadi (event navbatda)	Auto-restart. Event 7 kun saqlanadi
13	**AI provayder (biri)**	🟢	Hammasi. CB keyingi provayderga o'tadi	—	CB HALF_OPEN 60 s
14	**AI — barcha 6 provayder**	🟢	Hammasi. Chat: "Ofitsiantni chaqiring" + FAQ + qidiruv	AI tavsiya, tarjima (kesh ishlaydi)	Provayder tiklanishi
15	**To'lov provayder (biri)**	🟡	✅ Boshqa provayder tugmasi. Naqd har doim	O'sha provayder tugmasi yashiriladi	CB 120 s
16	**To'lov — barchasi**	🔴	✅ Buyurtma, oshxona, chek — hammasi. **Naqd to'lov**	Onlayn to'lov	CB + qo'lda tekshiruv
17	**OFD (fiskal) — ikkalasi**	🟡	✅ Buyurtma, to'lov — hammasi	QR-chek darhol emas	Navbat + 48 s retry. Qo'lda kassa cheki
18	**CDN (Cloudflare)**	🟡	Hammasi, lekin sekinroq (origin'dan)	—	DNS failover 5 daq yoki Cloudflare o'zi
19	**S3 / MinIO**	🟡	✅ Menyu, buyurtma, to'lov, narx	AR ko'rish (rasm-fallback ko'rsatiladi), yangi rasm yuklash	Replica bucket almashuvi 5 daq
20	**Telegram API**	🟢	Hammasi. Admin panelda badge ko'rinadi	Telegram push, bot buyurtma	Retry 1 soat, keyin DLQ
21	**Tarmoq bo'linishi (split-brain)**	🔴	Ko'pchilik (quorum) tomonda hammasi ishlaydi	Ozchilik tomon: fencing, read-only	etcd quorum. Ozchilik node o'zini o'chiradi
22	**Butun DC / region**	🟣	Boshqa regiondagi CDN statik snapshot	Hammasi	DR: boshqa regionda ko'tarish, RTO 4 soat

3.1 ⚡ "Yadro yo'l" — nima bo'lganda ham ishlashi shart



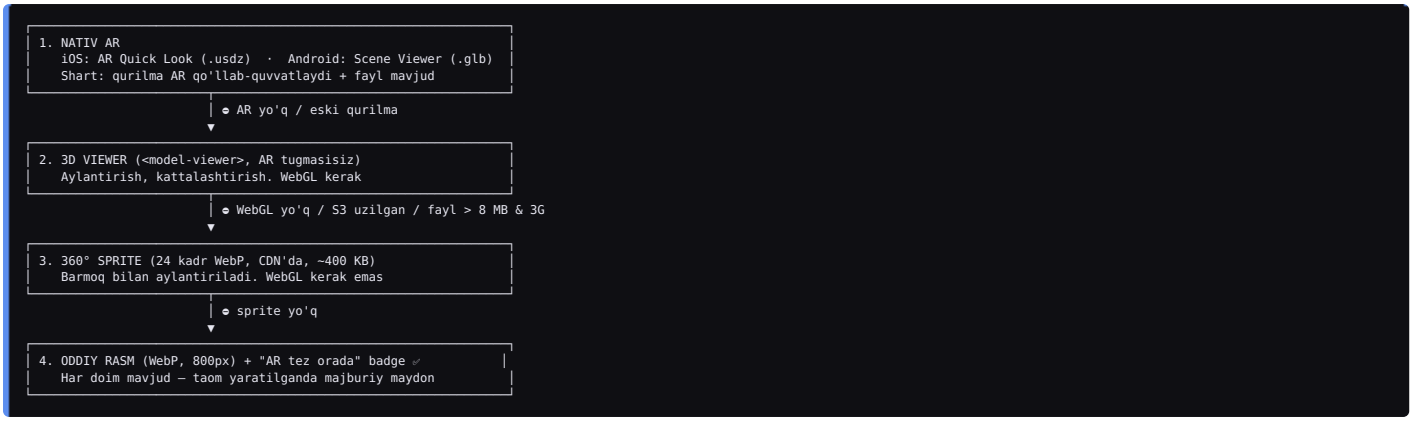
Tekshiruv: chaos testda Redis + WS + barcha worker + S3 + AI birgalikda o'chiriladi → buyurtma yaratish testi **o'tishi shart**.

---

4. GRACEFUL DEGRADATION ZANJIRLARI

> Har zanjir: yuqoridan pastga. Yuqorigi ishlamasa — keyingisi.
> **Hech bir zanjir "xato sahifasi" bilan tugamaydi** — har doim ishlaydigan variant bor.

4.1 👁 AR ko'rish



Kod (klient tanlovi):

```
// static/js/ar/ar_chain.js
async function pickArMode(dish, device) {
  if (device.supportsAR && dish.ar_ready && device.netType !== 'slow-2g')
    return {mode: 'native-ar',  src: dish.usdz_url || dish.glb_url};
  if (device.hasWebGL && dish.glb_url && dish.glb_size < 8 * 1024 * 1024)
    return {mode: 'viewer-3d',  src: dish.glb_url};
  if (dish.sprite360_url)
    return {mode: 'sprite-360', src: dish.sprite360_url};
  return {mode: 'image', src: dish.image_url};   // ✔ har doim bor
}
```

4.2 🤖 AI chat



4.3 📡 Real-time (KDS, ofitsiant, mijoz statusi)




```
// static/js/rt/transport.js — avtomatik pasayish va ko'tarilish
class RealtimeTransport {
  constructor(url) { this.level = 0; this.fails = 0; }   // 0=WS 1=SSE 2=poll 3=manual
  degrade() { this.level = Math.min(this.level + 1, 3); this.connect(); }
  // ⚡ Har 5 daqiqada bir marta yuqoriga qaytishga urinadi
  scheduleUpgrade() {
    if (this.level === 0) return;
    setTimeout(() => { this.level = 0; this.fails = 0; this.connect(); }, 300000);
  }
}
```

4.4 🧾 Fiskal (OFD)

```
┌──────────────────────────────────────────────────────┐
│ 1. OFD-1 (UZKASSA) — sinxron, 5 s timeout            │
└──────────────────────┬───────────────────────────────┘
                       │ CB OPEN yoki 3× xato
                       ▼
┌──────────────────────────────────────────────────────┐
│ 2. OFD-2 (e-POS) — zaxira provayder                  │
└──────────────────────┬───────────────────────────────┘
                       │ ikkalasi ham yiqilgan
                       ▼
┌──────────────────────────────────────────────────────┐
│ 3. NAVBAT (fiscal queue) — chek `pending_fiscal` holatida    │
│    Retry: 5m→15m→1s→6s→24s (48 soat davomida)        │
│    ⚡ Mijozga: "Chek tez orada SMS/Telegram orqali keladi"   │
└──────────────────────┬───────────────────────────────┘
                       │ 48 soat o'tdi
                       ▼
┌──────────────────────────────────────────────────────┐
│ 4. QO'LDA: kassa apparatidan chek + admin panelda "qo'lda    │
│    fiskallashtirildi" belgisi + soliq hisobotiga qo'shiladi ↙ │
└──────────────────────────────────────────────────────┘
```

> ⚡ **Muhim:** fiskal HECH QACHON buyurtmani bloklamaydi. Buyurtma `transaction.atomic()` ichida yaratiladi, fiskal `on_commit` da navbatga tushadi.

4.5 💳 To'lov

```
┌──────────────────────────────────────────────────────┐
│ 1. Payme (asosiy) → 2. Click → 3. Uzum               │
│    Har biri CB bilan; OPEN bo'lgani UI'da YASHIRILADI │
└──────────────────────┬───────────────────────────────┘
                       │ 3 tasi ham OPEN
                       ▼
┌──────────────────────────────────────────────────────┐
│ 2. TERMINAL: "Ofitsiantga terminal orqali to'lang"   │
│    Buyurtma `awaiting_payment`, ofitsiant qo'lda tasdiqlaydi  │
└──────────────────────┬───────────────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────┐
│ 3. NAQD: ofitsiant qabul qiladi → PIN bilan tasdiqlaydi ↙ │
│    ⚡ Naqd YO'L HECH QACHON o'chirilmaydi — oxirgi zaxira  │
└──────────────────────────────────────────────────────┘
```

4.6 🖼 3D fayl yetkazish

```
CDN edge kesh (Cloudflare, 30 kun TTL)
   │ ⊘ CDN 5xx / miss
   ▼
S3/MinIO origin (presigned URL, 1 soat)
   │ ⊘ S3 uzilgan
   ▼
Replica bucket (boshqa region)
   │ ⊘ u ham yo'q
   ▼
Brauzer Cache API (PWA service worker, oldin ko'rilgan modellar)
   │ ⊘ kesh bo'sh
   ▼
Rasm (WebP) + "3D vaqtincha mavjud emas" ↙
```

4.7 📲 O'qish yo'li (menyu)

```
L1: Django process-local kesh (LocMem, 60 s)   ← ~0,1 ms
   │ miss
   ▼
L2: Redis kesh (menu:<rest_id>:<lang>, 5 daq)  ← ~2 ms
   │ miss / Redis o'chgan
   ▼
L3: PostgreSQL read replica                    ← ~20 ms
   │ replica lag > 10 s / o'chgan
   ▼
L4: PostgreSQL primary                         ← ~25 ms
   │ ⊘ butun DB o'chgan
   ▼
L5: CDN'dagi statik snapshot JSON              ← ~50 ms  ↙
    (har 15 daqiqada generatsiya qilinadi, faqat-o'qish)
    Mijoz menyuni ko'radi, "buyurtma vaqtincha mavjud emas"
```

Snapshot generatori:

```python
# apps/menu/tasks.py
@periodic(every=timedelta(minutes=15), queue='analytics')
def publish_menu_snapshot():
    """Har restoran menyusini CDNga statik JSON qilib chiqaradi.
    DB butunlay yiqilsa mijoz shu snapshotni ko'radi."""
    for rest in Restaurant.objects.filter(is_active=True).only('id', 'slug'):
        for lang in ('uz', 'ru', 'en'):
            data = build_menu_payload(rest.id, lang)      # read replica'dan
            key = f'snapshots/{rest.slug}/{lang}.json'
            s3.put_object(
                Bucket=settings.CDN_BUCKET, Key=key,
                Body=json.dumps(data, ensure_ascii=False).encode(),
                ContentType='application/json',
                CacheControl='public, max-age=900, stale-if-error=86400',
            )
```

---

5. CIRCUIT BREAKER — TO'LIQ AMALGA OSHIRISH

5.1 Holat mashinasi



Holat o'tishlari jadvali:

Joriy holat	Hodisa	Yangi holat	Qo'shimcha
CLOSED	muvaffaqiyat	CLOSED	`fail_count = 0`
CLOSED	xato	CLOSED	`fail_count += 1`
CLOSED	`fail_count >= threshold`	**OPEN**	`opened_at = now`, alert P2
OPEN	so'rov keldi, `now < opened_at + timeout`	OPEN	fallback, `rejected += 1`
OPEN	so'rov keldi, `now >= opened_at + timeout`	**HALF_OPEN**	1 ta probe o'tkaziladi
HALF_OPEN	probe muvaffaqiyat	HALF_OPEN	`success_count += 1`
HALF_OPEN	`success_count >= threshold`	**CLOSED**	tiklandi, alert "✅ tiklandi"
HALF_OPEN	probe xato	**OPEN**	`timeout × 2` (maks 900 s)

5.2 To'liq kod

```
# core/resilience/breaker.py
"""
Redis'ga tayangan taqsimlangan Circuit Breaker.
Barcha web/worker node'lar bitta holatni ko'radi.
"""
import time
import random
import logging
from dataclasses import dataclass
from django.core.cache import caches
from core.metrics import metrics
from core.alerts import alert

logger = logging.getLogger('armenu.resilience')
_redis = caches['default'].client.get_client(write=True)


class CircuitOpenError(Exception):
    """Xizmat OPEN — chaqiruv umuman yuborilmadi."""
    def __init__(self, name, retry_after):
        self.name, self.retry_after = name, retry_after
        super().__init__(f"Circuit '{name}' OPEN, {retry_after}s dan keyin")


@dataclass(frozen=True)
class BreakerConfig:
    fail_threshold: int = 5        # necha xatodan keyin OPEN
    timeout: int = 60              # OPEN'da necha soniya turadi
    success_threshold: int = 2     # HALF_OPEN'da necha muvaffaqiyat kerak
    max_timeout: int = 900         # backoff cheki
    backoff_mult: float = 2.0      # takroriy yiqilishda timeout ×
    call_timeout: float = 5.0      # bitta chaqiruv timeout (soniya)
    error_types: tuple = (Exception,)   # qaysi xato "yiqilish" hisoblanadi


class CircuitBreaker:

    KEY = 'cb:{name}:{field}'

    def __init__(self, name: str, config: BreakerConfig = None):
        self.name = name
        self.cfg = config or BreakerConfig()

    # ---------- holat ----------
    def _k(self, field):
        return self.KEY.format(name=self.name, field=field)

    def state(self) -> str:
        raw = _redis.hgetall(self._k('state'))
        if not raw:
            return 'CLOSED'
        st = raw.get(b'state', b'CLOSED').decode()
        if st == 'OPEN':
            opened_at = float(raw.get(b'opened_at', b'0'))
            timeout = float(raw.get(b'timeout', self.cfg.timeout))
            if time.monotonic() - opened_at >= timeout:
                return 'HALF_OPEN'
        return st

    def _write(self, **fields):
        _redis.hset(self._k('state'), mapping={
            k: str(v).encode() for k, v in fields.items()
        })
        _redis.expire(self._k('state'), 86400)
```

```
    # ---------- o'tishlar ----------
    def _open(self, prev_timeout=None):
        timeout = min(
            (prev_timeout or self.cfg.timeout) * (self.cfg.backoff_mult if prev_timeout else 1),
            self.cfg.max_timeout,
        )
        self._write(state='OPEN', opened_at=time.monotonic(),
                    timeout=timeout, fail_count=0, success_count=0)
        metrics.gauge('armenu_circuit_breaker_state', 2, labels={'service': self.name})
        logger.error("[CB] 🔴 %s → OPEN (%.0fs)", self.name, timeout)
        alert(f"🔴 Circuit breaker OPEN: {self.name} ({timeout:.0f}s)", severity='P2')

    def _close(self):
        _redis.delete(self._k('state'))
        metrics.gauge('armenu_circuit_breaker_state', 0, labels={'service': self.name})
        logger.info("[CB] 🟢 %s → CLOSED (tiklandi)", self.name)
        alert(f"🟢 Circuit breaker tiklandi: {self.name}", severity='INFO')

    def _on_success(self, state):
        if state == 'HALF_OPEN':
            n = _redis.hincrby(self._k('state'), 'success_count', 1)
            if n >= self.cfg.success_threshold:
                self._close()
        else:
            _redis.hdel(self._k('state'), 'fail_count')

    def _on_failure(self, state):
        if state == 'HALF_OPEN':
            prev = float(_redis.hget(self._k('state'), 'timeout') or self.cfg.timeout)
            self._open(prev_timeout=prev)
            return
        n = _redis.hincrby(self._k('state'), 'fail_count', 1)
        _redis.expire(self._k('state'), 86400)
        if n >= self.cfg.fail_threshold:
            self._open()

    # ---------- asosiy API ----------
    def call(self, fn, *args, fallback=None, **kwargs):
        state = self.state()

        if state == 'OPEN':
            metrics.inc('armenu_cb_rejected_total', labels={'service': self.name})
            if fallback is not None:
                return fallback()
            retry_after = float(_redis.hget(self._k('state'), 'timeout') or self.cfg.timeout)
            raise CircuitOpenError(self.name, retry_after)

        if state == 'HALF_OPEN':
            # ⚡ Faqat 1 ta probe o'tsin — qolganlari fallback
            got = _redis.set(self._k('probe'), b'1', nx=True, ex=int(self.cfg.call_timeout) + 2)
            if not got:
                if fallback is not None:
                    return fallback()
                raise CircuitOpenError(self.name, self.cfg.timeout)

        started = time.monotonic()
        try:
            result = fn(*args, **kwargs)
        except self.cfg.error_types as exc:
            self._on_failure(state)
            metrics.inc('armenu_cb_failure_total', labels={'service': self.name})
            logger.warning("[CB] %s xato: %s", self.name, exc)
            if fallback is not None:
                return fallback()
            raise
        else:
            self._on_success(state)
            metrics.observe('armenu_external_api_duration_seconds',
                            time.monotonic() - started, labels={'provider': self.name})
            return result
        finally:
            if state == 'HALF_OPEN':
                _redis.delete(self._k('probe'))

    # Dekorator shakli
    def __call__(self, fallback=None):
        def deco(fn):
            def wrapper(*a, **kw):
                return self.call(fn, *a, fallback=fallback, **kw)
            wrapper.__name__ = fn.__name__
            return wrapper
        return deco
```

5.3 Har tashqi xizmat uchun sozlamalar

Xizmat	fail_threshold	timeout	success_threshold	call_timeout	max_timeout	Fallback
payme	3	120 s	2	8 s	900 s	Click → Uzum → naqd
click	3	120 s	2	8 s	900 s	Uzum → naqd
uzum	3	120 s	2	8 s	900 s	Naqd
ofd_uzkassa	3	120 s	2	5 s	600 s	OFD-2 → navbat
ofd_epos	3	120 s	2	5 s	600 s	Navbat
ikpu_tasnif	5	300 s	1	4 s	1800 s	Lokal IKPU kesh
ai_gemini	5	60 s	2	8 s	300 s	Keyingi AI provayder
ai_claude_haiku	5	60 s	2	8 s	300 s	Keyingi AI provayder
ai_openai_mini	5	60 s	2	8 s	300 s	Keyingi AI provayder
ai_groq	8	30 s	2	5 s	180 s	Keyingi AI provayder
ai_deepseek	8	30 s	2	8 s	180 s	Keyingi AI provayder
ai_claude_sonnet	3	180 s	2	15 s	900 s	FAQ + pgvector
telegram	10	60 s	2	10 s	600 s	Navbat + panel badge
s3_media	5	30 s	2	6 s	300 s	Replica bucket → rasm
meshy_3d	3	300 s	1	30 s	1800 s	Keyingi 3D provayder
smtp_mail	5	300 s	1	10 s	1800 s	Telegram xabar

Ro'yxatga olish (registry):

```
# core/resilience/registry.py
BREAKERS = {
    'payme':       CircuitBreaker('payme',       BreakerConfig(3, 120, 2, call_timeout=8)),
    'ofd_uzkassa': CircuitBreaker('ofd_uzkassa', BreakerConfig(3, 120, 2, call_timeout=5,
                                                               max_timeout=600)),
    'ai_gemini':   CircuitBreaker('ai_gemini',   BreakerConfig(5, 60, 2, call_timeout=8,
                                                               max_timeout=300)),
    # ...
}

def breaker(name) -> CircuitBreaker:
    return BREAKERS[name]
```

Qo'llanilishi:

```
# apps/fiscal/services.py
from core.resilience.registry import breaker
from core.queue import enqueue

def send_fiscal_receipt(order):
    """Chek yuborish — CB + ikki provayder + navbat."""
    def _to_queue():
        enqueue('fiscal', 'retry_receipt', order_id=order.id, delay=300)
        return {'status': 'queued'}

    def _epos():
        return breaker('ofd_epos').call(epos_send, order, fallback=_to_queue)

    return breaker('ofd_uzkassa').call(uzkassa_send, order, fallback=_epos)
```

6. RETRY SIYOSATI

6.1 Operatsiyalar jadvali

Operatsiya	Urinish	Kechikish ketma-ketligi	Jitter	Maks umumiy vaqt	Idempotency kaliti	Tugagach
To'lov webhook qayta ishlash	8	2, 4, 8, 16, 32, 64, 128, 256 s	±30%	24 soat	`provider_txn_id`	DLQ + P1 alert
To'lov holatini so'rash (poll)	24	60 s × 24 (tekis)	±10%	24 soat	`order_id`	Qo'lda tekshiruv
Fiskal chek (OFD)	10	5m, 15m, 1s, 3s, 6s, 12s, 24s, 24s, 24s, 24s	±20%	48 soat	`receipt_uuid`	Qo'lda chek + P1
IKPU qidiruv	3	1, 5, 20 s	±20%	30 s	—	Lokal kesh
Telegram push	5	1, 5, 30 s, 2m, 10m	±20%	1 soat	`notification_id`	DLQ (jim)
AI so'rov (bitta provayder)	2	0,5, 2 s	—	30 s	—	Keyingi provayder
3D generatsiya	3	1m, 5m, 15m	±10%	1 soat	`job_id`	Keyingi provayder → qo'lda
S3 yuklash / o'qish	4	0,2, 1, 3, 8 s	±25%	15 s	`object_key`	Replica bucket
DB deadlock (`40P01`)	3	0,05, 0,2, 1 s	±50%	5 s	tranzaksiya	503 + klient retry
DB serialization fail (`40001`)	3	0,05, 0,2, 1 s	±50%	5 s	tranzaksiya	503
Redis timeout	2	0,05, 0,2 s	±50%	1 s	—	Fail-open (DBga o'tadi)
WS klient reconnect	∞	1, 2, 4, 8, 16, 30, 30… s	±20%	—	—	SSE → polling
CDN purge	3	5, 30, 120 s	±20%	5 daq	`path`	Kesh eskirishini kutish

6.2 Eksponensial backoff kodi

```
# core/resilience/retry.py
import time
import random
import functools
import logging

logger = logging.getLogger('armenu.resilience')


def backoff_delays(base=1.0, factor=2.0, attempts=5, cap=300.0, jitter=0.2):
    """Kechikishlar generatori: base * factor^n, jitter bilan, cap gacha."""
    for n in range(attempts):
        raw = min(base * (factor ** n), cap)
        yield raw * (1 + random.uniform(-jitter, jitter))


# Qayta urinilmaydigan xatolar — bularni retry qilish ZARARLI
NON_RETRYABLE = (
    ValueError,            # noto'g'ri ma'lumot — qayta urinsa ham xato
    PermissionError,
    'HTTP 400', 'HTTP 401', 'HTTP 403', 'HTTP 404', 'HTTP 409', 'HTTP 422',
)


def retryable(exc) -> bool:
    """Xatoni qayta urinish mumkinmi?"""
    if isinstance(exc, NON_RETRYABLE[:2]):
        return False
    status = getattr(exc, 'status_code', None)
    if status is not None:
        if status == 429:
            return True                    # rate limit — kutib qayta urinamiz
        if 400 <= status < 500:
            return False                   # klient xatosi — retry foydasiz
        return True                        # 5xx — retry
    return True                            # tarmoq/timeout — retry


def with_retry(attempts=3, base=1.0, factor=2.0, cap=300.0, jitter=0.2,
               max_total=None, on_give_up=None):
    """
    Dekorator. ⚠ FAQAT idempotent funksiyalarda ishlatiladi!
    """
    def deco(fn):
        @functools.wraps(fn)
        def wrapper(*args, **kwargs):
            started = time.monotonic()
            last = None
            for i, delay in enumerate(backoff_delays(base, factor, attempts, cap, jitter), 1):
                try:
                    return fn(*args, **kwargs)
                except Exception as exc:
                    last = exc
                    if not retryable(exc):
                        logger.warning("[RETRY] %s — qayta urinilmaydi: %s", fn.__name__, exc)
                        raise
                    if i >= attempts:
                        break
                    if max_total and (time.monotonic() - started + delay) > max_total:
                        logger.warning("[RETRY] %s — maks vaqt tugadi", fn.__name__)
                        break
                    logger.info("[RETRY] %s urinish %d/%d, %.2fs kutish: %s",
                                fn.__name__, i, attempts, delay, exc)
                    time.sleep(delay)
            if on_give_up:
                on_give_up(last, *args, **kwargs)
            raise last
        return wrapper
    return deco
```

Namuna:

```
# apps/payments/services.py
@with_retry(attempts=8, base=2.0, factor=2.0, cap=256, jitter=0.3,
            max_total=86400, on_give_up=lambda e, *a, **kw: move_to_dlq('payments', e))
def process_payment_webhook(provider, payload, idempotency_key):
    with transaction.atomic():
        txn, created = PaymentTransaction.objects.select_for_update().get_or_create(
            provider=provider, provider_txn_id=idempotency_key,
            defaults={'raw': payload, 'status': 'processing'},
        )
        if not created and txn.status == 'done':
            return txn          # ⚡ idempotent — ikkinchi marta hech narsa qilmaydi
        ...
```

6.3 ⛔ Qachon retry QILINMAYDI

Holat	Nima uchun	Nima qilinadi
4xx (400/401/403/404/409/422)	Klient xatosi — qayta urinish natijani o'zgartirmaydi	Darhol xato qaytarish + log
Idempotent bo'lmagan yozuv	Dublikat buyurtma/to'lov yaratiladi	Avval idempotency kalit qo'yiladi
Foydalanuvchi kutayotgan HTTP so'rov	Timeout ortadi, UX buziladi	Maks 1 retry, keyin 503 + klient qayta uradi
Circuit breaker OPEN	Yiqilgan xizmatni bombalash — kaskad	Darhol fallback
Poison message (3× crash)	Cheksiz sikl, navbat bloklanadi	DLQ karantín
Validatsiya xatosi	Ma'lumot noto'g'ri — hech qachon o'tmaydi	Darhol rad + admin xabar
`INSUFFICIENT_FUNDS` **(to'lov)**	Biznes xatosi, texnik emas	Foydalanuvchiga xabar
IKPU "kod topilmadi"	Ma'lumot yo'q, qayta urinish yordam bermaydi	Qo'lda kiritish
Migratsiya lock timeout	Retry butun deployni cho'zadi	Deploy to'xtatiladi + rollback

---

7. BULKHEAD (bo'linma izolyatsiyasi)

> **Printsip:** kemaning bir bo'limi suvga to'lsa, boshqalari quruq qoladi.
> Sekin AI so'rovi buyurtma yaratishni bloklamasligi kerak.

7.1 Worker navbatlari izolyatsiyasi

```
┌──────────────────────────────────────────────────────────┐
│                  5 ALOHIDA NAVBAT (bulkhead)             │
├──────────┬──────────┬──────────┬──────────┬──────────────┤
│ model3d  │  fiscal  │  notify  │    ai    │   analytics  │
├──────────┼──────────┼──────────┼──────────┼──────────────┤
│ 2 worker │ 2 worker │ 3 worker │ 2 worker │  1 worker    │
│ 30 daq   │  30 s    │  20 s    │  60 s    │  5 daq  ← job timeout │
│ maks 500 │ maks 5000│ maks 5000│ maks 2000│  maks 50000 ← depth cap │
│ P4       │   P1     │   P3     │   P4     │  P4     ← alert darajasi │
└──────────┴──────────┴──────────┴──────────┴──────────────┘
     ▲          ▲                              ▲
     │          │                              │
  Og'ir,     ⚡ ENG      Navbat to'lsa — faqat o'sha turdagi ish
  sekin      MUHIM       kechikadi. Boshqalar TA'SIRLANMAYDI.
```

Navbat	Worker soni	Job timeout	Maks depth	Prioritet	To'lganda
`fiscal`	2 (S0) / 4 (S3)	30 s	5 000	🟡 P1	Alert + qo'shimcha worker
`notify`	3 / 6	20 s	5 000	🟢 P3	Eskilarini tashlab yuborish (TTL 1 soat)
`ai`	2 / 4	60 s	2 000	🔵 P4	Yangi so'rov rad — "band" javobi
`model3d`	2 / 4	30 daq	500	🔵 P4	Navbatda kutadi, ETA ko'rsatiladi
`analytics`	1 / 2	5 daq	50 000	🔵 P4	Batch'ga o'tadi

systemd shablon:

```
# /etc/systemd/system/armenu-worker@.service
[Unit]
Description=ARMENU worker — %i navbati
After=network.target postgresql.service redis.service
PartOf=armenu.target

[Service]
Type=simple
User=armenu
WorkingDirectory=/srv/armenu
Environment=ARMENU_QUEUE=%i
EnvironmentFile=/srv/armenu/.env
ExecStart=/srv/armenu/venv/bin/python manage.py run_worker --queue=%i
Restart=always
RestartSec=5
# ⚡ Bulkhead: har navbat o'z resurs cheklovida
MemoryMax=1500M
MemoryHigh=1200M
CPUQuota=150%
TasksMax=64
# Sog'lomlik: heartbeat 90 s kelmasa — o'ldiriladi
WatchdogSec=90
NotifyAccess=main

[Install]
WantedBy=armenu.target
```

```
# 5 navbat, har biri kerakli nusxada
systemctl enable --now armenu-worker@fiscal.service
systemctl enable --now armenu-worker@notify.service
systemctl enable --now armenu-worker@ai.service
systemctl enable --now armenu-worker@model3d.service
systemctl enable --now armenu-worker@analytics.service
```

7.2 DB connection pool bo'linishi

> ⚡ **Muammo:** analitika hisoboti 40 ta ulanish egallab, buyurtma yaratish uchun ulanish qolmaydi. Yechim — PgBouncer'da alohida pool.

```
# /etc/pgbouncer/pgbouncer.ini
[databases]
; Yadro — buyurtma, to'lov, menyu (eng katta ulush)
armenu_core   = host=pg-primary dbname=armenu pool_size=60  reserve_pool=10
; Faqat o'qish — menyu, AR, katalog (replica'ga)
armenu_read   = host=pg-replica dbname=armenu pool_size=40  reserve_pool=5
; Worker'lar — fon ishlar
armenu_worker = host=pg-primary dbname=armenu pool_size=25  reserve_pool=5
; Analitika — sekin, og'ir so'rovlar (ALOHIDA, cheklangan)
armenu_report = host=pg-replica dbname=armenu pool_size=8   reserve_pool=0

[pgbouncer]
pool_mode = transaction
max_client_conn = 2000
default_pool_size = 20
server_idle_timeout = 60
query_wait_timeout = 5          ; 5 s kutib ulanish yo'q bo'lsa — xato (tez fail)
```

```
# config/settings.py
DATABASES = {
    'default':   {**BASE_DB, 'NAME': 'armenu_core',   'CONN_MAX_AGE': 0},
    'read':      {**BASE_DB, 'NAME': 'armenu_read',   'CONN_MAX_AGE': 0},
    'worker':    {**BASE_DB, 'NAME': 'armenu_worker', 'CONN_MAX_AGE': 0},
    'report':    {**BASE_DB, 'NAME': 'armenu_report', 'CONN_MAX_AGE': 0,
                  'OPTIONS': {'options': '-c statement_timeout=30000'}},
}

# core/db/router.py
class BulkheadRouter:
    """So'rovni to'g'ri poolga yo'naltiradi."""
    READ_APPS   = {'menu', 'ar3d'}
    REPORT_APPS = {'analytics', 'reports'}

    def db_for_read(self, model, **hints):
        ctx = get_current_context()            # thread-local: 'web' | 'worker' | 'report'
        if ctx == 'report' or model._meta.app_label in self.REPORT_APPS:
            return 'report'
        if ctx == 'worker':
            return 'worker'
        if model._meta.app_label in self.READ_APPS and not in_atomic_block():
            return 'read'                      # ⚡ replica lag xavfi yo'q joyda
        return 'default'

    def db_for_write(self, model, **hints):
        return 'worker' if get_current_context() == 'worker' else 'default'
```

Pool taqsimoti (S3, PG `max_connections = 200`):

Pool	O'lcham	Reserve	Kimga	Nima uchun shuncha
`armenu_core`	60	10	Web — buyurtma, to'lov	Eng kritik, eng katta ulush
`armenu_read`	40	5	Web — menyu, AR (replica)	Trafikning 70% i, lekin yengil
`armenu_worker`	25	5	5 navbat workeri	Fon ishlar, kutishi mumkin
`armenu_report`	8	0	Analitika, eksport	⚡ Qattiq cheklangan — DBni bosmasin
Zaxira	~30		superuser, migratsiya, monitoring	`reserved_connections`
JAMI	**133 + 20 reserve**			200 dan xavfsiz masofada

7.3 Thread / process pool izolyatsiyasi

```python
# core/resilience/bulkhead.py
from concurrent.futures import ThreadPoolExecutor
from threading import Semaphore

class Bulkhead:
    """Bir vaqtda bajarilayotgan chaqiruvlar sonini cheklaydi.
    To'lsa — kutmaydi, darhol fallback (fail fast)."""

    def __init__(self, name, max_concurrent, max_wait=0.0):
        self.name = name
        self._sem = Semaphore(max_concurrent)
        self._max_wait = max_wait

    def run(self, fn, *args, fallback=None, **kwargs):
        acquired = self._sem.acquire(timeout=self._max_wait) if self._max_wait \
                   else self._sem.acquire(blocking=False)
        if not acquired:
            metrics.inc('armenu_bulkhead_rejected_total', labels={'name': self.name})
            if fallback is not None:
                return fallback()
            raise ServiceBusy(self.name)
        try:
            return fn(*args, **kwargs)
        finally:
            self._sem.release()


# Har tashqi yo'nalish uchun alohida bulkhead
BULKHEADS = {
    'ai':      Bulkhead('ai',      max_concurrent=8,  max_wait=0.0),   # web'da AI bloklamasin
    'payment': Bulkhead('payment', max_concurrent=20, max_wait=2.0),
    'ofd':     Bulkhead('ofd',     max_concurrent=10, max_wait=1.0),
    's3':      Bulkhead('s3',      max_concurrent=30, max_wait=0.5),
    'telegram':Bulkhead('telegram',max_concurrent=15, max_wait=0.0),
}
```

gunicorn darajasidagi bulkhead:

```python
# config/gunicorn.py
workers = 4                    # (2 × CPU) + 1
worker_class = 'gthread'
threads = 4                    # worker × threads = 16 bir vaqtda so'rov
worker_connections = 200
max_requests = 1000            # ⚡ memory leak himoyasi
max_requests_jitter = 100      # bir vaqtda restart bo'lmasin
timeout = 30                   # ⚡ 30 s dan uzun so'rov — worker o'ldiriladi
graceful_timeout = 25
keepalive = 5
preload_app = True
```

---

8. TIMEOUT IERARXIYASI

> **Oltin qoida:** har qatlam o'zidan pastdagidan **UZUNROQ**, o'zidan yuqoridagidan **QISQAROQ** timeout'ga ega bo'lishi shart.
> Buzilsa — yuqori qatlam uziladi, past qatlam ish qilishda davom etadi ("orphan work"), resurs behuda sarflanadi.

8.1 Ierarxiya sxemasi (buyurtma yaratish yo'li)

```
   Klient (fetch)        ──────────────────────────────────  25 s
      │
      ▼
   Cloudflare            ──────────────────────────────────   100 s (o'zgarmas)
      │                     (bizdan uzun — biz oldin uzamiz)
      ▼
   nginx proxy_read      ──────────────────────────────────     35 s
      │
      ▼
   gunicorn timeout      ──────────────────────────────        30 s
      │
      ▼
   Django view budget    ──────────────────────────           25 s  (soft)
      │
      ├──► DB statement_timeout    ──────────────            10 s
      │       └──► lock_timeout      ────────                 3 s
      │
      ├──► Redis socket_timeout    ────                      1 s
      │
      └──► Tashqi API (CB call_timeout)
             ├─ To'lov      ──────────                       8 s
             ├─ OFD         ──────────                       5 s
             ├─ AI          ──────────                       8 s
             ├─ S3          ────────                         6 s
             └─ Telegram    ──────────────                  10 s

   ⚡ Qoida: 25 < 30 < 35 < 100 ва 8 < 10 < 25
```

8.2 Qiymatlar jadvali va sabablari

#	Qatlam	Sozlama	Qiymat	Nima uchun aynan shu
1	Klient (JS `AbortController`)	`signal`	**25 s**	Foydalanuvchi 25 s dan ko'p kutmaydi; retry o'zi bo'ladi
2	Cloudflare	Origin timeout	**100 s**	Cloudflare Free'da o'zgarmas; biz undan oldin uzamiz
3	nginx	`proxy_connect_timeout`	**3 s**	Backend tirik bo'lmasa tez aniqlash → keyingi upstream
4	nginx	`proxy_read_timeout`	**35 s**	gunicorn (30 s) dan 5 s uzun — gunicorn o'zi tugatsin
5	nginx	`proxy_send_timeout`	**35 s**	Yuqoridagi bilan bir xil
6	nginx (fayl yuklash)	`client_max_body_size`	**50 M**	3D fayl (glb) maks hajmi
7	gunicorn	`timeout`	**30 s**	Web so'rov 30 s dan uzun bo'lmasligi kerak (>2 s → worker)
8	gunicorn	`graceful_timeout`	**25 s**	Deploy'da inflight so'rov tugashiga vaqt
9	daphne (WS)	`--websocket_timeout`	**3600 s**	WS uzoq yashaydi; ping 30 s bilan tirikligi tekshiriladi
10	daphne	`--application-close-timeout`	**10 s**	Restart'da uzilish tez bo'lsin, klient reconnect qiladi
11	Django view	Soft budget (middleware)	**25 s**	gunicorn 30 s dan oldin o'zimiz 503 qaytaramiz (toza log)
12	PostgreSQL	`statement_timeout` (web)	**10 s**	Bitta so'rov 10 s → aniq bug; DBni himoya qiladi
13	PostgreSQL	`statement_timeout` (report)	**30 s**	Analitika og'irroq, lekin cheksiz emas
14	PostgreSQL	`statement_timeout` (worker)	**300 s**	3D/import ishlari uzun bo'lishi mumkin
15	PostgreSQL	`lock_timeout`	**3 s**	⚡ Migratsiya jadval lock'ini kutib turmasin
16	PostgreSQL	`idle_in_transaction_session_timeout`	**30 s**	Ochiq tranzaksiya lock ushlab turmasin
17	PgBouncer	`query_wait_timeout`	**5 s**	Pool to'lsa 5 s kutib xato — tez fail
18	Redis	`socket_timeout`	**1 s**	Kesh sekin bo'lsa DBga o'tgan tezroq
19	Redis	`socket_connect_timeout`	**0,5 s**	Redis o'chgan bo'lsa darhol bilish
20	Channels layer	`expiry`	**60 s**	Yetkazilmagan WS xabar 60 s dan ko'p yashamasin
21	To'lov API	CB `call_timeout`	**8 s**	Provayder SLA ~3 s; 8 s — xavfsiz chegara
22	OFD API	CB `call_timeout`	**5 s**	Fiskal navbatga tushishi normal — uzoq kutmaymiz
23	AI API	CB `call_timeout`	**8 s**	8 s dan sekin bo'lsa keyingi provayder tezroq
24	S3	`connect/read timeout`	**2 s / 6 s**	Fayl yetkazish; CDN oldida turadi
25	Telegram	`read_timeout`	**10 s**	Fon ish, shoshilinch emas
26	Worker job	`job_timeout` (navbat bo'yicha)	**20 s – 30 daq**	§7.1 jadvali
27	Healer check	Har check	**10 s**	Bitta check daemon siklini bloklamasin

8.3 Konfiguratsiya kodi

```
# /etc/nginx/sites-available/armenu.conf
upstream armenu_web {
    least_conn;
    server 10.0.1.11:8060 max_fails=3 fail_timeout=10s;
    server 10.0.1.12:8060 max_fails=3 fail_timeout=10s;
    server 10.0.1.13:8060 max_fails=3 fail_timeout=10s backup;
    keepalive 64;
}

upstream armenu_ws {
    ip_hash;                                # ⚡ sticky — WS uchun muhim
    server 10.0.1.21:8061 max_fails=2 fail_timeout=10s;
    server 10.0.1.22:8061 max_fails=2 fail_timeout=10s;
}

server {
    listen 443 ssl http2;
    server_name armenu.uz *.armenu.uz;

    # ---- Timeout ierarxiyasi ----
    proxy_connect_timeout   3s;
    proxy_send_timeout      35s;
    proxy_read_timeout      35s;
    send_timeout            35s;
    client_body_timeout     15s;
    client_header_timeout   10s;
    client_max_body_size    50M;

    location / {
        proxy_pass http://armenu_web;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;

        # ⚡ Fault tolerance: node xato bersa keyingisiga o'tadi
        proxy_next_upstream error timeout http_502 http_503 http_504;
        proxy_next_upstream_tries 2;
        proxy_next_upstream_timeout 10s;
    }

    location /ws/ {
        proxy_pass http://armenu_ws;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_read_timeout  3600s;          # ⚡ WS uchun uzun
        proxy_send_timeout  3600s;
    }

    # ⚡ Statik menyu snapshot — DB yiqilganda ishlaydigan zaxira
    location /snapshot/ {
        proxy_pass https://cdn.armenu.uz/snapshots/;
        proxy_cache_valid 200 15m;
        add_header X-Fallback-Mode "static-snapshot" always;
    }

    location = /healthz {
        proxy_pass http://armenu_web;
        proxy_read_timeout 3s;              # health-check tez bo'lsin
        access_log off;
    }
}
```

```
# config/settings.py — DB timeout'lari
DATABASES['default']['OPTIONS'] = {
    'options': '-c statement_timeout=10000 '
               '-c lock_timeout=3000 '
               '-c idle_in_transaction_session_timeout=30000',
    'connect_timeout': 3,
}

CACHES['default']['OPTIONS'] = {
    'socket_timeout': 1.0,
    'socket_connect_timeout': 0.5,
    'retry_on_timeout': True,
    'ignore_exceptions': True,      # ⚡ Redis o'chsa — kesh miss, xato emas
}
```

```
# core/middleware/budget.py — soft timeout
class RequestBudgetMiddleware:
    """gunicorn 30 s da o'ldirishidan oldin biz o'zimiz 503 qaytaramiz —
    log toza bo'ladi, klient aniq xabar oladi."""
    BUDGET = 25.0

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        request._budget_started = time.monotonic()
        response = self.get_response(request)
        elapsed = time.monotonic() - request._budget_started
        if elapsed > self.BUDGET * 0.8:
            logger.warning("[BUDGET] %s %s — %.1fs (byudjet %.0fs)",
                           request.method, request.path, elapsed, self.BUDGET)
        response['X-Response-Time'] = f'{elapsed:.3f}'
        return response
```

---

9. ⚡ SELF-HEALING — 15 MEXANIZM

> **Falsafa:** tunda soat 02:00 da odam uyg'otilmasin. Tizim o'zi tuzatsin.
> Tuzata olmasa — **aniq nima qilish kerakligini yozib** alert yuborsin.

9.1 Mexanizmlar jadvali

#	Mexanizm	Nima aniqlaydi	Qanday tuzatadi	Davriylik	⛔ Tuzata olmasa
1	**Process supervisor**	Process o'ldi (systemd)	`Restart=always`, `RestartSec=5`	Darhol	5 marta 60 s ichida qayta o'lsa → `start-limit` → P1 alert
2	**LB health-check**	Node `/healthz` 3× xato	Node upstream'dan chiqariladi	3 s	Barcha node chiqsa → P1 + statik snapshot rejimi
3	**Orphan job recovery**	`running` job, `heartbeat_at` > 15 daq	`queued` ga qaytaradi, `attempts += 1`	60 s	`attempts >= max` → DLQ + P3 alert
4	**Stuck order detector**	`cooking` holatda > 45 daq	KDS + admin ga WS alert, badge	5 daq	Odam kerak → restoran adminiga Telegram
5	**DB connection reaper**	`idle in transaction` > 5 daq ulanish	`pg_terminate_backend()`	5 daq	Pool 95% > 10 daq → P2 + web restart taklifi
6	**Redis memory guard**	`used_memory` > 85% `maxmemory`	Eng eski kesh prefikslarini `SCAN+DEL`	2 daq	> 95% → `allkeys-lru` majburiy + P2
7	**Disk cleanup**	Disk > 85%	Log rotate, `/tmp` tozalash, eski 3D temp o'chirish	10 daq	> 95% → P1 + write bloklanadi (read-only rejim)
8	**Worker memory guard**	Worker RSS > 80% `MemoryMax`	Graceful restart (joriy job tugagach)	60 s	3× ketma-ket → P2 + memory leak tekshiruvi
9	**Fiscal queue drainer**	`fiscal` navbat depth > 50, 10 daq	Qo'shimcha worker ko'taradi, CB holatini tekshiradi	2 daq	OFD ikkalasi ham OPEN → P1 + qo'lda chek rejimi
10	**Cache warm-up**	Deploy/restart'dan keyin `cache_hit_ratio` < 30%	Top-100 restoran menyusini oldindan yuklaydi	Restart'da	Warm-up xato → jim (kesh o'zi to'ladi)
11	**CB auto-probe**	CB OPEN > 5 daq	HALF_OPEN probe'ni majburiy ishga tushiradi	60 s	30 daq OPEN → P2 provayder alert
12	**WS ghost cleanup**	Redis'da ulanish yozuvi bor, socket yo'q	`channel_layer.group_discard`, yozuv o'chiriladi	5 daq	Ghost > 1000 → daphne restart taklifi (P3)
13	**Replica lag healer**	`pg_stat_replication.lag` > 10 s	Replica'ni router'dan chiqaradi (o'qish primary'ga)	30 s	Lag > 300 s → replica qayta seed (P2)
14	**Payment reconciler**	`pending` to'lov > 5 daq, webhook yo'q	Provayderdan `check_transaction` so'raydi, holatni sinxronlaydi	5 daq	24 soat `pending` → P1 + moliyaviy tekshiruv
15	**Backup verify**	Kechagi backup tiklanadimi?	Staging'ga `pg_restore`, `SELECT count(*)` tekshiruvi	Kunlik 04:00	Tiklash xato → 🔴 P1 (eng jiddiy — DR buzilgan)

9.2 To'liq daemon kodi

```
# core/healing/daemon.py — armenu-healer.service
"""
ARMENU Self-Healing Daemon.
Har 60 soniyada 15 ta tekshiruvni bajaradi va muammolarni O'ZI tuzatadi.
Tuzata olmasa — nima qilish kerakligini yozib alert yuboradi.

Ishga tushirish:  python manage.py run_healer
systemd:          armenu-healer.service (Restart=always)
"""
import os
import time
import shutil
import logging
import subprocess
from datetime import timedelta

import redis
from django.db import connections, transaction
from django.utils import timezone
from django.core.cache import caches

from core.metrics import metrics
from core.alerts import alert_oncall, notify_telegram
from core.resilience.registry import BREAKERS
from core.queue import enqueue, queue_depth, move_to_dlq
from apps.orders.models import Order
from apps.payments.models import PaymentTransaction
from apps.jobs.models import Job

logger = logging.getLogger('armenu.healer')

CYCLE_SECONDS = 60
CHECK_TIMEOUT = 10          # bitta check maks 10 s


# ═══════════════════════════════════════════════════════════════
#  1. ORPHAN JOB — worker o'lganda `running`da qolgan ishlar
# ═══════════════════════════════════════════════════════════════
def check_orphan_jobs():
    cutoff = timezone.now() - timedelta(minutes=15)
    orphans = list(Job.objects.using('worker')
                   .filter(status='running', heartbeat_at__lt=cutoff)[:200])
    return orphans or None


def heal_orphan_jobs(orphans):
    requeued, dead = 0, 0
    for job in orphans:
        with transaction.atomic(using='worker'):
            job = Job.objects.using('worker').select_for_update().get(pk=job.pk)
            if job.status != 'running':
                continue
            if job.attempts < job.max_attempts:
                job.status = 'queued'
                job.attempts += 1
                job.heartbeat_at = None
                job.save(update_fields=['status', 'attempts', 'heartbeat_at'])
                enqueue(job.queue, job_id=job.id)
                requeued += 1
            else:
                job.status = 'failed'
                job.save(update_fields=['status'])
                move_to_dlq(job)
                dead += 1
    logger.info("[HEAL] orphan_jobs: %d qayta navbatga, %d DLQ", requeued, dead)
    if dead:
```

```
        notify_telegram(f"🔴 {dead} ta job DLQ'ga tushdi (max urinish tugadi)")
    return True                      # ✔ har doim tuzata oladi


# ═══════════════════════════════════════════════════════════════
#  2. STUCK ORDER — 45 daqiqadan beri `cooking`
# ═══════════════════════════════════════════════════════════════
def check_stuck_orders():
    cutoff = timezone.now() - timedelta(minutes=45)
    stuck = list(Order.objects.filter(status='cooking', updated_at__lt=cutoff)
                 .select_related('restaurant')[:100])
    return stuck or None


def heal_stuck_orders(orders):
    from core.ws import ws_send
    for o in orders:
        mins = int((timezone.now() - o.updated_at).total_seconds() // 60)
        ws_send(f'rest_{o.restaurant_id}_kds',
                {'type': 'alert.stuck_order', 'order_id': o.id, 'minutes': mins})
        ws_send(f'rest_{o.restaurant_id}_admin',
                {'type': 'alert.stuck_order', 'order_id': o.id, 'minutes': mins})
        notify_telegram(
            f"⚠ {o.restaurant.name}: buyurtma #{o.number} {mins} daqiqadan beri "
            f"tayyorlanmoqda. Oshxonani tekshiring.",
            chat_id=o.restaurant.admin_telegram_id,
        )
    return False       # ☻ avtomatik tuzatilmaydi — odam kerak, lekin xabar berildi


# ═══════════════════════════════════════════════════════════════
#  3. DB CONNECTION LEAK — uzoq `idle in transaction`
# ═══════════════════════════════════════════════════════════════
def check_db_connections():
    with connections['default'].cursor() as cur:
        cur.execute("""
            SELECT count(*) FILTER (WHERE state = 'idle in transaction'
                                    AND state_change < now() - interval '5 minutes'),
                   count(*),
                   current_setting('max_connections')::int
            FROM pg_stat_activity WHERE datname = current_database()
        """)
        leaked, total, maxc = cur.fetchone()
    metrics.gauge('armenu_db_connections_used', total)
    if leaked > 0 or total > maxc * 0.85:
        return {'leaked': leaked, 'total': total, 'max': maxc}
    return None


def heal_db_connections(problem):
    if problem['leaked'] > 0:
        with connections['default'].cursor() as cur:
            cur.execute("""
                SELECT pg_terminate_backend(pid) FROM pg_stat_activity
                WHERE datname = current_database()
                  AND state = 'idle in transaction'
                  AND state_change < now() - interval '5 minutes'
            """)
            killed = cur.rowcount
        logger.warning("[HEAL] db_connections: %d ta osilgan ulanish yopildi", killed)
        return True
    # Faqat to'lib ketgan bo'lsa — o'zimiz tuzata olmaymiz
    alert_oncall(
        f"🔴 DB pool {problem['total']}/{problem['max']} (85%+).\n"
        f"QILISH: 1) `SELECT query, count(*) FROM pg_stat_activity GROUP BY 1 "
        f"ORDER BY 2 DESC LIMIT 10;` 2) sekin so'rovni toping 3) web node qo'shing",
        severity='P2')
    return False


# ═══════════════════════════════════════════════════════════════
#  4. REDIS MEMORY
# ═══════════════════════════════════════════════════════════════
def check_redis_memory():
    r = caches['default'].client.get_client(write=True)
    info = r.info('memory')
    used, maxmem = info['used_memory'], info.get('maxmemory') or 0
    if not maxmem:
        return None
    ratio = used / maxmem
    metrics.gauge('armenu_redis_memory_ratio', ratio)
    return {'ratio': ratio, 'used': used, 'max': maxmem} if ratio > 0.85 else None


def heal_redis_memory(problem):
    r = caches['default'].client.get_client(write=True)
    # Eng arzon keshni birinchi tashlaymiz (qayta hisoblash oson)
    for prefix in ('menu_render:*', 'ai_answer:*', 'analytics:*', 'session_meta:*'):
        deleted = 0
        for key in r.scan_iter(match=prefix, count=500):
            r.unlink(key)
            deleted += 1
            if deleted >= 20000:
                break
        logger.info("[HEAL] redis: %s → %d kalit o'chirildi", prefix, deleted)
        if r.info('memory')['used_memory'] / problem['max'] < 0.70:
            return True
    if problem['ratio'] > 0.95:
        r.config_set('maxmemory-policy', 'allkeys-lru')
        alert_oncall("🔴 Redis > 95% — allkeys-lru yoqildi. RAM qo'shing yoki "
                     "cluster shard'ini ko'paytiring.", severity='P2')
        return False
    return True


# ═══════════════════════════════════════════════════════════════
#  5. DISK
# ═══════════════════════════════════════════════════════════════
CLEANUP_PATHS = [
    ('/srv/armenu/logs',      30),     # yo'l, necha kundan eski)
    ('/srv/armenu/tmp',        1),
    ('/srv/armenu/media/3d_tmp', 2),
    ('/var/log/nginx',        14),
]


def check_disk():
    problems = []
    for mount in ('/', '/srv'):
```

```
        if not os.path.ismount(mount) and mount != '/':
            continue
        total, used, free = shutil.disk_usage(mount)
        pct = used / total
        metrics.gauge('armenu_disk_used_ratio', pct, labels={'mount': mount})
        if pct > 0.85:
            problems.append({'mount': mount, 'pct': pct, 'free_gb': free / 1e9})
    return problems or None


def heal_disk(problems):
    freed = 0
    for path, days in CLEANUP_PATHS:
        if not os.path.isdir(path):
            continue
        out = subprocess.run(
            ['find', path, '-type', 'f', '-mtime', f'+{days}', '-delete', '-print'],
            capture_output=True, text=True, timeout=60)
        freed += len(out.stdout.splitlines())
    subprocess.run(['journalctl', '--vacuum-size=500M'], timeout=60)
    logger.warning("[HEAL] disk: %d ta fayl o'chirildi", freed)

    still_bad = check_disk()
    if still_bad and any(p['pct'] > 0.95 for p in still_bad):
        # ⚡ Oxirgi chora: read-only rejim
        caches['default'].set('armenu:readonly_mode', '1', 3600)
        alert_oncall(
            "🔴 Disk > 95% — READ-ONLY rejim yoqildi (yozish bloklandi).\n"
            "QILISH: 1) `du -sh /srv/armenu/* | sort -h` 2) media/backup ko'chiring "
            "3) disk kengaytiring 4) `redis-cli DEL armenu:readonly_mode`",
            severity='P1')
        return False
    return not still_bad


# ═══════════════════════════════════════════════════════════════
#  6. WORKER TIRIKLIGI VA XOTIRASI
# ═══════════════════════════════════════════════════════════════
QUEUES = ('fiscal', 'notify', 'ai', 'model3d', 'analytics')


def check_workers():
    problems = []
    for q in QUEUES:
        unit = f'armenu-worker@{q}.service'
        st = subprocess.run(['systemctl', 'is-active', unit],
                            capture_output=True, text=True).stdout.strip()
        alive = st == 'active'
        metrics.gauge('armenu_worker_alive', 1 if alive else 0, labels={'queue': q})
        if not alive:
            problems.append({'queue': q, 'kind': 'dead', 'unit': unit})
            continue
        rss = _unit_memory_bytes(unit)
        if rss and rss > 0.80 * 1500 * 1024 * 1024:      # MemoryMax=1500M
            problems.append({'queue': q, 'kind': 'memory', 'unit': unit, 'rss': rss})
    return problems or None


def heal_workers(problems):
    ok = True
    for p in problems:
        if p['kind'] == 'dead':
            subprocess.run(['systemctl', 'restart', p['unit']], timeout=30)
            logger.warning("[HEAL] worker %s qayta ishga tushirildi", p['queue'])
        else:
            # Graceful: joriy job tugagach o'zi chiqadi
            subprocess.run(['systemctl', 'kill', '-s', 'SIGTERM', p['unit']], timeout=30)
            logger.warning("[HEAL] worker %s xotira %.0f MB — graceful restart",
                           p['queue'], p['rss'] / 1e6)
        time.sleep(2)
        if subprocess.run(['systemctl', 'is-active', p['unit']],
                          capture_output=True, text=True).stdout.strip() != 'active':
            ok = False
    if not ok:
        alert_oncall("🟠 Worker qayta ishga tushmadi.\n"
                     "QILISH: `journalctl -u armenu-worker@<q> -n 100`", severity='P2')
    return ok


# ═══════════════════════════════════════════════════════════════
#  7. FISKAL NAVBAT
# ═══════════════════════════════════════════════════════════════
def check_fiscal_queue():
    depth = queue_depth('fiscal')
    metrics.gauge('armenu_fiscal_queue_depth', depth)
    oldest = (PaymentTransaction.objects
              .filter(fiscal_status='pending')
              .order_by('created_at').values_list('created_at', flat=True).first())
    age_min = (timezone.now() - oldest).total_seconds() / 60 if oldest else 0
    if depth > 50 or age_min > 30:
        return {'depth': depth, 'age_min': age_min}
    return None


def heal_fiscal_queue(problem):
    # 1) OFD circuit breaker holatini tekshiramiz
    states = {n: BREAKERS[n].state() for n in ('ofd_uzkassa', 'ofd_epos')}
    if all(s == 'OPEN' for s in states.values()):
        alert_oncall(
            f"🔴 Ikkala OFD ham OPEN. Navbat: {problem['depth']} chek, "
            f"eng eskisi {problem['age_min']:.0f} daq.\n"
            f"QILISH: 1) OFD status sahifasini tekshiring 2) kassa apparatidan "
            f"qo'lda chek 3) tiklangach `manage.py drain_fiscal_queue`",
            severity='P1')
        return False
    # 2) Vaqtincha qo'shimcha worker
    subprocess.run(['systemctl', 'start', 'armenu-worker@fiscal-burst.service'], timeout=30)
    logger.warning("[HEAL] fiscal: burst worker yoqildi (depth=%d)", problem['depth'])
    return True


# ═══════════════════════════════════════════════════════════════
#  8. WS "GHOST" ULANISHLAR
# ═══════════════════════════════════════════════════════════════
def check_ws_ghosts():
    r = caches['default'].client.get_client(write=True)
    cutoff = time.time() - 300
    ghosts = [k for k in r.scan_iter(match='asgi:group:*', count=500)
              if (r.zcount(k, '-inf', cutoff) or 0) > 0]
```

```
    return ghosts or None


def heal_ws_ghosts(ghosts):
    r = caches['default'].client.get_client(write=True)
    cutoff = time.time() - 300
    removed = sum(r.zremrangebyscore(k, '-inf', cutoff) for k in ghosts)
    logger.info("[HEAL] ws_ghosts: %d ta o'lik ulanish tozalandi", removed)
    metrics.inc('armenu_ws_ghosts_cleaned_total', removed)
    return True


# ═══════════════════════════════════════════════════════════
#  9. CIRCUIT BREAKER PROBE
# ═══════════════════════════════════════════════════════════
def check_stale_breakers():
    stale = []
    for name, cb in BREAKERS.items():
        st = cb.state()
        metrics.gauge('armenu_circuit_breaker_state',
                      {'CLOSED': 0, 'HALF_OPEN': 1, 'OPEN': 2}[st],
                      labels={'service': name})
        if st == 'OPEN':
            stale.append(name)
    return stale or None


def heal_stale_breakers(names):
    from core.resilience.probes import PROBES
    healed = []
    for name in names:
        probe = PROBES.get(name)
        if not probe:
            continue
        try:
            BREAKERS[name].call(probe)     # HALF_OPEN bo'lsa probe o'tadi
            healed.append(name)
        except Exception:
            pass
    if healed:
        logger.info("[HEAL] breakers tiklandi: %s", healed)
    return bool(healed)


# ═══════════════════════════════════════════════════════════
# 10. TO'LOV RECONCILIATION — webhook kelmagan to'lovlar
# ═══════════════════════════════════════════════════════════
def check_pending_payments():
    cutoff = timezone.now() - timedelta(minutes=5)
    pending = list(PaymentTransaction.objects
                   .filter(status='pending', created_at__lt=cutoff)
                   .exclude(created_at__lt=timezone.now() - timedelta(hours=24))[:200])
    return pending or None


def heal_pending_payments(pending):
    from apps.payments.gateways import get_gateway
    resolved = 0
    for txn in pending:
        try:
            gw = get_gateway(txn.provider)
            remote = BREAKERS[txn.provider].call(gw.check_transaction, txn.provider_txn_id)
        except Exception as e:
            logger.warning("[HEAL] payment %s tekshirilmadi: %s", txn.id, e)
            continue
        if remote['status'] in ('paid', 'cancelled', 'failed'):
            gw.apply_status(txn, remote)      # ⚡ idempotent
            resolved += 1
    logger.info("[HEAL] payments: %d/%d ta hal qilindi", resolved, len(pending))

    very_old = [t for t in pending
                if (timezone.now() - t.created_at) > timedelta(hours=6)]
    if very_old:
        alert_oncall(
            f"🔴 {len(very_old)} ta to'lov 6 soatdan beri `pending`.\n"
            f"QILISH: provayder kabinetida tekshiring, "
            f"`manage.py reconcile_payments --since=6h --force`",
            severity='P1')
        return False
    return True


# ═══════════════════════════════════════════════════════════
# 11. REPLICA LAG
# ═══════════════════════════════════════════════════════════
def check_replica_lag():
    try:
        with connections['read'].cursor() as cur:
            cur.execute("SELECT EXTRACT(EPOCH FROM (now() - pg_last_xact_replay_timestamp()))")
            lag = cur.fetchone()[0] or 0
    except Exception as e:
        return {'lag': None, 'error': str(e)}
    metrics.gauge('armenu_replica_lag_seconds', lag)
    return {'lag': lag} if lag > 10 else None


def heal_replica_lag(problem):
    # Replica'ni o'qish marshrutidan chiqaramiz — primary'ga o'tamiz
    caches['default'].set('armenu:replica_disabled', '1', 300)
    logger.warning("[HEAL] replica lag %.0fs — o'qish primary'ga o'tkazildi",
                   problem.get('lag') or -1)
    if (problem.get('lag') or 0) > 300 or problem.get('error'):
        alert_oncall(
            f"🟠 Replica lag {problem.get('lag')}s / xato: {problem.get('error')}.\n"
            f"QILISH: 1) `SELECT * FROM pg_stat_replication;` primary'da "
            f"2) replica diskini tekshiring 3) kerak bo'lsa `pg_basebackup` bilan qayta seed",
            severity='P2')
        return False
    return True


# ═══════════════════════════════════════════════════════════
# 12. KESH ISITISH (warm-up)
# ═══════════════════════════════════════════════════════════
def check_cache_warmth():
    r = caches['default'].client.get_client(write=True)
    st = r.info('stats')
    hits, misses = st.get('keyspace_hits', 0), st.get('keyspace_misses', 0)
    ratio = hits / (hits + misses) if (hits + misses) > 1000 else 1.0
```

```
    metrics.gauge('armenu_cache_hit_ratio', ratio)
    return {'ratio': ratio} if ratio < 0.30 else None


def heal_cache_warmth(problem):
    enqueue('analytics', 'warm_menu_cache', top_n=100)
    logger.info("[HEAL] cache warm-up navbatga qo'yildi (hit=%.2f)", problem['ratio'])
    return True


# ═══════════════════════════════════════════════════════════════
# 13. STATIK SNAPSHOT YANGILIGI
# ═══════════════════════════════════════════════════════════════
def check_snapshot_freshness():
    from core.storage import s3_head
    meta = s3_head('snapshots/_manifest.json')
    if not meta:
        return {'age_min': 9999}
    age = (timezone.now() - meta['last_modified']).total_seconds() / 60
    return {'age_min': age} if age > 45 else None


def heal_snapshot_freshness(problem):
    enqueue('analytics', 'publish_menu_snapshot')
    logger.warning("[HEAL] snapshot %.0f daqiqa eski — qayta generatsiya", problem['age_min'])
    return True


# ═══════════════════════════════════════════════════════════════
# 14. NAVBAT CHUQURLIGI (umumiy bulkhead nazorati)
# ═══════════════════════════════════════════════════════════════
QUEUE_CAPS = {'fiscal': 5000, 'notify': 5000, 'ai': 2000,
              'model3d': 500, 'analytics': 50000}


def check_queue_depths():
    bad = {}
    for q, cap in QUEUE_CAPS.items():
        d = queue_depth(q)
        metrics.gauge('armenu_queue_depth', d, labels={'queue': q})
        if d > cap * 0.8:
            bad[q] = {'depth': d, 'cap': cap}
    return bad or None


def heal_queue_depths(bad):
    ok = True
    for q, info in bad.items():
        if q == 'notify':
            from core.queue import drop_expired
            dropped = drop_expired('notify', ttl_seconds=3600)
            logger.warning("[HEAL] notify: %d ta eskirgan xabar tashlandi", dropped)
        elif q == 'ai':
            caches['default'].set('armenu:ai_degraded', '1', 600)   # FAQ rejimi
            logger.warning("[HEAL] ai navbat to'la — degraded rejim 10 daq")
        else:
            alert_oncall(f"🟧 `{q}` navbat {info['depth']}/{info['cap']}.\n"
                         f"QILISH: worker qo'shing yoki job manbasini tekshiring",
                         severity='P2')
            ok = False
    return ok


# ═══════════════════════════════════════════════════════════════
# 15. BACKUP TEKSHIRUVI (kunlik)
# ═══════════════════════════════════════════════════════════════
def check_backup_health():
    from core.backup import latest_backup_meta
    meta = latest_backup_meta()
    if not meta:
        return {'reason': 'backup topilmadi'}
    age_h = (timezone.now() - meta['created_at']).total_seconds() / 3600
    if age_h > 26:
        return {'reason': f'backup {age_h:.0f} soat eski'}
    if not meta.get('verified_at') or \
       (timezone.now() - meta['verified_at']).days > 1:
        return {'reason': 'tekshirilmagan'}
    return None


def heal_backup_health(problem):
    from core.backup import run_backup, verify_backup_on_staging
    if 'eski' in problem['reason'] or 'topilmadi' in problem['reason']:
        run_backup(kind='full')
        return True
    ok = verify_backup_on_staging()          # pg_restore + count tekshiruvi
    if not ok:
        alert_oncall(
            "🔴🔴 BACKUP TIKLANMADI — falokatdan tiklash IMKONSIZ.\n"
            "QILISH: 1) `manage.py backup_verify --verbose` log 2) yangi to'liq "
            "backup 3) S3 bucket va shifrlash kalitini tekshiring",
            severity='P1')
    return ok


# ═══════════════════════════════════════════════════════════════
#  REESTR VA ASOSIY SIKL
# ═══════════════════════════════════════════════════════════════
CHECKS = [
    # (nom,                check,                      heal,                      har-necha-sikl)
    ('orphan_jobs',        check_orphan_jobs,          heal_orphan_jobs,          1),
    ('stuck_orders',       check_stuck_orders,         heal_stuck_orders,         5),
    ('db_connections',     check_db_connections,       heal_db_connections,       5),
    ('redis_memory',       check_redis_memory,         heal_redis_memory,         2),
    ('disk_space',         check_disk,                 heal_disk,                10),
    ('workers_alive',      check_workers,              heal_workers,              1),
    ('fiscal_queue',       check_fiscal_queue,         heal_fiscal_queue,         2),
    ('ws_ghosts',          check_ws_ghosts,            heal_ws_ghosts,            5),
    ('stale_breakers',     check_stale_breakers,       heal_stale_breakers,       1),
    ('pending_payments',   check_pending_payments,     heal_pending_payments,     5),
    ('replica_lag',        check_replica_lag,          heal_replica_lag,          1),
    ('cache_warmth',       check_cache_warmth,         heal_cache_warmth,        15),
    ('snapshot_fresh',     check_snapshot_freshness,   heal_snapshot_freshness, 15),
    ('queue_depths',       check_queue_depths,         heal_queue_depths,         2),
    ('backup_health',      check_backup_health,        heal_backup_health,     1440),
]


def run_cycle(tick: int):
```

```
    for name, check, heal, every in CHECKS:
        if tick % every != 0:
            continue
        started = time.monotonic()
        try:
            problem = check()
            if not problem:
                metrics.gauge('armenu_health_check', 1, labels={'check': name})
                continue

            logger.warning("[HEAL] 🔴 Muammo: %s — %s", name, problem)
            metrics.gauge('armenu_health_check', 0, labels={'check': name})
            metrics.inc('armenu_heal_attempt_total', labels={'check': name})

            healed = heal(problem)

            if healed:
                logger.info("[HEAL] ✅ Tuzatildi: %s", name)
                metrics.inc('armenu_heal_success_total', labels={'check': name})
                notify_telegram(f"🔧 Avtomatik tuzatildi: `{name}`")
            else:
                logger.error("[HEAL] ❌ Tuzatib bo'lmadi: %s", name)
                metrics.inc('armenu_heal_failed_total', labels={'check': name})
                # alert check funksiyasi ichida aniq ko'rsatma bilan yuborilgan

        except Exception:
            logger.exception("[HEAL] Check xatosi: %s", name)
            metrics.inc('armenu_heal_error_total', labels={'check': name})
        finally:
            metrics.observe('armenu_heal_duration_seconds',
                            time.monotonic() - started, labels={'check': name})


def main():
    logger.info("[HEAL] Self-healing daemon ishga tushdi (%d check, %ds sikl)",
                len(CHECKS), CYCLE_SECONDS)
    tick = 0
    while True:
        tick += 1
        started = time.monotonic()
        run_cycle(tick)
        sd_notify_watchdog()                       # systemd WatchdogSec
        elapsed = time.monotonic() - started
        if elapsed > CYCLE_SECONDS:
            logger.warning("[HEAL] Sikl %.1fs davom etdi (byudjet %ds)",
                           elapsed, CYCLE_SECONDS)
        time.sleep(max(0.0, CYCLE_SECONDS - elapsed))
```

```
# /etc/systemd/system/armenu-healer.service
[Unit]
Description=ARMENU Self-Healing Daemon
After=network.target postgresql.service redis.service

[Service]
Type=notify
User=root                       # systemctl restart va pg_terminate uchun
WorkingDirectory=/srv/armenu
EnvironmentFile=/srv/armenu/.env
ExecStart=/srv/armenu/venv/bin/python manage.py run_healer
Restart=always
RestartSec=10
WatchdogSec=180                 # daemon o'zi osilib qolsa — restart
MemoryMax=512M

[Install]
WantedBy=multi-user.target
```

9.3 Healer'ning o'zi yiqilsa?

Himoya	Mexanizm
Process o'ldi	`Restart=always` + `RestartSec=10`
Osilib qoldi	`WatchdogSec=180` — 3 daqiqa signal kelmasa systemd o'ldiradi
Butunlay ishlamayapti	Tashqi monitoring (UptimeRobot) `armenu_heal_cycle_total` metrikasini kuzatadi — 5 daqiqa o'smasa P2 alert
Xato tuzatish qilyapti	Har `heal_*` faqat **teskari qaytariladigan** amal bajaradi; hech qachon `DELETE FROM`, hech qachon `DROP`
Healer alert bombardimoni	Har `check` uchun alert 1 soatda 1 marta (`alert_oncall` ichida dedup)

---

10. CHAOS ENGINEERING

10.1 Prinsiplar

#	Qoida
1	**Gipoteza avval.** "Redis o'chsa, menyu DBdan 250 ms da keladi" — yozib qo'yiladi
2	**Staging'da boshlanadi**, 3 muvaffaqiyatli takrordan keyin production'da
3	**Ish vaqtida** (10:00–15:00), jamoa onlayn, restoranlar past yuklamada
4	**"Blast radius" cheklangan** — 1 node, 1 provayder, 1 navbat
5	**Abort tugmasi bor** — `manage.py chaos_abort` hammasini tiklaydi
6	**Natija hujjatlashtiriladi** — kutilgan vs haqiqiy, farqi bo'lsa → ticket

10.2 Oylik test rejasi

Hafta	Kun	Ssenariy	Blast radius
1	Seshanba	CH-1, CH-2 (node, worker)	1 node
2	Seshanba	CH-3, CH-4 (Redis, WS)	Butun kesh qatlami
3	Seshanba	CH-5, CH-6 (DB failover, replica lag)	DB qatlami
4	Seshanba	CH-7, CH-8 (tashqi API, tarmoq)	Tashqi bog'lanishlar
Chorak	Shanba	**DR mashqi** — to'liq backupdan tiklash	Staging (izolyatsiya)

10.3 8 ta ssenariy

ID	Test	Qanday qilinadi	✅ Kutilgan natija	⛔ Muvaffaqiyatsiz agar
CH-1	Tasodifiy web node o'chirish	`systemctl stop armenu` (1 node)	Foydalanuvchi sezmaydi. LB 3–9 s da chiqaradi. 5xx < 0,1%	5xx > 1% yoki > 15 s uzilish
CH-2	Worker o'ldirish (`fiscal`)	`systemctl kill -s SIGKILL armenu-worker@fiscal`	Job `running` da qoladi → healer 60 s da qayta navbatga. Chek yo'qolmaydi	Chek yo'qolsa yoki 2× yuborilsa
CH-3	Redis to'liq o'chirish	`systemctl stop redis` (barcha node)	Menyu DBdan (p95 < 400 ms). WS uziladi → polling. Buyurtma **ishlaydi**	Buyurtma yaratish 500 bersa
CH-4	Barcha WS node o'chirish	`systemctl stop armenu-ws`	KDS 10 s polling'ga o'tadi. Buyurtma, to'lov ishlaydi. Klient reconnect	KDS buyurtmani ko'rmasa
CH-5	DB primary o'chirish	`patronictl switchover` / `systemctl stop postgresql`	30–60 s yozish yo'q (503 + "qayta urinib ko'ring"). **O'qish uzilmaydi**. Failover'dan keyin normal	Failover 90 s dan uzun yoki ma'lumot yo'qolsa
CH-6	Replica lag +60 s	`SELECT pg_wal_replay_pause()` replica'da	Healer 30 s da aniqlaydi → o'qish primary'ga. Eskirgan ma'lumot ko'rsatilmaydi	Mijoz eski menyu/narx ko'rsa
CH-7	Tashqi API bloklash	`iptables -A OUTPUT -d <payme_ip> -j DROP`	CB 3 xatodan keyin OPEN. UI'da Payme yashiriladi, Click ko'rinadi. 8 s dan uzun kutish yo'q	Foydalanuvchi 30 s spinner ko'rsa
CH-8	Tarmoq kechikishi +2 s	`tc qdisc add dev eth0 root netem delay 2000ms` (DBga)	`statement_timeout` 10 s ishlaydi. p95 oshadi lekin xato bermaydi. CB tashqi APIda OPEN	Kaskadli yiqilish (barcha worker band)
CH-9[1]	Disk 95% to'ldirish	`fallocate -l <N>G /srv/armenu/chaos.bin`	Healer 10 daq da aniqlaydi → cleanup. 95% da read-only rejim + P1	Cleanup ishlamasa yoki DB corrupt bo'lsa
CH-10[1]	⚡ Yadro yo'l testi	Redis + WS + barcha worker + S3 + AI **birgalikda** o'chiriladi	**Buyurtma yaratish ishlaydi** (p95 < 500 ms). Menyu ko'rinadi	Buyurtma yaratilmasa — 🔴 arxitektura xatosi

[1] *CH-9 va CH-10 — chorakda 1 marta, faqat staging'da.*

10.4 Avtomatlashtirish

```
# core/chaos/experiments.py
"""
Chaos eksperiment freymvorki.
Ishga tushirish:  manage.py chaos run CH-3 --duration=180 --env=staging
Abort:            manage.py chaos abort
"""
from dataclasses import dataclass, field
from typing import Callable
import time, logging

logger = logging.getLogger('armenu.chaos')


@dataclass
class Experiment:
    id: str
    name: str
    hypothesis: str                      # nima bo'lishini kutamiz
    inject: Callable[[], None]           # nosozlikni kiritish
    rollback: Callable[[], None]         # tiklash
    verify: Callable[[], dict]           # SLO buzilmadimi?
    max_duration: int = 300
    allowed_envs: tuple = ('staging',)
    blast_radius: str = '1 node'


def run(exp: Experiment, duration: int, env: str, dry_run=False):
    assert env in exp.allowed_envs, f"{exp.id} {env} muhitida ruxsat etilmagan"
    baseline = exp.verify()
    logger.warning("[CHAOS] %s boshlandi. Gipoteza: %s", exp.id, exp.hypothesis)
    notify_telegram(f"🧨 CHAOS {exp.id} boshlandi ({duration}s) — {exp.name}")

    if dry_run:
        return {'dry_run': True, 'baseline': baseline}

    results = []
    try:
        exp.inject()
        deadline = time.monotonic() + min(duration, exp.max_duration)
        while time.monotonic() < deadline:
            m = exp.verify()
            results.append(m)
            # ⚡ Avtomatik abort: SLO qattiq buzilsa
            if m['error_rate'] > 0.05 or m['core_path_ok'] is False:
                logger.error("[CHAOS] SLO buzildi — AVTO ABORT")
                break
            time.sleep(10)
    finally:
        exp.rollback()
        notify_telegram(f"✅ CHAOS {exp.id} tugadi, tizim tiklandi")

    return {
        'experiment': exp.id,
        'baseline': baseline,
        'during': results,
        'after': exp.verify(),
        'hypothesis_held': all(r['core_path_ok'] for r in results),
    }
```

```
# core/chaos/catalog.py — CH-3 namunasi
CH3 = Experiment(
    id='CH-3',
    name='Redis to\'liq o\'chirish',
    hypothesis="Menyu DB'dan p95<400ms keladi, buyurtma ishlaydi, WS polling'ga o'tadi",
    inject=lambda: run_ssh_all('redis', 'systemctl stop redis'),
    rollback=lambda: run_ssh_all('redis', 'systemctl start redis'),
    verify=lambda: {
        'error_rate':   prom_query('rate(armenu_http_requests_total{status=~"5.."}[1m])'),
        'menu_p95':     prom_query('histogram_quantile(0.95, armenu_http_request_duration_seconds{endpoint="menu"})'),
        'core_path_ok': synthetic_order_test(),      # ⚡ haqiqiy buyurtma yaratib ko'radi
    },
    max_duration=180,
    allowed_envs=('staging', 'production'),
    blast_radius='butun kesh qatlami',
)
```

GitHub Actions (oylik jadval):

```
# .github/workflows/chaos.yml
name: Chaos Engineering
on:
  schedule:
    - cron: '0 6 * * 2'        # har seshanba 11:00 (UZT)
  workflow_dispatch:
    inputs:
      experiment: {description: 'CH-1..CH-10', required: true}

jobs:
  chaos:
    runs-on: [self-hosted, staging]
    steps:
      - name: Error budget tekshiruvi
        run: |
          BUDGET=$(curl -s "$PROM/api/v1/query?query=armenu:order_error_budget_remaining" \
                   | jq -r '.data.result[0].value[1]')
          # ⚡ Byudjet 25% dan kam bo'lsa chaos test O'TKAZILMAYDI
          awk -v b="$BUDGET" 'BEGIN{exit !(b>0.25)}' || { echo "Byudjet past"; exit 1; }
      - name: Eksperiment
        run: python manage.py chaos run "${{ inputs.experiment || 'CH-1' }}" --env=staging
      - name: Hisobot
        run: python manage.py chaos report --last --telegram
```

---

11. FALOKATDAN TIKLASH (DR)

11.1 RPO / RTO jadvali

> **RPO** (Recovery Point Objective) — qancha ma'lumot yo'qolishi mumkin
> **RTO** (Recovery Time Objective) — qancha vaqtda tiklanishi kerak

Ma'lumot turi	RPO	RTO	Qanday erishiladi
🔴 **To'lov tranzaksiyasi**	**0**	5 daq	Sinxron replikatsiya (`synchronous_commit=on`, 1 replica)
🔴 **Buyurtma**	**0**	5 daq	Sinxron replikatsiya
🔴 **Fiskal chek**	**0**	15 daq	Sinxron + OFD tomonida ham saqlanadi
🟠 **Menyu / taom / narx**	5 daq	15 daq	WAL streaming + kunlik full
🟠 **Restoran / xodim / stol**	5 daq	15 daq	WAL streaming
🟠 **AuditLog**	5 daq	1 soat	WAL streaming (compliance uchun muhim)
🟡 **3D fayl / rasm**	1 soat	1 soat	S3 cross-region replication + versioning
🟡 **Sessiya / savat**	6 soat	30 daq	Redis AOF + RDB (yo'qolsa — mijoz qayta to'ldiradi)
🟢 **Analitika event**	1 soat	4 soat	Kunlik dump; qayta hisoblash mumkin
🟢 **Kesh**	∞	0	Tiklanmaydi — o'zi to'ladi (warm-up)
🟢 **Log**	1 kun	24 soat	Kunlik S3 arxiv

11.2 Backup strategiyasi

#	Nima	Qachon	Qayerda	Saqlash	Shifrlash	Tekshiruv
1	**PG full** (`pg_basebackup`)	Kunlik 03:00 UZT	S3 `armenu-backup-tas` + S3 `armenu-backup-eu`	30 kun	AES-256 (age)	Kunlik 04:00 avto-restore
2	**PG WAL** (arxiv)	Uzluksiz (`archive_command`)	S3 `armenu-backup-tas`	7 kun	AES-256	PITR sinovi haftalik
3	**PG logical dump** (`pg_dump -Fc`)	Haftalik yakshanba 02:00	S3 + lokal NAS	90 kun	AES-256	Oylik
4	**Redis RDB + AOF**	RDB 6 soatda, AOF `everysec`	S3	3 kun	AES-256	Haftalik
5	**S3 media (3D, rasm)**	Real-time CRR	S3 `armenu-media-eu`	Doimiy + versioning 90 kun	SSE-S3	Oylik namuna
6	**Kod**	Har commit	GitHub + lokal mirror (gitea)	Doimiy	—	CI har build
7	**Konfiguratsiya** (`.env`, nginx, systemd)	Har o'zgarish	Git (`armenu-infra`, SOPS bilan shifrlangan)	Doimiy	SOPS + age	Deploy'da
8	**Sirlar** (Vault/env)	Har rotatsiya	Offline (2 nusxa, seyfda)	Doimiy	GPG	Choraklik
9	**Prometheus metrikalar**	Uzluksiz	Thanos → S3	1 yil	SSE-S3	—
10	**AuditLog eksport**	Kunlik 05:00	S3 (WORM, o'chirilmaydigan)	5 yil (qonun)	AES-256	Choraklik

3-2-1 qoidasi: 3 nusxa · 2 xil turdagi saqlagichda · 1 tasi boshqa geografiyada ✅

```
#!/usr/bin/env bash
# ops/backup/pg_full.sh — kunlik to'liq backup
set -euo pipefail
TS=$(date -u +%Y%m%dT%H%M%SZ)
DIR=/srv/backup/pg/$TS
KEY=/etc/armenu/backup-age.pub

mkdir -p "$DIR"
# 1) Konsistent snapshot
pg_basebackup -h pg-primary -U replicator -D "$DIR" -Ft -z -Xs -P -c fast

# 2) Shifrlash
for f in "$DIR"/*.tar.gz; do
  age -R "$KEY" -o "$f.age" "$f" && rm "$f"
done

# 3) Ikki regionga yuklash
aws s3 sync "$DIR" "s3://armenu-backup-tas/pg/$TS/" --storage-class STANDARD_IA
aws s3 sync "$DIR" "s3://armenu-backup-eu/pg/$TS/"  --storage-class GLACIER_IR

# 4) Manifest (healer #15 shu faylni o'qiydi)
cat > /tmp/manifest.json <<EOF
{"kind":"pg_full","created_at":"$TS","size_bytes":$(du -sb "$DIR" | cut -f1),
 "pg_version":"$(psql -h pg-primary -tAc 'SHOW server_version')","verified_at":null}
EOF
aws s3 cp /tmp/manifest.json "s3://armenu-backup-tas/pg/latest/manifest.json"

# 5) Eskisini tozalash
find /srv/backup/pg -maxdepth 1 -type d -mtime +3 -exec rm -rf {} +
echo "✅ Backup tugadi: $TS"
```

11.3 Tiklash runbook (bosqichma-bosqich)

🔴 Ssenariy A — PG primary yiqildi (auto-failover ishlamadi)

```
┌─ 0 daq ──────────────────────────────────────────────────┐
│ ALERT: "PostgreSQL primary down" (P1) → Telegram + qo'ng'iroq   │
└──────────────────────────┬───────────────────────────────┘
                           ▼
[1] TASDIQLASH (2 daq)
    $ patronictl -c /etc/patroni.yml list
    $ pg_isready -h pg-primary
    → primary haqiqatan o'chganmi? tarmoq muammosimi?

[2] READ-ONLY REJIMNI YOQISH (1 daq)
    $ redis-cli SET armenu:readonly_mode 1 EX 3600
    → mijoz menyuni ko'radi, "buyurtma vaqtincha mavjud emas"

[3] QO'LDA FAILOVER (3 daq)
    $ patronictl -c /etc/patroni.yml failover --candidate pg-replica-1
    # Patroni ishlamasa:
    $ ssh pg-replica-1 'pg_ctl promote -D /var/lib/postgresql/16/main'
    $ ssh pg-replica-1 'psql -c "SELECT pg_is_in_recovery();"'   # false bo'lishi kerak

[4] MARSHRUTNI O'ZGARTIRISH (2 daq)
    $ ansible-playbook ops/playbooks/repoint_db.yml -e new_primary=pg-replica-1
    # PgBouncer + Django settings yangilanadi, reload
    $ systemctl reload pgbouncer && systemctl reload armenu

[5] TEKSHIRISH (2 daq)
    $ curl -s https://armenu.uz/healthz | jq
    $ python manage.py smoke_test --core-path      # buyurtma yaratib ko'radi
    $ redis-cli DEL armenu:readonly_mode

[6] YANGI REPLICA (fon, 30–60 daq)
    $ ansible-playbook ops/playbooks/rebuild_replica.yml -e host=pg-old-primary

⏱ RTO: ~10 daqiqa (qo'lda) / 30–60 soniya (Patroni avto)
```

🟠 Ssenariy B — Butun DB yo'qoldi (primary + barcha replica)

```
[1] E'LON (2 daq)
    → Status sahifa: "Texnik ishlar" · Telegram: barcha restoran adminlariga
    → nginx: `/snapshot/` rejimiga o'tkazish (statik menyu)
    $ ansible-playbook ops/playbooks/static_mode.yml

[2] ENG YANGI BACKUPNI TOPISH (3 daq)
    $ aws s3 ls s3://armenu-backup-tas/pg/ --recursive | tail -5
    $ aws s3 cp s3://armenu-backup-tas/pg/latest/manifest.json - | jq

[3] YANGI DB NODE (5 daq — Terraform tayyor)
    $ cd ops/terraform && terraform apply -target=module.pg_recovery -auto-approve

[4] BASE BACKUP TIKLASH (15–25 daq, hajmga qarab)
    $ aws s3 sync s3://armenu-backup-tas/pg/<TS>/ /restore/
    $ for f in /restore/*.age; do age -d -i /etc/armenu/backup-age.key -o "${f%.age}" "$f"; done
    $ tar -xzf /restore/base.tar.gz -C /var/lib/postgresql/16/main

[5] ⚡ PITR — WAL bilan eng oxirgi nuqtaga (10 daq)
    $ cat > /var/lib/postgresql/16/main/postgresql.auto.conf <<EOF
    restore_command = 'aws s3 cp s3://armenu-backup-tas/wal/%f %p'
    recovery_target_time = '<falokatdan 1 daqiqa oldin>'
    recovery_target_action = 'promote'
    EOF
    $ touch /var/lib/postgresql/16/main/recovery.signal
    $ systemctl start postgresql
    $ tail -f /var/log/postgresql/*.log      # "consistent recovery state reached"

[6] BUTUNLIKNI TEKSHIRISH (5 daq)
    $ python manage.py dr_verify
      ✓ Oxirgi buyurtma vaqti
      ✓ Oxirgi to'lov vaqti
      ✓ To'lov summasi = provayder hisoboti (Payme/Click API)
      ✓ Fiskal cheklar = OFD hisoboti
      ✓ Restoran soni, taom soni
      ✓ Cross-tenant sanity: `SELECT count(DISTINCT restaurant_id) FROM orders`

[7] XIZMATNI QAYTARISH (5 daq)
    $ ansible-playbook ops/playbooks/repoint_db.yml -e new_primary=pg-recovery-1
    $ systemctl reload armenu armenu-ws
    $ systemctl start armenu-worker@{fiscal,notify,ai,model3d,analytics}
    $ redis-cli DEL armenu:readonly_mode
    $ python manage.py smoke_test --full

[8] YO'QOLGAN OYNANI QOPLASH (30 daq)
    → RPO oynasidagi to'lovlar: provayder API'dan reconcile
    $ python manage.py reconcile_payments --since='<falokat vaqti>' --all-providers
    → Yo'qolgan cheklar: OFD'dan qayta so'rash
    $ python manage.py reconcile_fiscal --since='<falokat vaqti>'

[9] POST-MORTEM (72 soat ichida)

⏱ RTO: 45–70 daqiqa · RPO: ≤ 5 daqiqa (WAL bilan) / 0 (sinxron replica omon qolsa)
```

🔴 Ssenariy C — Butun region yo'qoldi

```
[1] DNS failover (5 daq)     → Cloudflare: origin'ni EU regionga o'tkazish
[2] EU'da stack ko'tarish    → terraform apply -var region=eu-central
[3] Backup tiklash           → Ssenariy B [4]-[6] (EU bucket'dan, tezroq)
[4] Media                    → S3 CRR allaqachon EU'da ✓
[5] To'lov/OFD               → ⚠ UZ provayderlari IP whitelist so'rashi mumkin
                               → oldindan EU IP'lar ro'yxatga olingan bo'lishi shart
⏱ RTO: 3–4 soat · RPO: ≤ 15 daqiqa
```

11.4 Choraklik DR sinovi

Bosqich	Kim	Vaqt	Muvaffaqiyat mezoni
1. Rejalashtirish	SRE	−7 kun	Ssenariy tanlandi (A/B/C rotatsiya), jamoa xabardor
2. Izolyatsiya	DevOps	−1 kun	Alohida VPC/hisob, production'ga tegmaydi
3. **Ijro**	Navbatchi (🚶 *yangi odam har chorak*)	0	Runbookdan **chetga chiqmasdan** tiklash
4. O'lchov	SRE	+0	Haqiqiy RTO ≤ maqsad × 1,5 · RPO = maqsad
5. Butunlik	Backend	+1 soat	`dr_verify` 100% o'tadi
6. Hisobot	SRE	+2 kun	Runbook yangilandi, farqlar → ticket

Choraklik sinov jurnali (namuna):

Chorak	Ssenariy	Maqsad RTO	Haqiqiy RTO	RPO	Natija	Topilgan muammo
2026-Q1	A (failover)	10 daq	8 daq	0	✅	—
2026-Q2	B (to'liq)	60 daq	94 daq	3 daq	🟡	`age` kaliti dokumentda yo'q edi → qo'shildi
2026-Q3	C (region)	240 daq	—	—	⏳	Rejada

---

12. ISHONCHLILIK NAQSHLARI

12.1 Idempotency

> **Muammo:** klient 2 marta bosdi / tarmoq uzildi va retry bo'ldi / webhook 2× keldi
> → 2 ta buyurtma, 2 ta to'lov, 2 ta chek.




```python
# core/idempotency.py
import json, hashlib
from django.db import IntegrityError, transaction
from django.core.cache import caches

_r = caches['default'].client.get_client(write=True)
TTL = 86400


class IdempotencyConflict(Exception):
    """Bir xil kalit, boshqa payload — klient xatosi."""


def idempotent(request, key_header='X-Idempotency-Key', ttl=TTL):
    """Kontekst menejer: (cached_response, save_fn) qaytaradi."""
    key = request.headers.get(key_header)
    if not key:
        return None, lambda resp: None

    # ⚡ Kalit tenant bilan chegaralanadi — cross-tenant to'qnashuv bo'lmasin
    rk = f'idem:{request.restaurant_id}:{key}'
    body_hash = hashlib.sha256(request.body).hexdigest()[:16]

    stored = _r.get(rk)
    if stored:
        data = json.loads(stored)
        if data.get('_body') != body_hash:
            raise IdempotencyConflict("Bir xil kalit, boshqa ma'lumot")
        if data.get('_status') == 'in_progress':
            raise IdempotencyConflict("So'rov hali bajarilmoqda")
        return data['_response'], lambda resp: None

    # Lock: bir vaqtda 2 ta bir xil so'rov kelmasin
    if not _r.set(rk, json.dumps({'_status': 'in_progress', '_body': body_hash}),
                  nx=True, ex=60):
        raise IdempotencyConflict("So'rov hali bajarilmoqda")

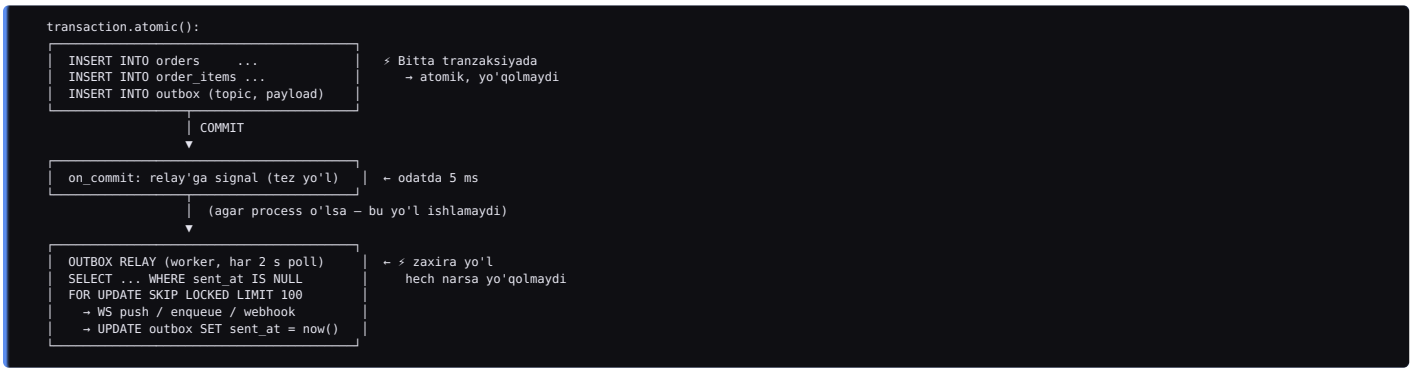
    def save(response_data):
        _r.set(rk, json.dumps({'_status': 'done', '_body': body_hash,
                               '_response': response_data}), ex=ttl)
    return None, save
```

```python
# apps/orders/models.py — ikkinchi mudofaa chizig'i (DB darajasi)
class Order(models.Model):
    restaurant = models.ForeignKey('tenants.Restaurant', on_delete=models.PROTECT)
    idempotency_key = models.CharField(max_length=64, null=True, blank=True)
    ...
    class Meta:
        constraints = [
            # ⚡ Redis yiqilsa ham dublikat bo'lmaydi
            models.UniqueConstraint(
                fields=['restaurant', 'idempotency_key'],
                condition=models.Q(idempotency_key__isnull=False),
                name='uniq_order_idem_per_restaurant',
            ),
        ]
```

Operatsiya	Idempotency kaliti	Manbasi	TTL
Buyurtma yaratish	`X-Idempotency-Key`	Klient (UUIDv4, savat sessiyasidan)	24 soat
To'lov boshlash	`order_id + amount`	Server	24 soat
To'lov webhook	`provider_txn_id`	Provayder	∞ (DB unique)
Fiskal chek	`receipt_uuid`	Server (UUIDv4)	∞ (DB unique)
Telegram xabar	`notification_id`	Server	1 soat
Ofitsiant chaqirish	`table_id + minute`	Server (1 daq oynada 1 ta)	60 s
3D generatsiya	`dish_id + source_hash`	Server	∞

12.2 Outbox pattern

> **Muammo:** `transaction.atomic()` ichida WS yuborsak va tranzaksiya rollback bo'lsa — oshxona yo'q buyurtmani ko'radi. `on_commit`'da yuborsak va process o'lsa — xabar butunlay yo'qoladi.



```
# core/outbox/models.py
class OutboxEvent(models.Model):
    id           = models.BigAutoField(primary_key=True)
    restaurant_id= models.IntegerField(db_index=True)     # ⚡ tenant chegarasi
    topic        = models.CharField(max_length=64)        # 'order.created'
    payload      = models.JSONField()
    created_at   = models.DateTimeField(auto_now_add=True)
    sent_at      = models.DateTimeField(null=True, db_index=True)
    attempts     = models.SmallIntegerField(default=0)
    last_error   = models.TextField(blank=True)

    class Meta:
        indexes = [
            models.Index(fields=['sent_at', 'id'], name='outbox_pending_idx',
                         condition=models.Q(sent_at__isnull=True)),
        ]


# core/outbox/publisher.py
def publish(topic, payload, restaurant_id):
    """⚠ FAQAT transaction.atomic() ichida chaqiriladi."""
    assert transaction.get_connection().in_atomic_block, "Outbox tranzaksiya ichida bo'lishi shart"
    ev = OutboxEvent.objects.create(topic=topic, payload=payload,
                                    restaurant_id=restaurant_id)
    transaction.on_commit(lambda: _nudge_relay(ev.id))    # tez yo'l
    return ev


# core/outbox/relay.py — worker
HANDLERS = {
    'order.created':   handle_order_created,      # WS + notify + analytics
    'order.status':    handle_order_status,
    'payment.paid':    handle_payment_paid,       # fiscal + WS + Telegram
    'dish.updated':    handle_dish_updated,       # kesh invalidatsiya + snapshot
}

def relay_loop():
    while True:
        with transaction.atomic():
            batch = list(OutboxEvent.objects
                         .select_for_update(skip_locked=True)
                         .filter(sent_at__isnull=True, attempts__lt=10)
                         .order_by('id')[:100])
            for ev in batch:
                try:
                    HANDLERS[ev.topic](ev.payload, ev.restaurant_id)
                    ev.sent_at = timezone.now()
                except Exception as e:
                    ev.attempts += 1
                    ev.last_error = str(e)[:500]
                    if ev.attempts >= 10:
                        move_to_dlq(ev)
                OutboxEvent.objects.bulk_update(
                    batch, ['sent_at', 'attempts', 'last_error'])
        time.sleep(2 if not batch else 0)
```

Tozalash: `sent_at` to'ldirilgan yozuvlar 7 kundan keyin partition drop bilan o'chiriladi.

12.3 Saga — to'lov + fiskal

> **Muammo:** to'lov o'tdi, lekin fiskal chek yaratilmadi. Yoki teskarisi.
> Ikki tashqi tizim — distributed tranzaksiya yo'q. **Yechim: saga + kompensatsiya.**

```
  BOSQICH              MUVAFFAQIYAT                 ⛔ XATO → KOMPENSATSIYA
  ──────────────────────────────────────────────────────────────────────────
  S1  Buyurtma lock    order.status = 'paying'       → status = 'confirmed' (unlock)
      │
      ▼
  S2  To'lov yaratish  PaymentTransaction(pending)   → txn.status = 'cancelled'
      │                 + provider.create()
      ▼
  S3  To'lov tasdiq    webhook / poll → 'paid'       → provider.cancel() (refund)
      │                                                ⚠ pul qaytariladi
      ▼
  S4  Fiskal chek      OFD → receipt_id              → ⚡ KOMPENSATSIYA QILINMAYDI
      │                                                pul qaytarilmaydi!
      │                                                chek NAVBATGA tushadi
      ▼
  S5  Yakunlash        order.status = 'paid'         → qo'lda ko'rib chiqish
                       table.status = 'free'
                       WS + Telegram + chek QR
```

```python
# apps/payments/saga.py
from enum import Enum

class Step(str, Enum):
    LOCK_ORDER   = 'lock_order'
    CREATE_TXN   = 'create_txn'
    CONFIRM_PAY  = 'confirm_pay'
    FISCALIZE    = 'fiscalize'
    FINALIZE     = 'finalize'


class PaymentSaga:
    """
    ⚡ Muhim qaror: S4 (fiskal) MUVAFFAQIYATSIZ bo'lsa saga ORQAGA QAYTMAYDI.
    Sabab: mijoz to'lovni amalga oshirgan, pulni qaytarish mijoz uchun yomonroq
    tajriba. Chek navbatga tushadi va 48 soat ichida yuboriladi.
    """

    def __init__(self, order, provider, saga_state: 'SagaState'):
        self.order, self.provider, self.state = order, provider, saga_state

    def execute(self):
        try:
            self._step(Step.LOCK_ORDER,  self._lock_order,  self._unlock_order)
            self._step(Step.CREATE_TXN,  self._create_txn,  self._cancel_txn)
            self._step(Step.CONFIRM_PAY, self._confirm_pay, self._refund)
        except SagaFailure as e:
            self._compensate()
            raise

        # ⚡ S4 dan boshlab kompensatsiya YO'Q — faqat oldinga
        try:
            self._step(Step.FISCALIZE, self._fiscalize, compensation=None)
        except Exception:
            logger.error("[SAGA] Fiskal xato — navbatga qo'yildi, to'lov QAYTARILMAYDI")
            enqueue('fiscal', 'retry_receipt', order_id=self.order.id, delay=300)
            self.state.mark(Step.FISCALIZE, 'deferred')

        self._step(Step.FINALIZE, self._finalize, compensation=None)
        return self.state

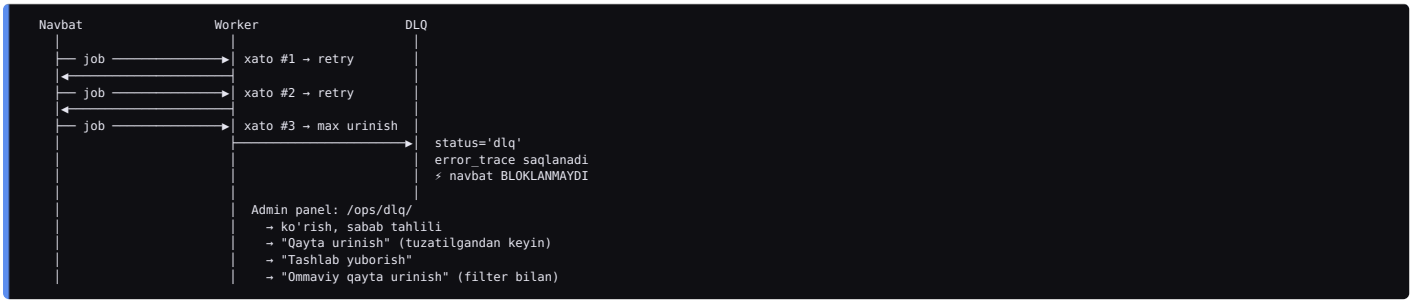
    def _step(self, step, fn, compensation):
        if self.state.is_done(step):
            return self.state.result(step)           # ⚡ idempotent qayta ishga tushirish
        self.state.mark(step, 'running')
        result = fn()
        self.state.mark(step, 'done', result=result, compensation=compensation)
        return result

    def _compensate(self):
        """Teskari tartibda bekor qilish."""
        for step in reversed(self.state.completed_steps()):
            comp = self.state.compensation(step)
            if comp is None:
                continue
            try:
                comp()
                self.state.mark(step, 'compensated')
            except Exception:
                logger.exception("[SAGA] Kompensatsiya xatosi: %s", step)
                alert_oncall(f"🔴 Saga kompensatsiya xatosi: order={self.order.id}, "
                             f"step={step}. QO'LDA TEKSHIRING.", severity='P1')
```

```python
# apps/payments/models.py — saga holati DBda saqlanadi
class SagaState(models.Model):
    """Worker o'lsa — boshqa worker shu joydan davom etadi."""
    order       = models.OneToOneField('orders.Order', on_delete=models.CASCADE)
    restaurant_id = models.IntegerField(db_index=True)
    steps       = models.JSONField(default=dict)     # {step: {status, result, ts}}
    status      = models.CharField(max_length=20, default='running')
    updated_at  = models.DateTimeField(auto_now=True)
```

Healer bog'lanishi: `check_pending_payments` (#14) 5 daqiqada osilgan saga'larni topib `execute()` ni qayta chaqiradi — har qadam idempotent bo'lgani uchun xavfsiz.

12.4 Dead Letter Queue (DLQ)



```
# core/queue/dlq.py
class DeadLetter(models.Model):
    queue         = models.CharField(max_length=32, db_index=True)
    task_name     = models.CharField(max_length=128)
    restaurant_id = models.IntegerField(null=True, db_index=True)
    payload       = models.JSONField()
    attempts      = models.SmallIntegerField()
    error_type    = models.CharField(max_length=128)
    error_message = models.TextField()
    traceback     = models.TextField()
    created_at    = models.DateTimeField(auto_now_add=True, db_index=True)
    resolved_at   = models.DateTimeField(null=True)
    resolution    = models.CharField(max_length=20, blank=True)  # retried|discarded|fixed


def move_to_dlq(job, exc=None):
    DeadLetter.objects.create(
        queue=job.queue, task_name=job.task_name,
        restaurant_id=job.payload.get('restaurant_id'),
        payload=job.payload, attempts=job.attempts,
        error_type=type(exc).__name__ if exc else 'MaxRetriesExceeded',
        error_message=str(exc)[:2000] if exc else '',
        traceback=traceback.format_exc()[:8000] if exc else '',
    )
    metrics.inc('armenu_dlq_total', labels={'queue': job.queue})
    # ⚡ Fiskal va to'lov DLQ'si — darhol P1
    if job.queue in ('fiscal', 'payments'):
        alert_oncall(f"🔴 DLQ: {job.queue}/{job.task_name} — moliyaviy ta'sir!",
                     severity='P1')
```

Navbat	DLQ alert darajasi	Saqlash	Avto-retry
`fiscal`	🔴 P1 (darhol)	90 kun	✅ har 6 soatda 48 soat davomida
`payments`	🔴 P1 (darhol)	90 kun	✅ har 1 soatda 24 soat davomida
`notify`	🟡 P3 (kunlik svodka)	7 kun	⛔ (eskirgan xabar kerak emas)
`ai`	🟢 P4 (haftalik)	7 kun	⛔
`model3d`	🟢 P4 (kunlik svodka)	30 kun	✅ qo'lda tugma bilan
`analytics`	🟢 P4	7 kun	✅ kechasi ommaviy

12.5 Poison message himoyasi

> **Muammo:** bitta buzuq xabar workerni har safar crash qiladi → navbat abadiy bloklanadi → butun `fiscal` navbat to'xtaydi.

```
# core/queue/worker.py
POISON_THRESHOLD = 3          # 3 marta CRASH (xato emas — crash!)


def execute_job(job):
    crash_key = f'poison:{job.queue}:{job.id}'
    crashes = int(_r.get(crash_key) or 0)

    if crashes >= POISON_THRESHOLD:
        logger.error("[POISON] Job %s %d marta crash qildi — karantin", job.id, crashes)
        move_to_dlq(job, PoisonMessage(f'{crashes} crash'))
        _r.delete(crash_key)
        return

    # ⚡ Crash hisoblagichi ISH BOSHLANISHIDAN OLDIN oshiriladi.
    # Worker SIGKILL bo'lsa ham hisob saqlanadi.
    _r.incr(crash_key)
    _r.expire(crash_key, 86400)

    try:
        with time_limit(QUEUE_TIMEOUTS[job.queue]):
            result = TASKS[job.task_name](**job.payload)
    except Exception as exc:
        # Oddiy xato — crash emas, hisoblagichni qaytaramiz
        _r.decr(crash_key)
        raise
    else:
        _r.delete(crash_key)
        return result
```

Qo'shimcha himoyalar:

Himoya	Nima qiladi
Payload hajmi cheki	> 256 KB payload navbatga qabul qilinmaydi (S3 havolasi ishlatiladi)
Sxema validatsiyasi	`enqueue()` da payload sxemaga tekshiriladi — buzuq xabar navbatga tushmaydi
Job timeout	`time_limit()` — cheksiz sikl workerni ushlab turmaydi
Bulkhead	Poison faqat 1 navbatni ta'sirlaydi (§7.1)
`SKIP LOCKED`	Bitta osilgan job boshqa worker'larni bloklamaydi
Karantin ko'rinishi	`/ops/dlq/?error_type=PoisonMessage` — sababni tahlil qilish

---

13. ISHONCHLILIK CHEK-LISTI (kod ko'rigi uchun)

#	Tekshiruv	✅/⛔
1	Har tashqi chaqiruv circuit breaker ichidami?	
2	Har tashqi chaqiruvda `timeout` berilganmi?	
3	Retry faqat idempotent operatsiyadami?	
4	4xx xatolarda retry QILINMAYAPTIMI?	
5	Yozuv operatsiyasida idempotency kaliti bormi?	
6	DB darajasida `UniqueConstraint` zaxira sifatida bormi?	
7	WS/notify `transaction.on_commit` yoki outbox orqalimi?	
8	Uzoq ish (> 2 s) workerga o'tkazilganmi?	
9	Worker job'ining `job_timeout` i bormi?	
10	Fallback yo'li bormi va u **hech qachon xato bermaydimi**?	
11	Yangi tashqi xizmat uchun CB sozlamasi registryga qo'shildimi?	
12	Yangi navbat uchun bulkhead limiti belgilandimi?	
13	Yangi kritik yo'l chaos testiga qo'shildimi?	
14	Yangi ma'lumot turi backup jadvaliga kirdimi (RPO/RTO)?	
15	Alert xabari **nima qilish kerakligini** yozadimi?	

---

14. XULOSA — ISHONCHLILIK QISQACHA

```
┌──────────────────────────────────────────────────────┐
│  ⚡ 5 TA ENG MUHIM QAROR                             │
├──────────────────────────────────────────────────────┤
│  1. YADRO YO'L himoyalangan — Redis, WS, worker, S3, AI, OFD,  │
│     to'lov birgalikda yiqilsa ham buyurtma yaratiladi          │
│                                                      │
│  2. HAR ZANJIR ODAM BILAN TUGAYDI — AR→rasm, AI→ofitsiant,     │
│     to'lov→naqd, fiskal→qo'lda chek. Hech qachon "xato sahifa"  │
│                                                      │
│  3. FISKAL BUYURTMANI BLOKLAMAYDI — chek navbatga tushadi,     │
│     saga S4 dan keyin orqaga qaytmaydi               │
│                                                      │
│  4. 15 TA SELF-HEALING MEXANIZM — tunda odam uyg'otilmaydi;    │
│     tuzata olmasa alert AYNAN NIMA QILISH kerakligini yozadi   │
│                                                      │
│  5. OUTBOX + IDEMPOTENCY — hech bir buyurtma, to'lov, chek     │
│     yo'qolmaydi va ikki marta bajarilmaydi           │
└──────────────────────────────────────────────────────┘
```

Raqamlarda:

Ko'rsatkich	Qiymat
Yadro SLO (buyurtma)	99,90% (8 s 46 daq/yil)
Menyu SLO	99,95% (4 s 23 daq/yil)
Eng yomon RPO (to'lov)	**0**
Eng yomon RTO (to'liq DB)	45–70 daqiqa
Self-healing mexanizmlar	15
Circuit breaker himoyalangan xizmatlar	16
Degradation zanjirlari	7
Izolyatsiyalangan navbatlar (bulkhead)	5
Chaos ssenariylari	10
Backup nusxalari	3 (2 tur, 2 geografiya)
